ORACLE

Java Application Performance Tuning

Don’t guess, measure!

Albert Attard
Principal Java Success Consultant
Java Success Consulting Team
November 2026



Safe harbor statement

The following is intended to outline our general product direction. It is intended for information purposes only, and may not be incorporated into any contract. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described for Oracle's products may change and remains at the sole discretion of Oracle Corporation.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Albert Attard
Principal Java Success
Consultant, Oracle

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



What do you expect to gain from this session about application performance tuning?



Objectives

- Raise awareness about Java application performance tuning
- Demonstrate how to use available Java tools to capture and measure the application performance in an efficient and effective manner

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Mastery takes about 10,000 hours…

Good news: We're not doing all that today.



You are not expected to become an expert after this workshop

but rather to simply identify gaps in your knowledge and begin a conversation



“If there’s something strange, in your Java code, who you gonna call?”

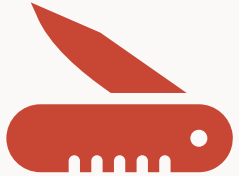
Java Success Team!



Agenda



Goals



Tools



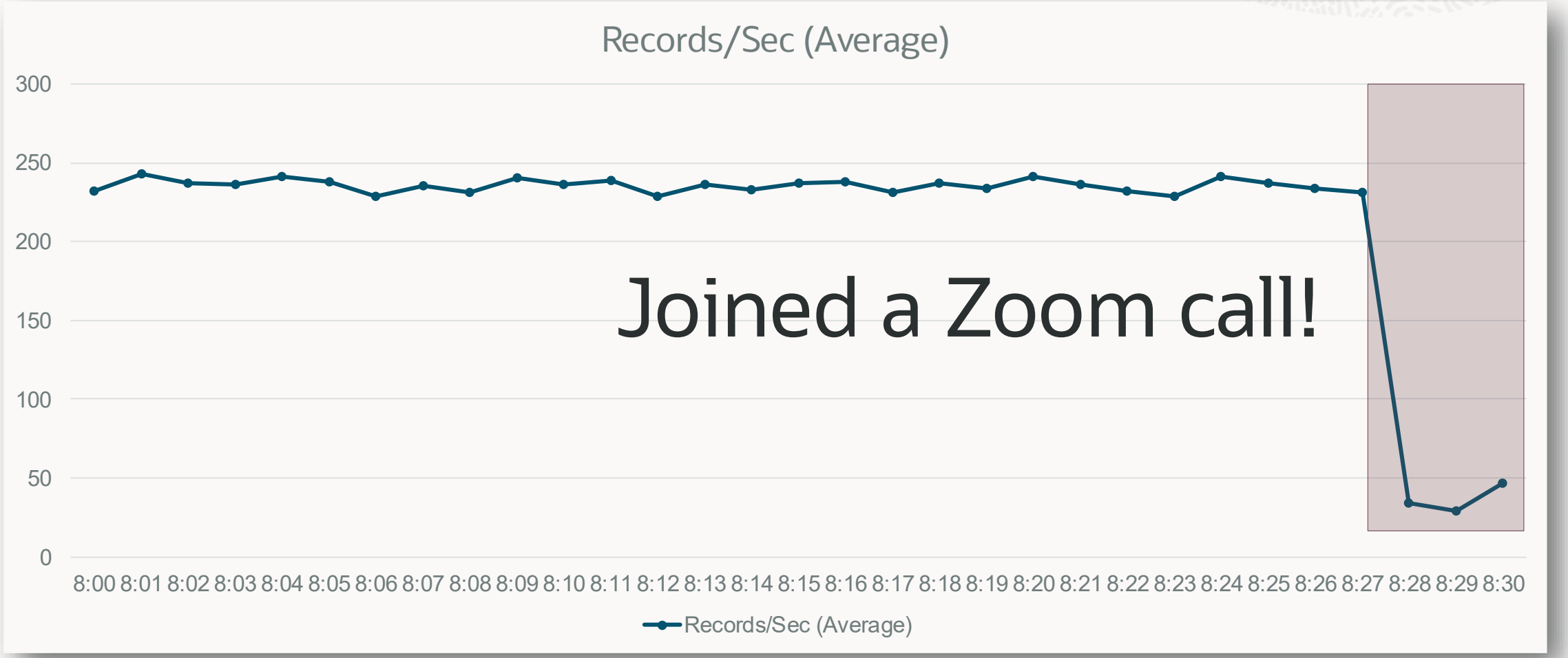
Next?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Disclaimer



Measuring performance



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Performance Disclaimer

- The performance values shown in this workshop are for reference only and should not be treated as absolute
- Results can vary depending on hardware, system configuration, runtime environment, and other factors

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

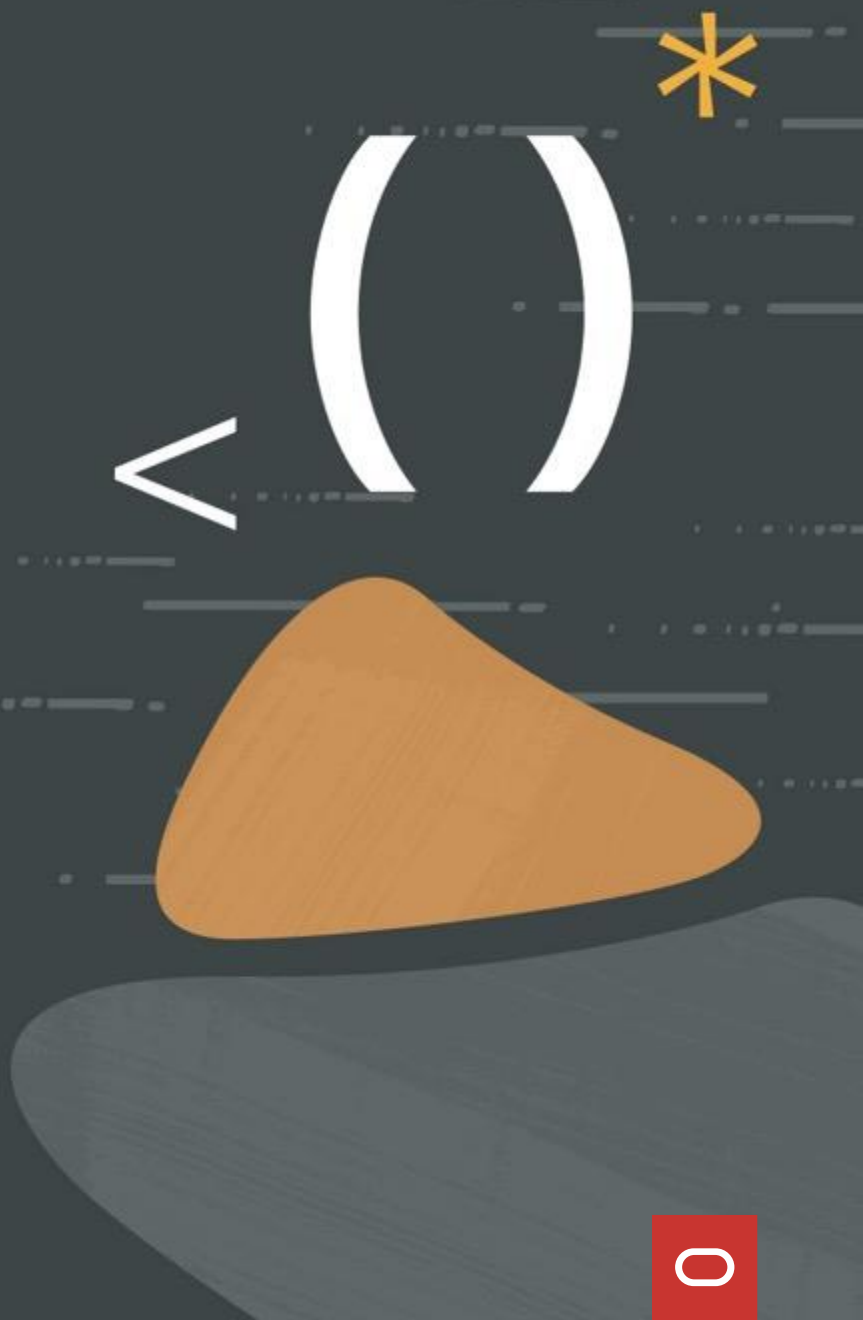
Performance Disclaimer

- Readers are encouraged to conduct their own tests under controlled conditions to obtain accurate measurements relevant to their specific use case

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

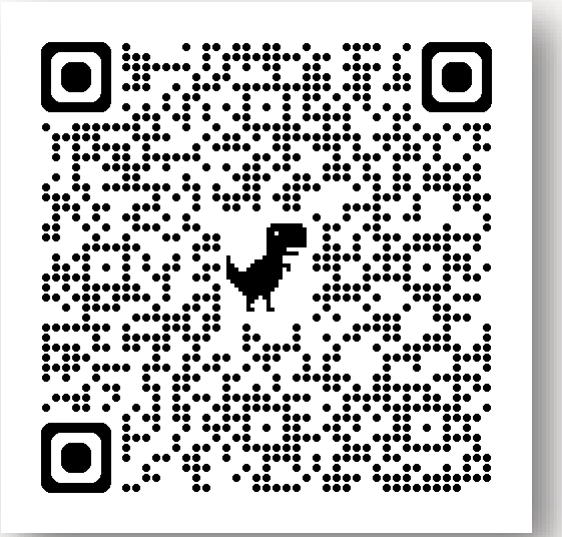
Application Performance Goals





Application Performance Goals

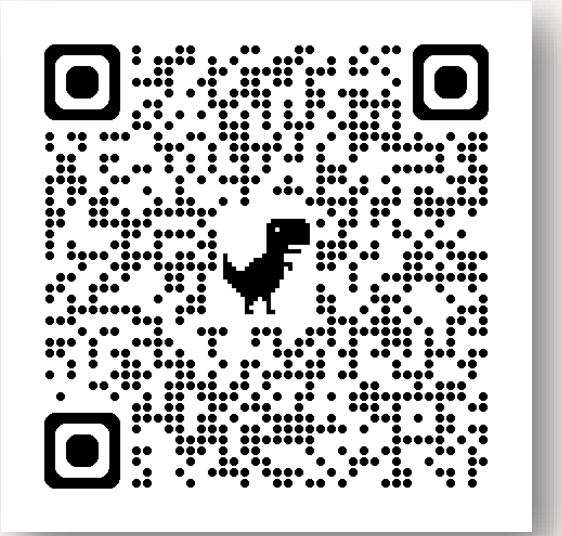
- Google has observed that a 400-millisecond delay results in a 0.44% decrease in search volume



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Application Performance Goals

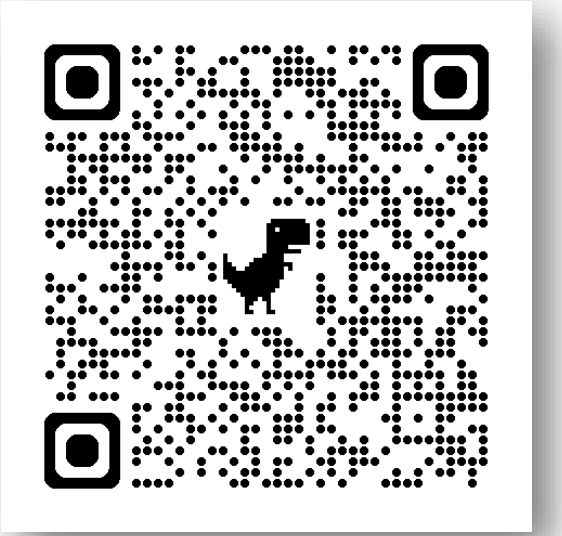
- Amazon found every 100ms of latency cost them 1% in sales



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Application Performance Goals

- Similarly, Walmart observed that for every 1-second improvement in page load time, conversions increased by 2%



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

What do all of these examples have in common?

These examples are measuring the impact of performance against the business goals



Application Performance Goals

- Application performance is often mistaken for the application's goal
- A flight aggregator website experienced the opposite effect when its performance was improved

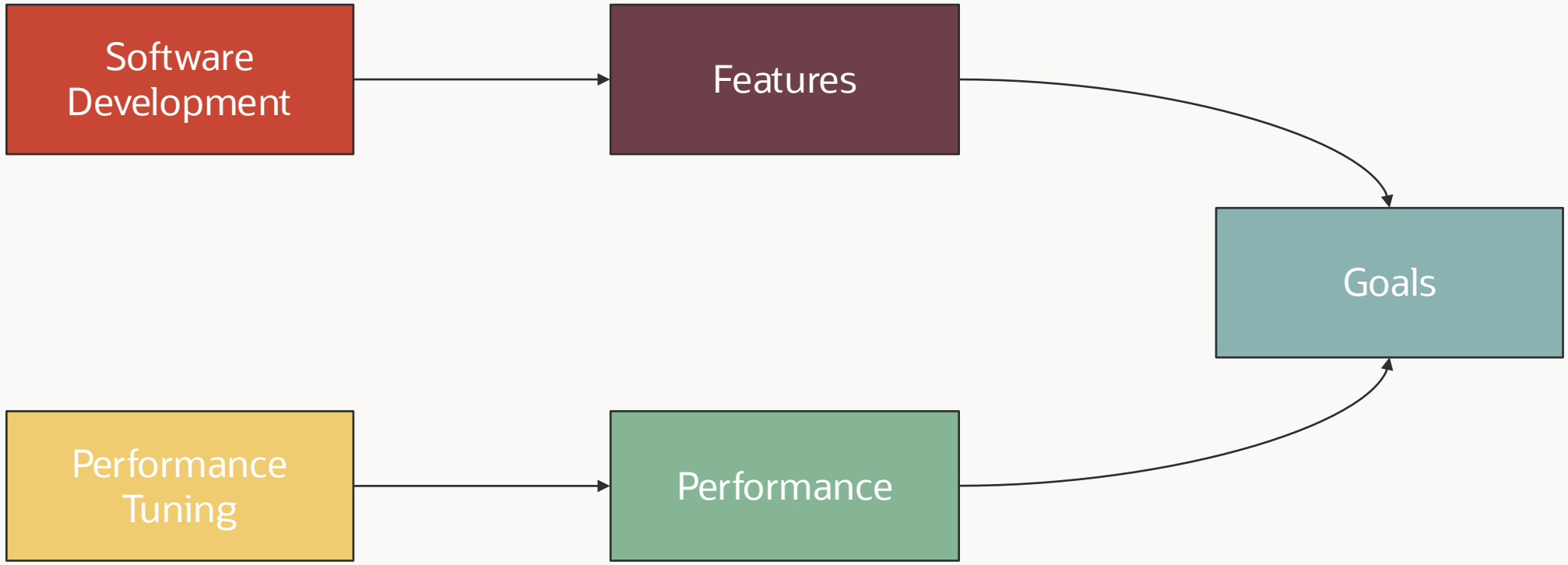
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

What is the purpose of an application?

To enable the business to reach its business goals



Application Performance Goals

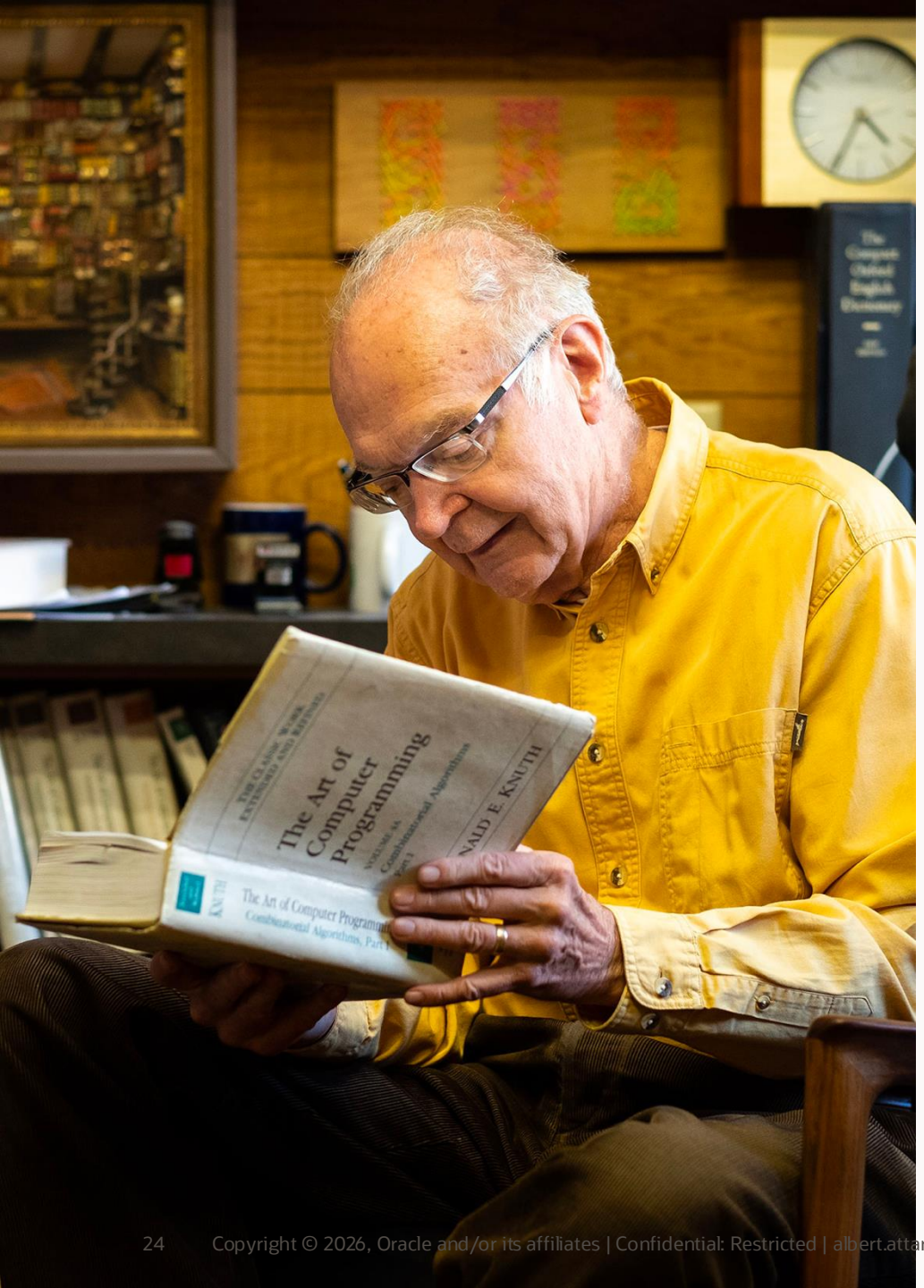


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Why would you want to improve your application's performance?

How will performance improvement help you achieve the application's business objectives?





"*The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming.*"

Donald Knuth

The Art of Computer Programming

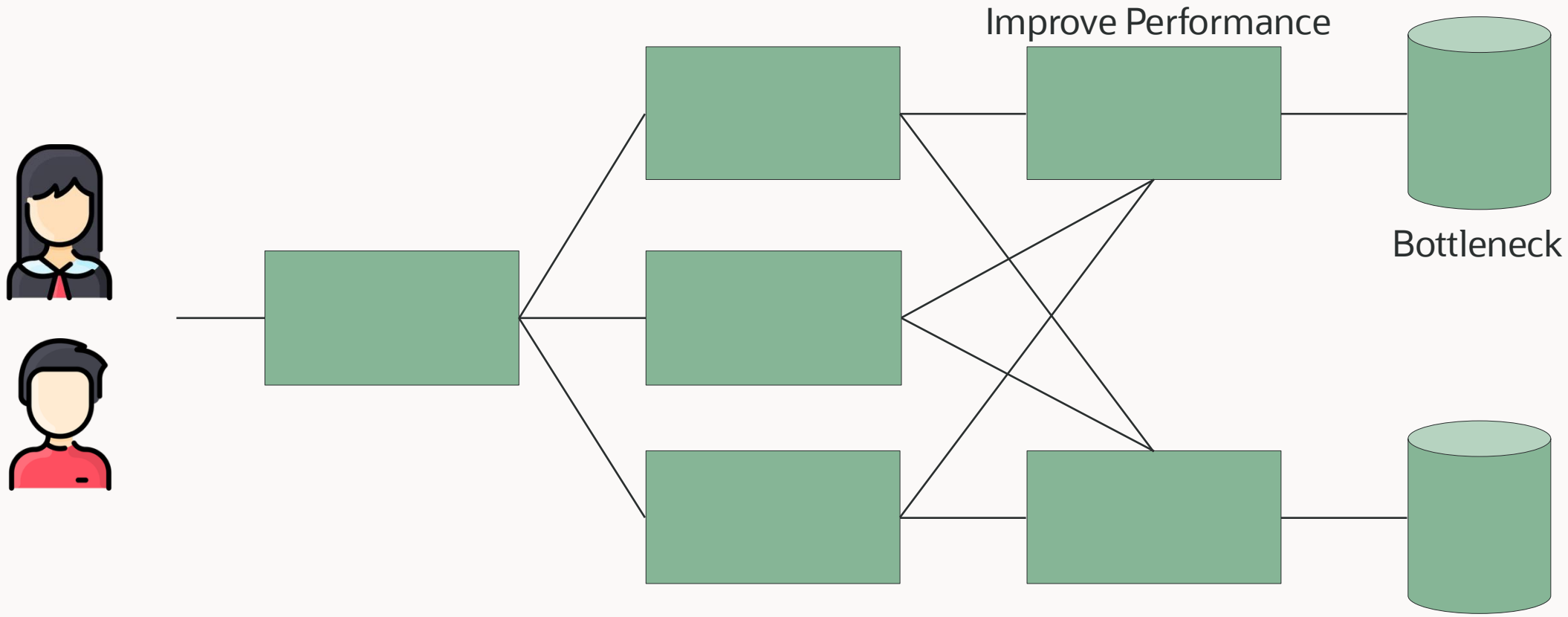
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

What are the potential risks associated with performance enhancements?

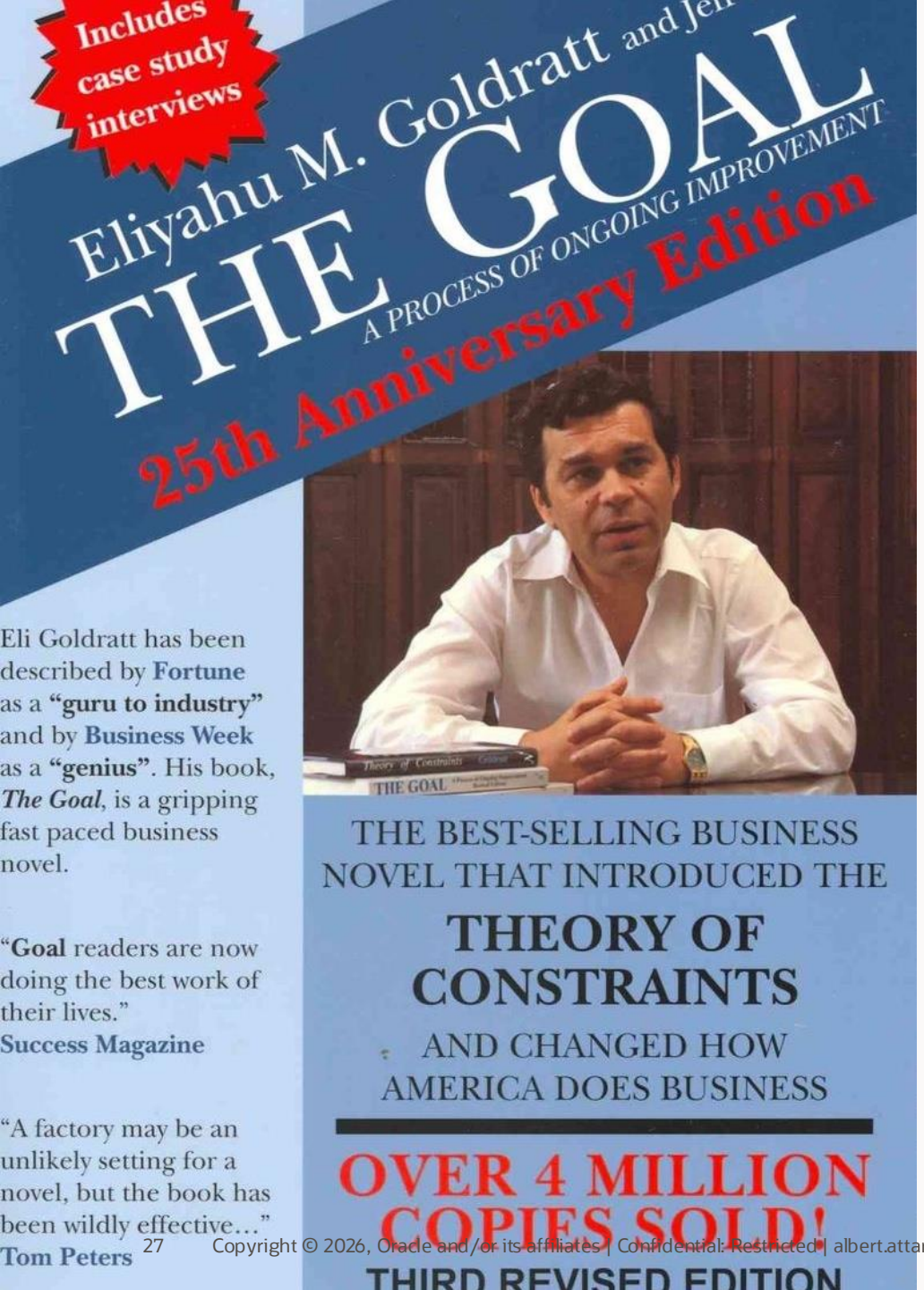


Theory of Constraints

Pressure Points



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



"*Optimizing anything other than the constraint is a waste*"

Eliyahu M. Goldratt

The Goal: A Process of Ongoing Improvement

Theory of Constraints

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Theory of Constraints

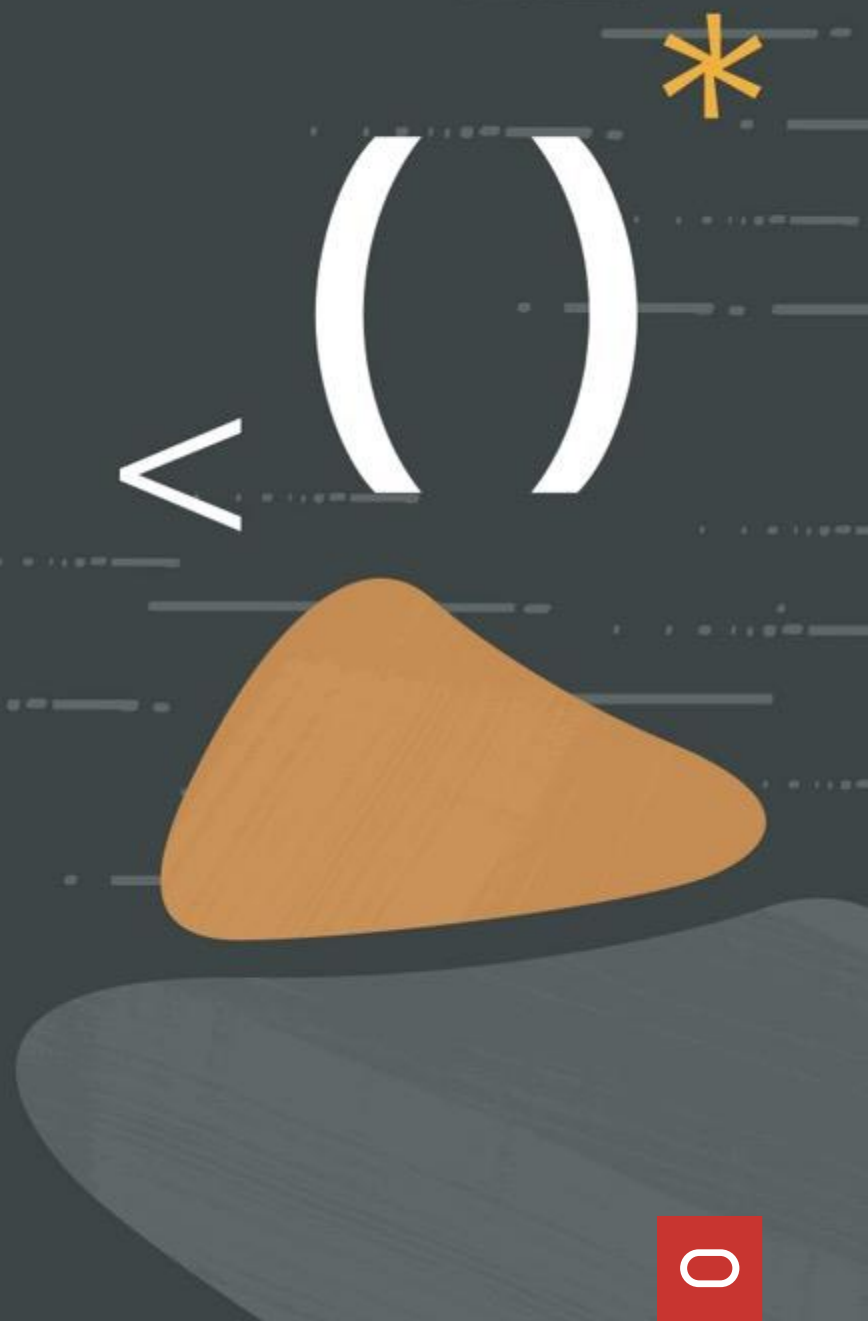
The five focusing steps

- Identify the system constraint
- Focus on the system constraint
- Subordinate everything else
- Elevate the system constraint
- If in the previous steps a constraint has been broken go back to step one

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

“Make this one simple fix to improve your application performance by tenfold”

There is no such thing!!

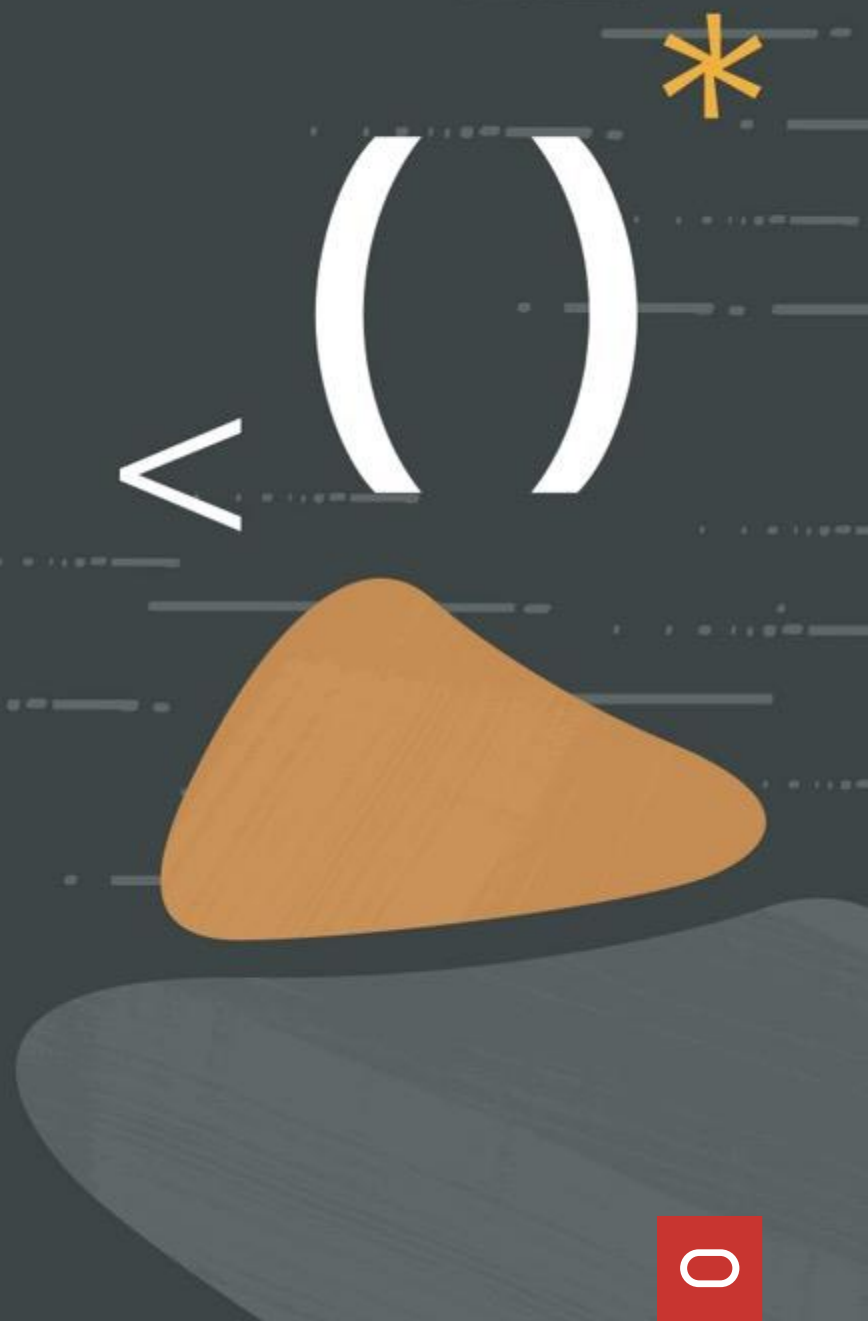


Application performance tuning is not something that you can Google!

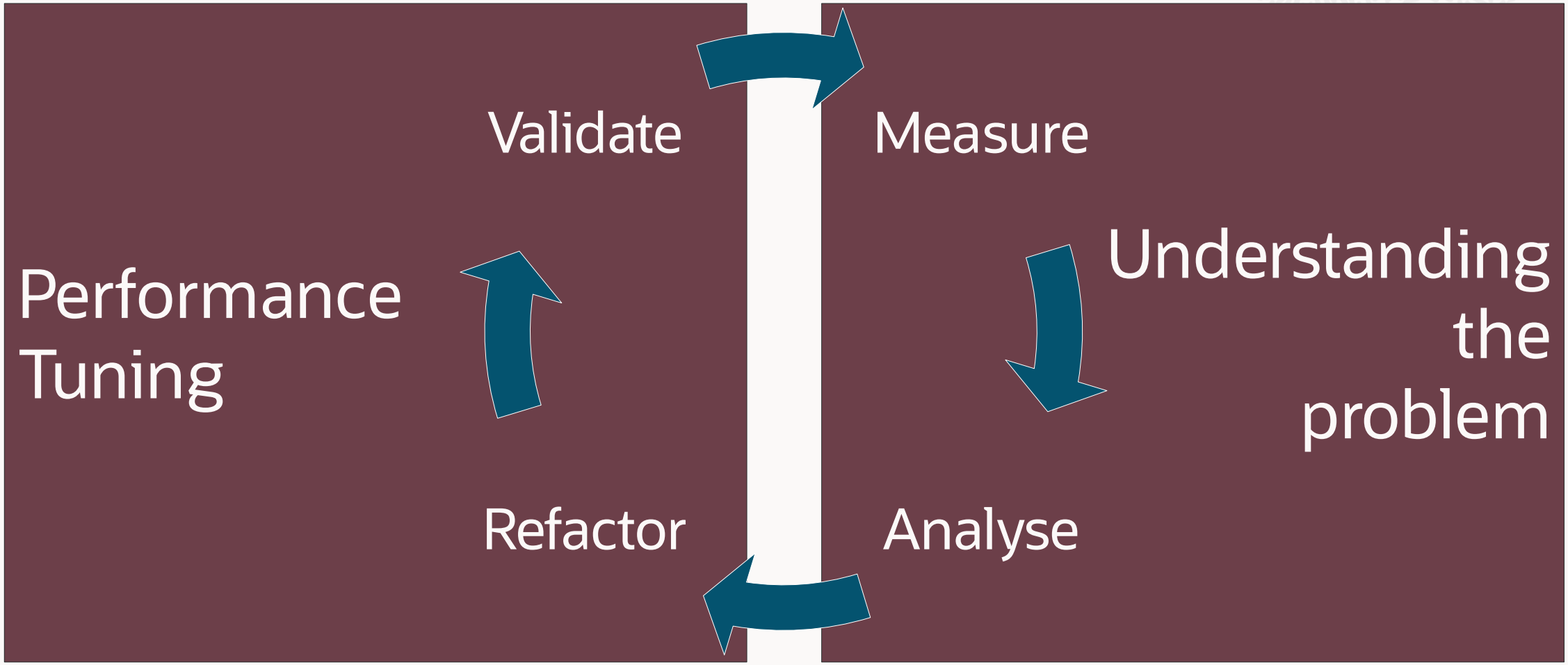
Nor ask ChatGPT, Gemini, or your preferred GenAI tool.



Application performance tuning is not the goal but a solution to a problem!



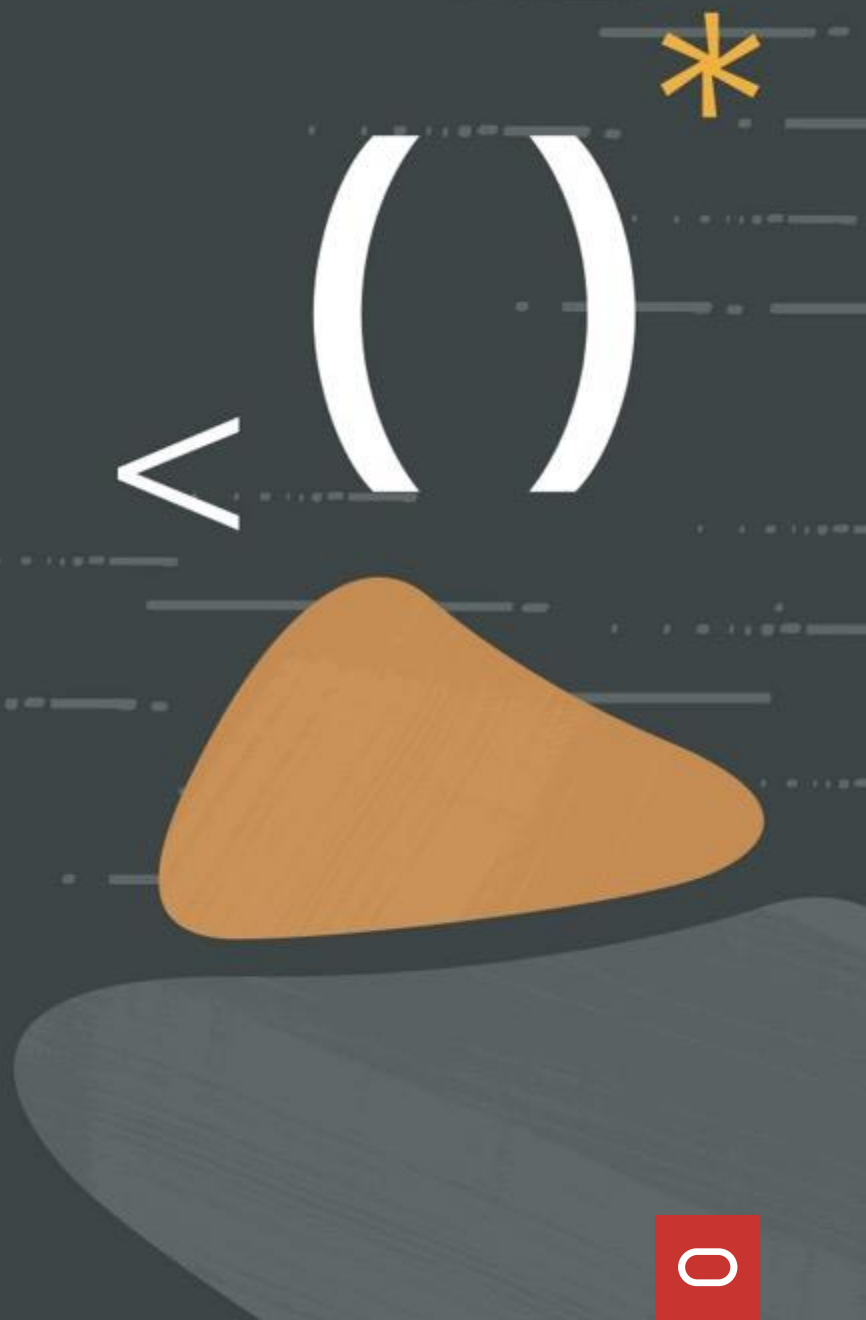
Application Performance Tuning



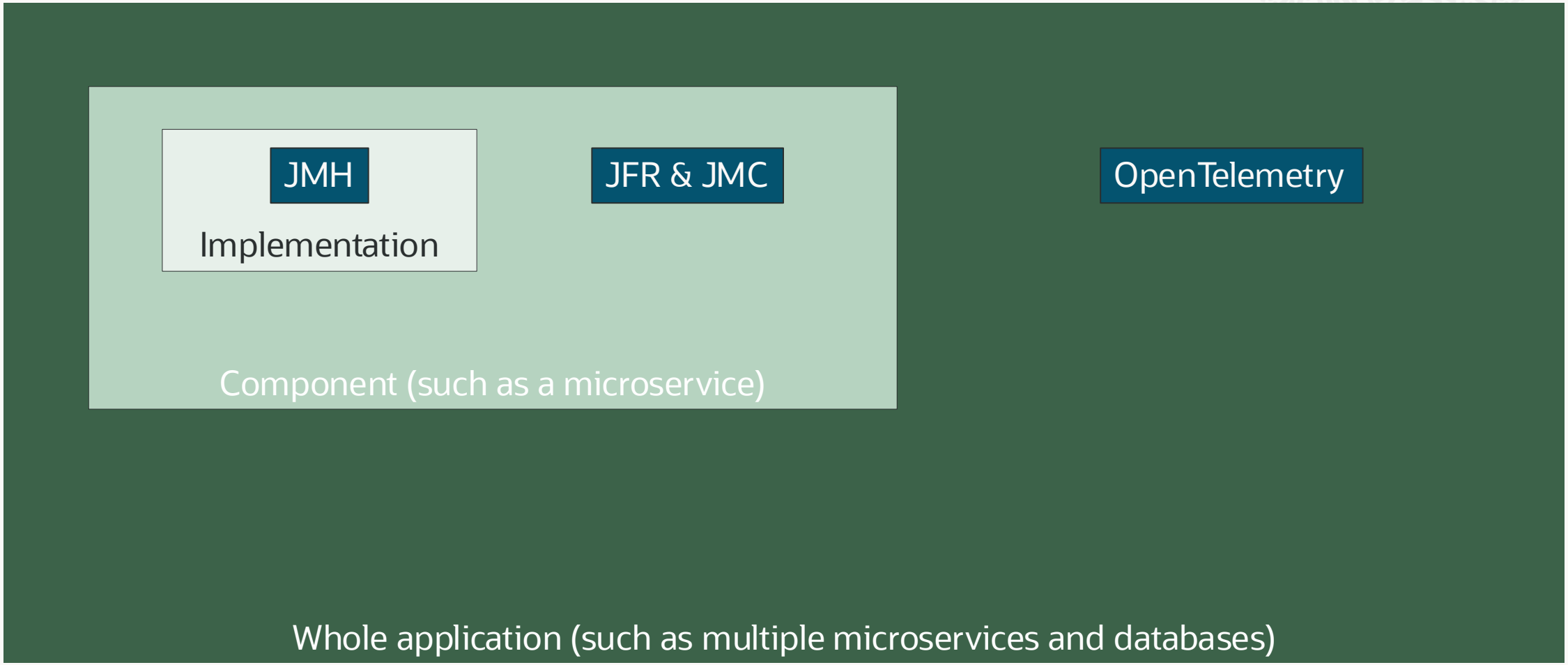
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Tools



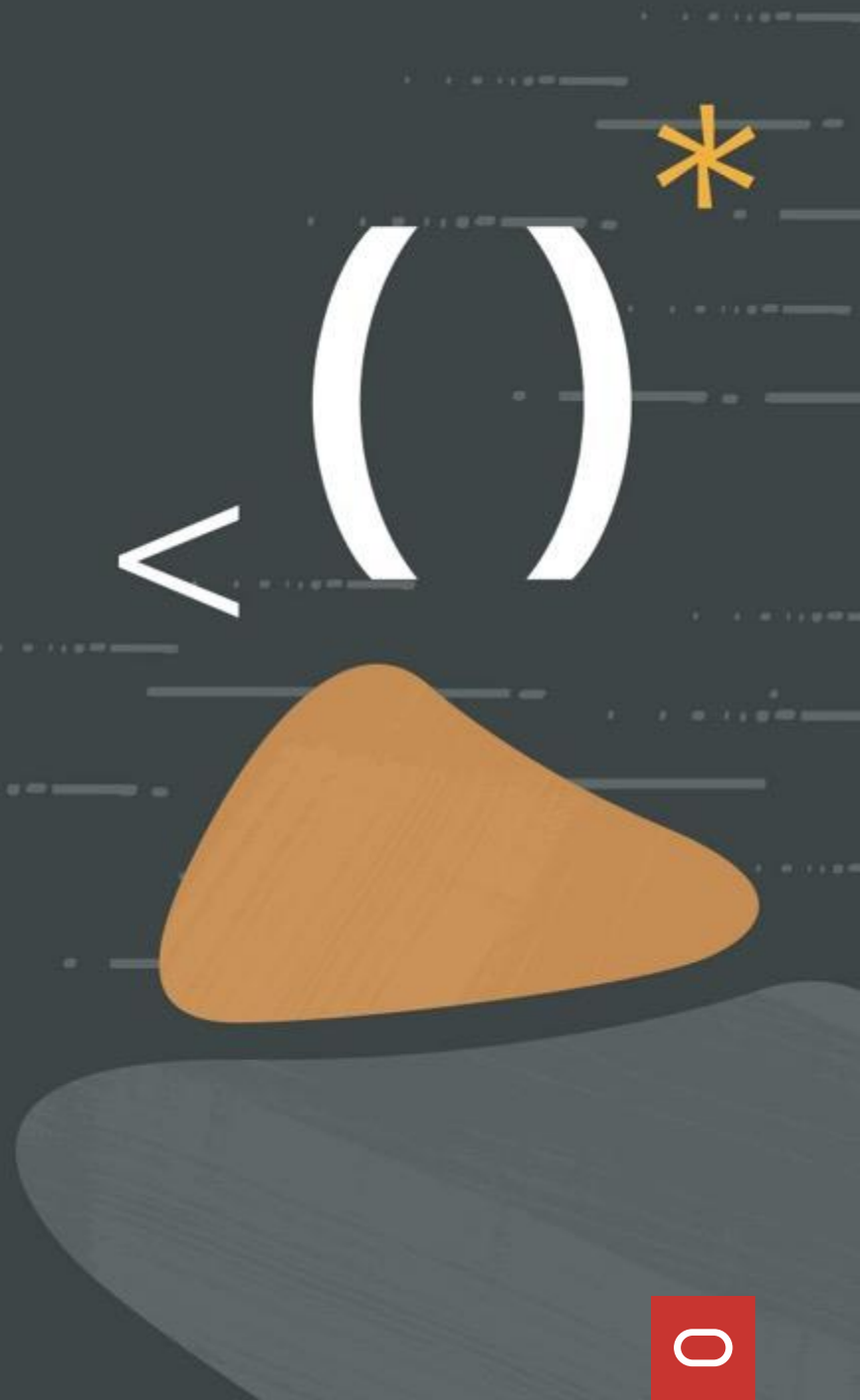


Application Performance
Tools



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

OpenTelemetry

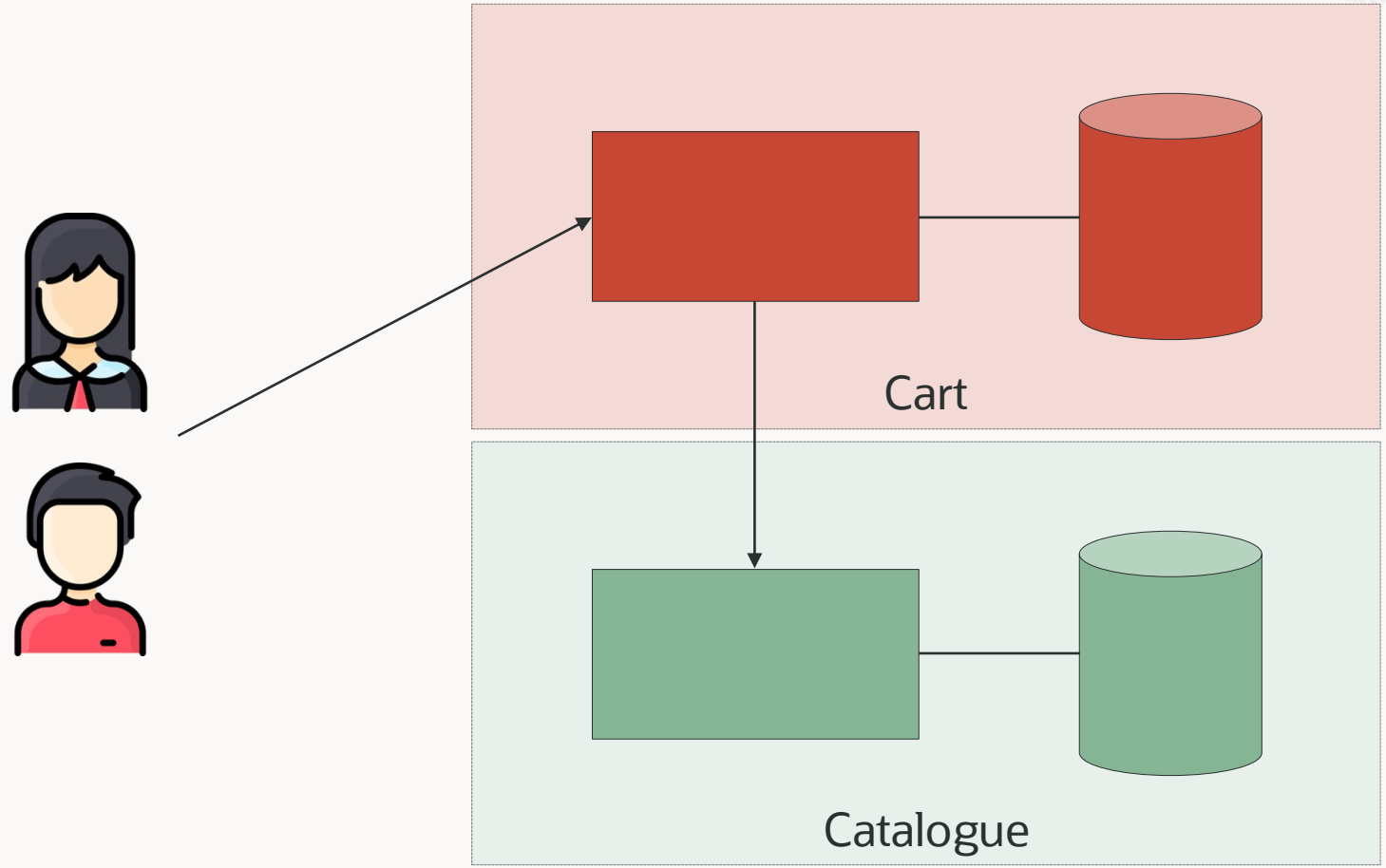


OpenTelemetry

- OpenTelemetry is an observability framework (*vendor*- and *tool*-agnostic) and toolkit designed to create and manage telemetry data, such as traces, metrics, and logs

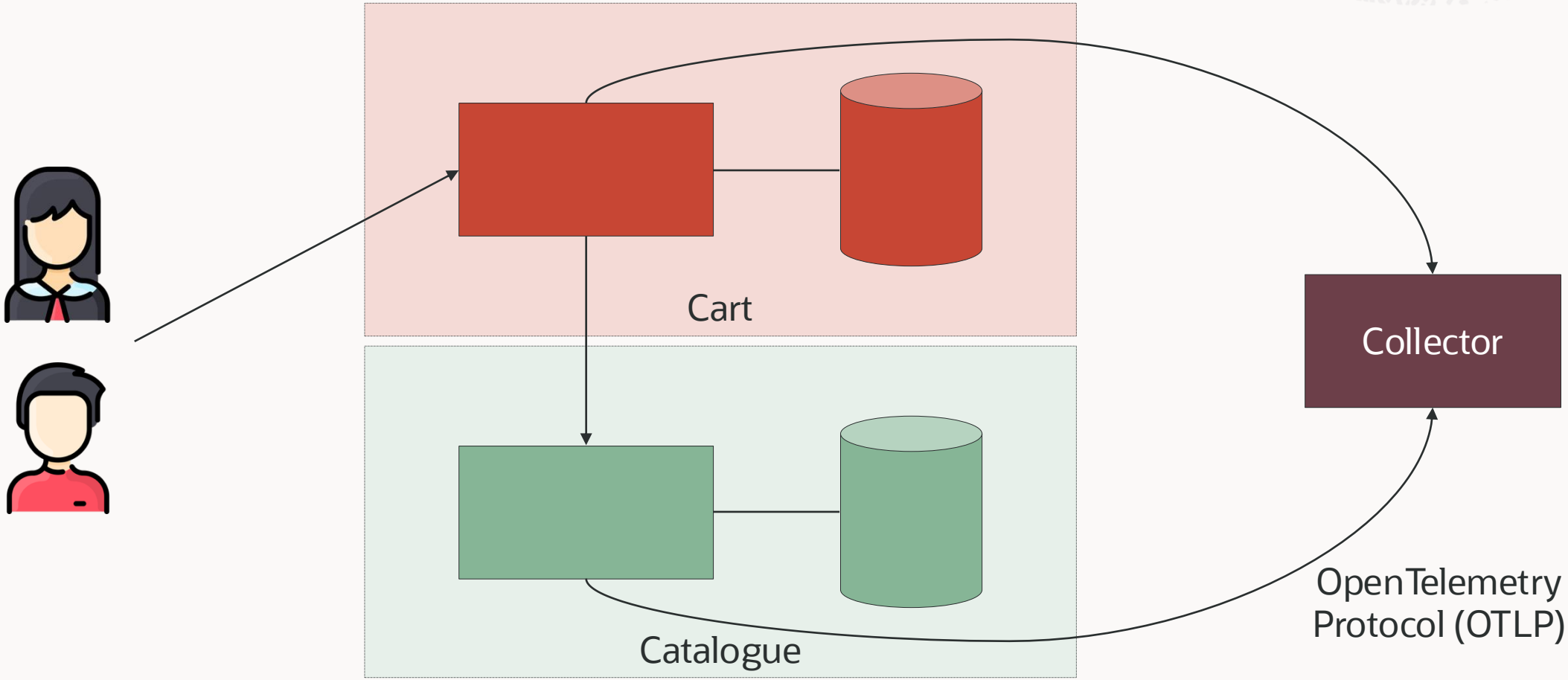
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

OpenTelemetry
Application



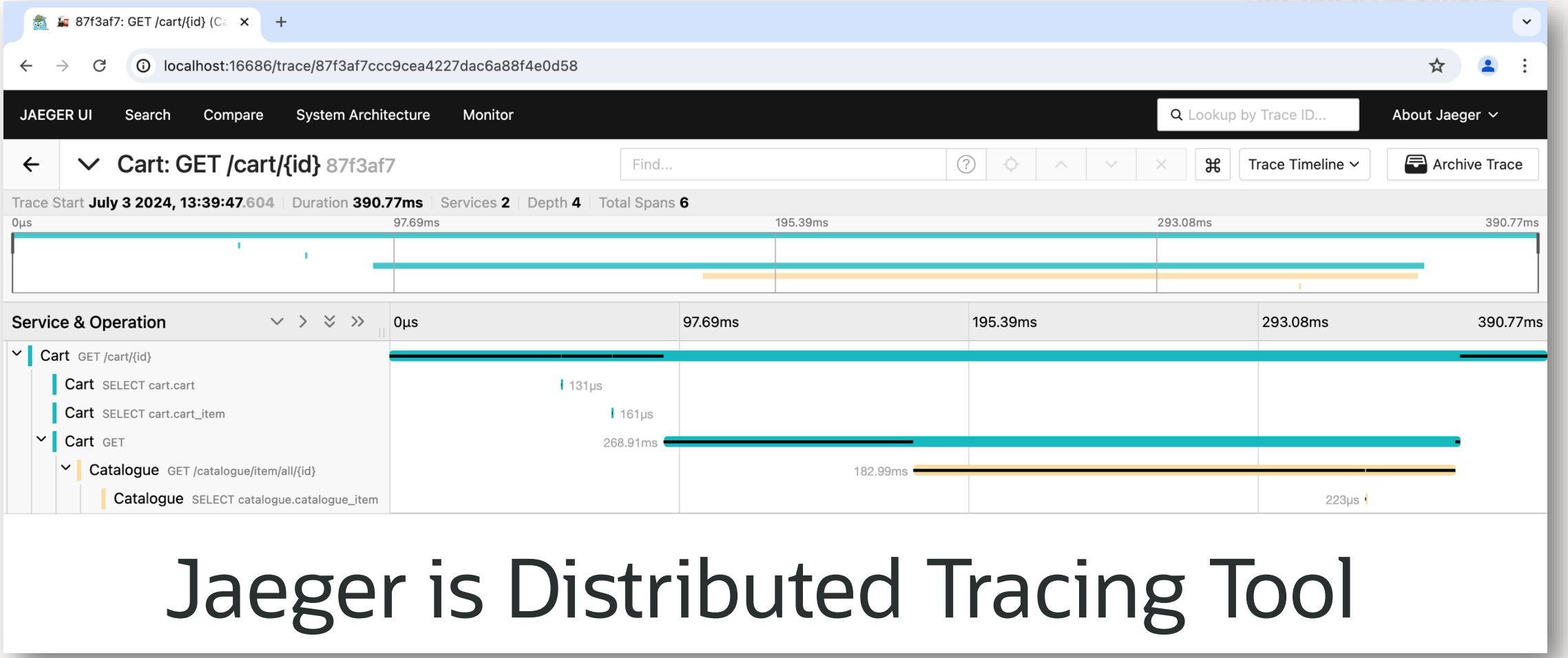
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

OpenTelemetry
Observability



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

OpenTelemetry
Jaeger



Jaeger is Distributed Tracing Tool

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

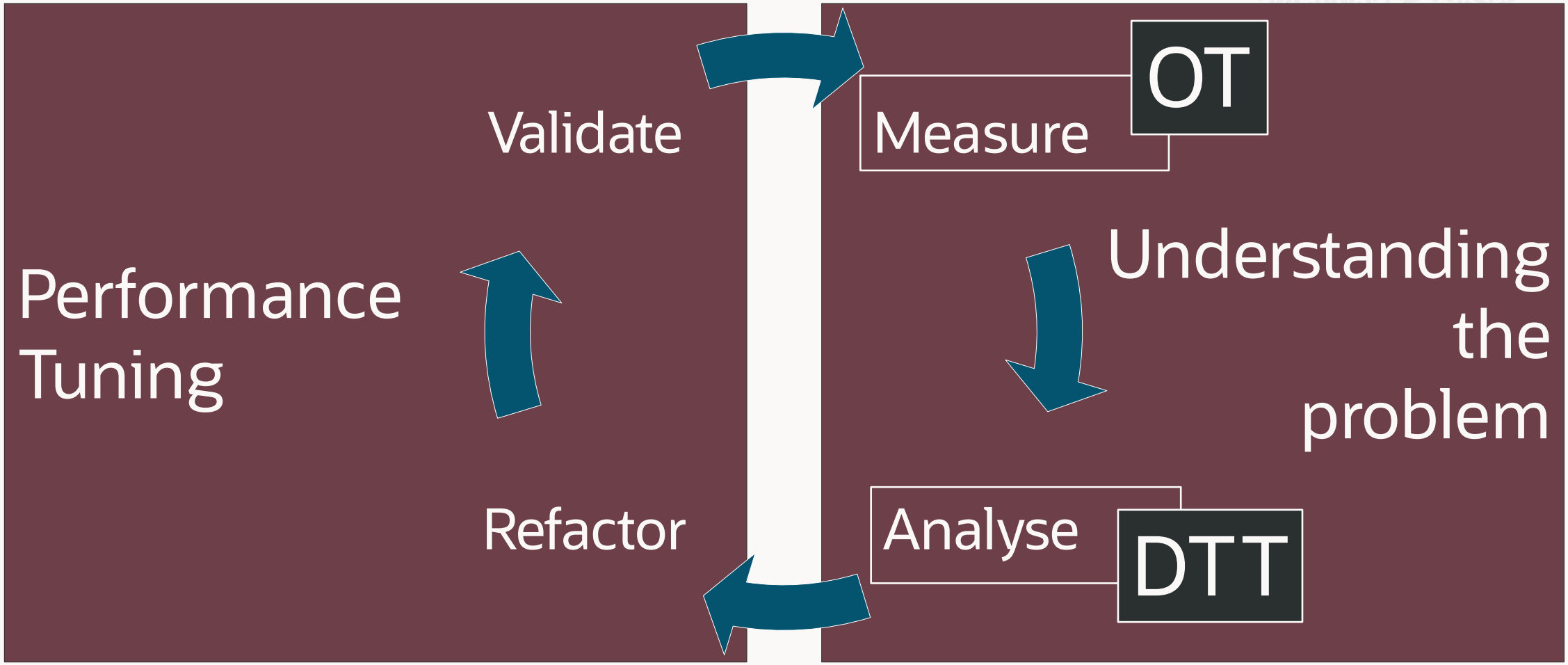
Where does OpenTelemetry fit in the application performance tuning cycle?





Application Performance Tuning

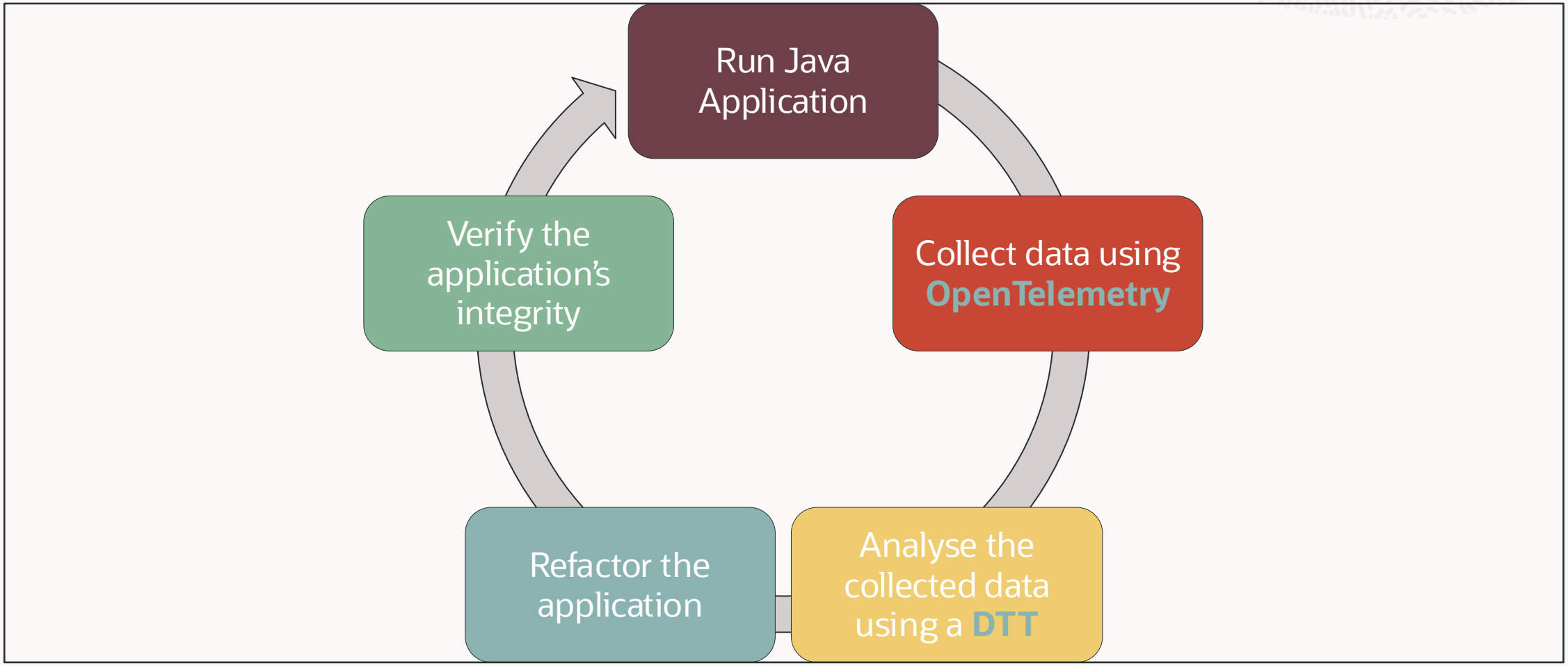
OpenTelemetry



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Application Performance Tuning

JDK Flight Recorder and JDK Mission Control



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Demo

`opentelemetry`

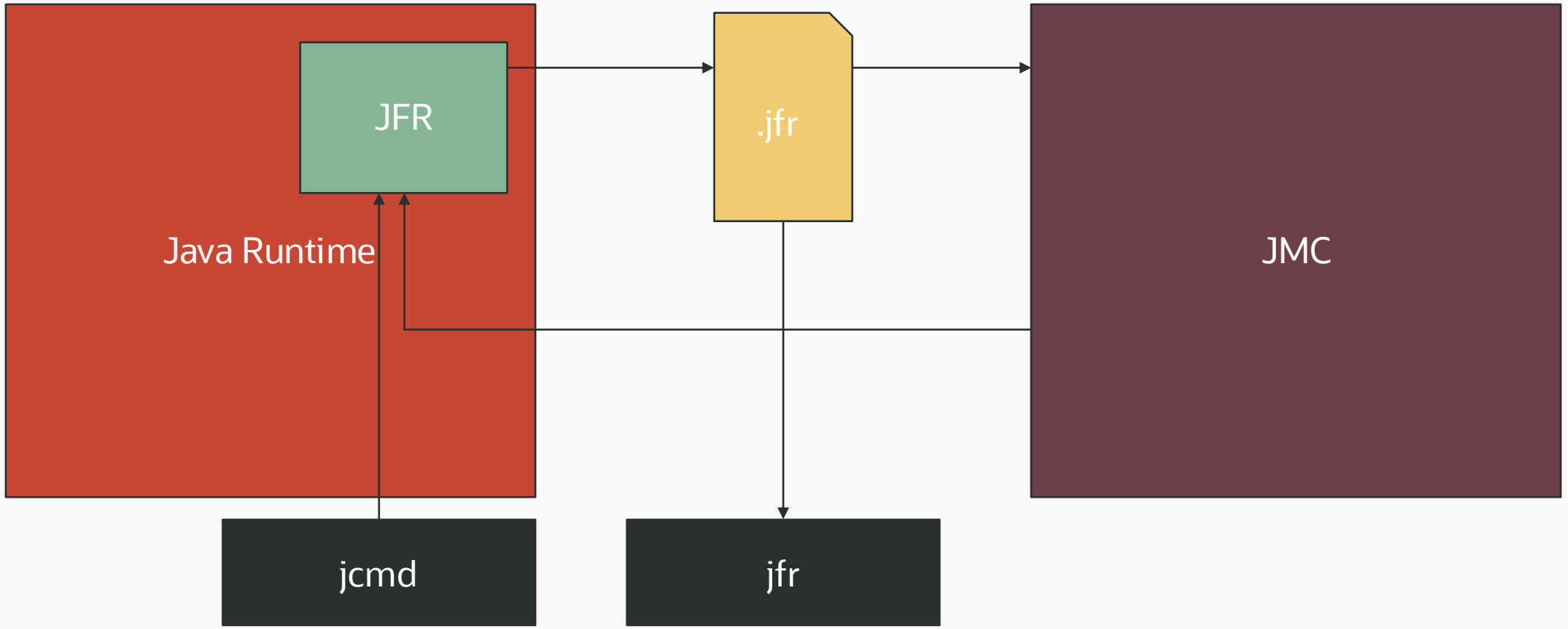
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JDK Flight Recorder and JDK Mission Control



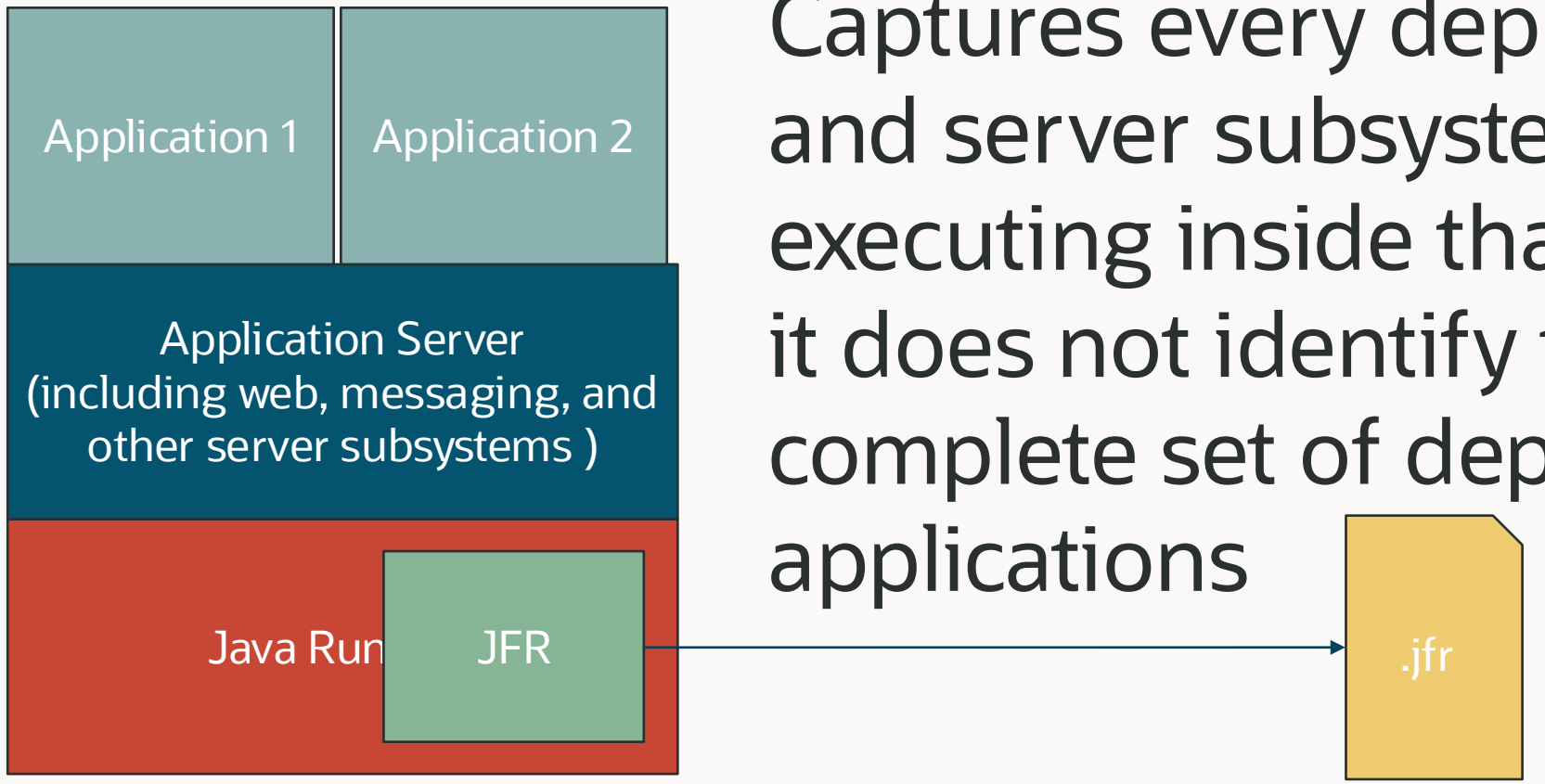
JDK Flight Recorder and JDK Mission Control

Overview



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JDK Flight Recording

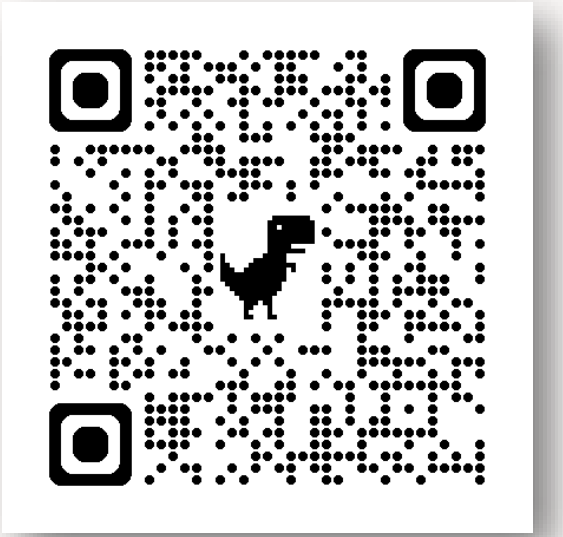


Captures every deployment and server subsystem executing inside that JVM, but it does not identify the complete set of deployed applications

Copyright © 2026, Oracle and/or its affiliates | Confidential: Internal | albert.attard@oracle.com

JDK Mission Control

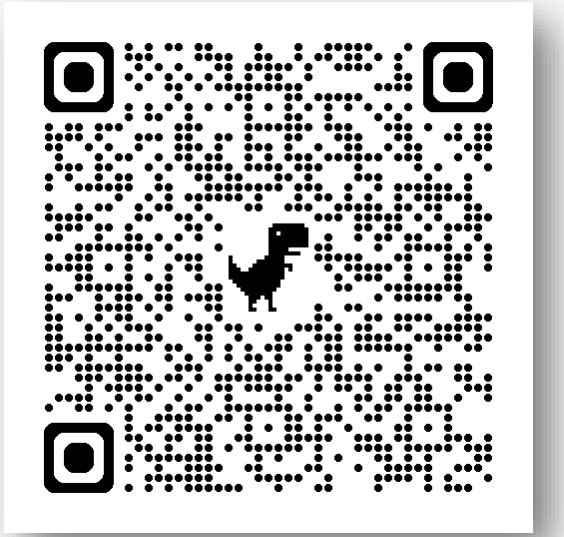
- JDK Mission Control (JMC) can be downloaded from here
- More information about JMC can be found here



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

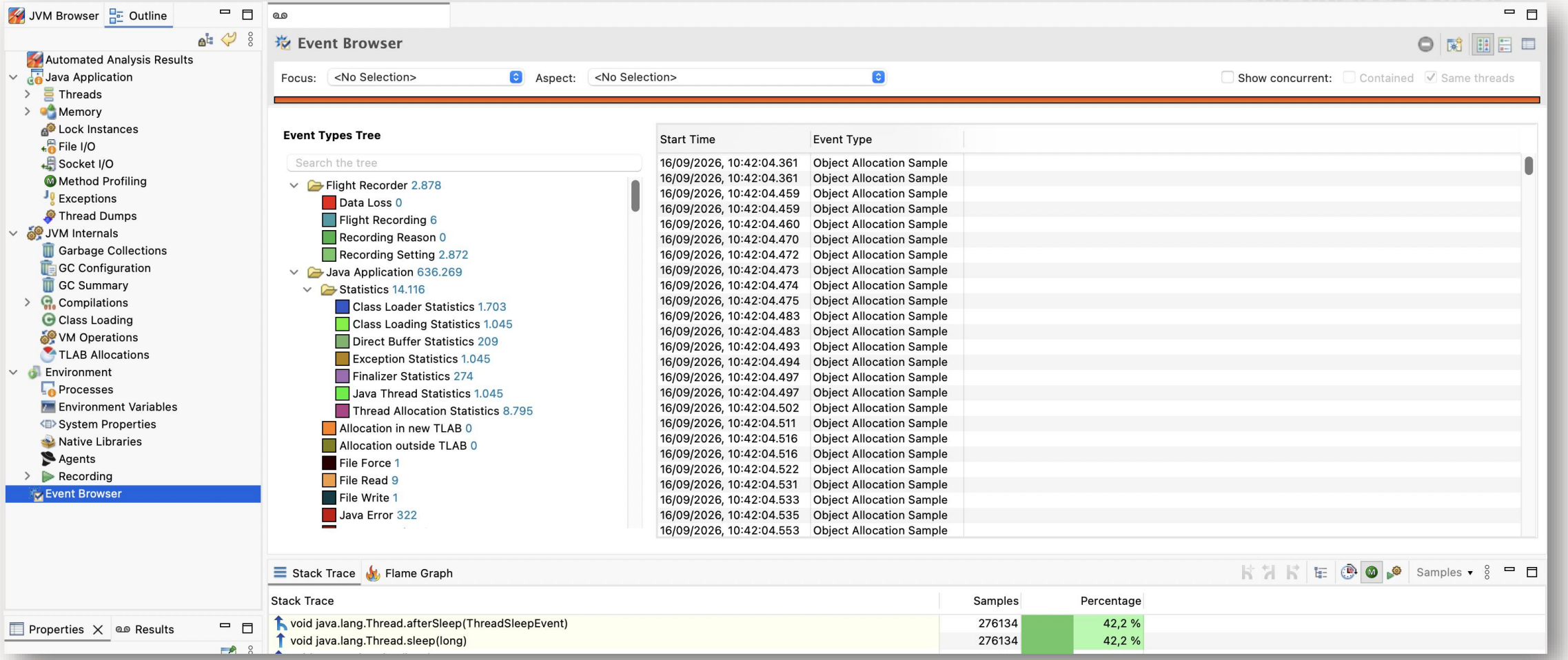
JDK Flight Recorder

- More information about JDK Flight Recorder (JFR) can be found here
- Note that JFR does not need to be downloaded as this is part of the Java



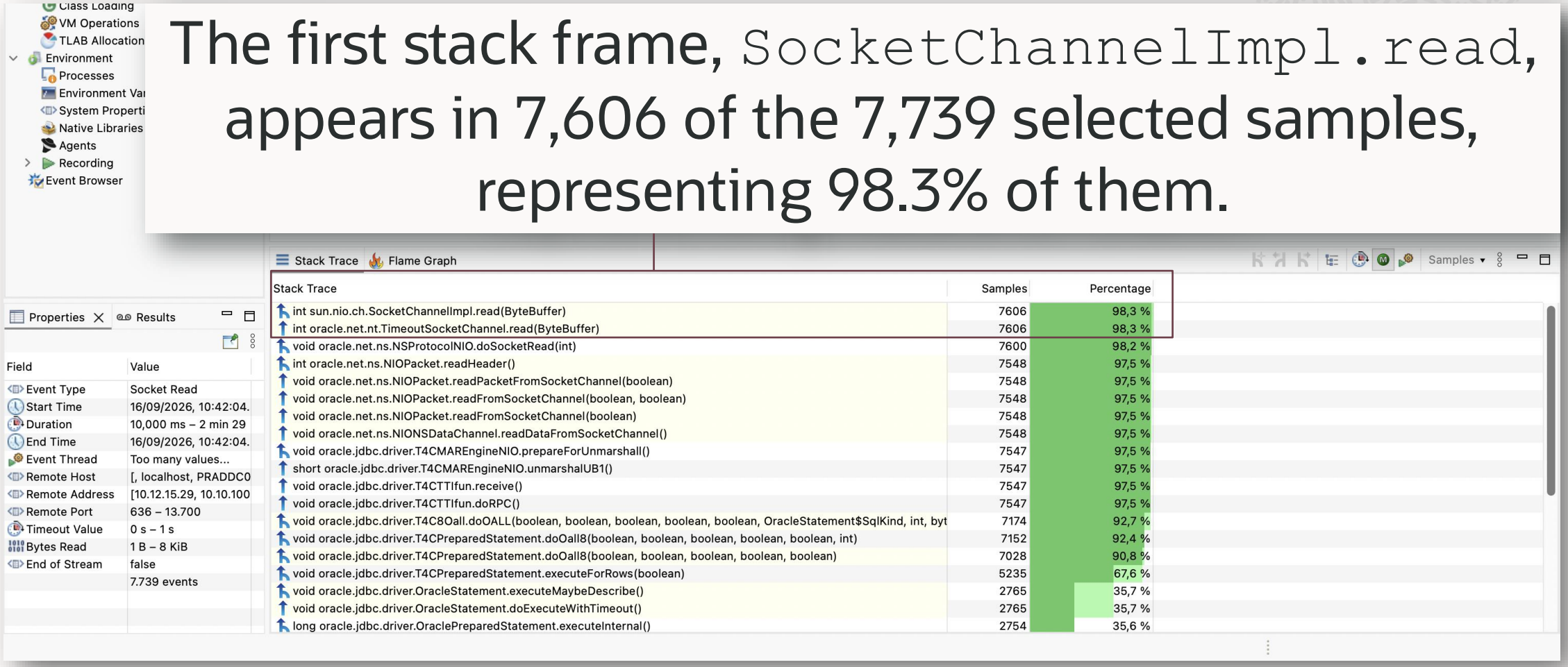
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JDK Flight Recorder



Copyright © 2026, Oracle and/or its affiliates | Confidential: Internal | albert.attard@oracle.com

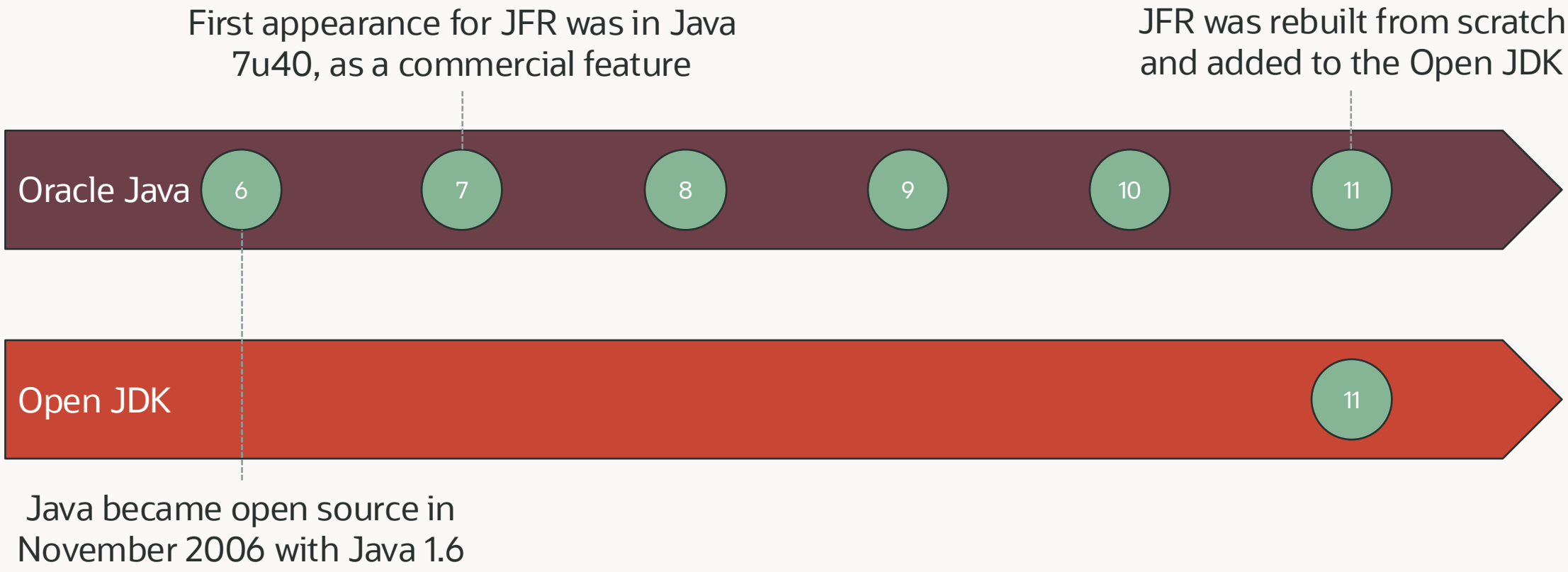
Stack trace frequency



Copyright © 2026, Oracle and/or its affiliates | Confidential: Internal | albert.attard@oracle.com

JEP 328: Flight Recorder

Timeline



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JDK Flight Recorder versions

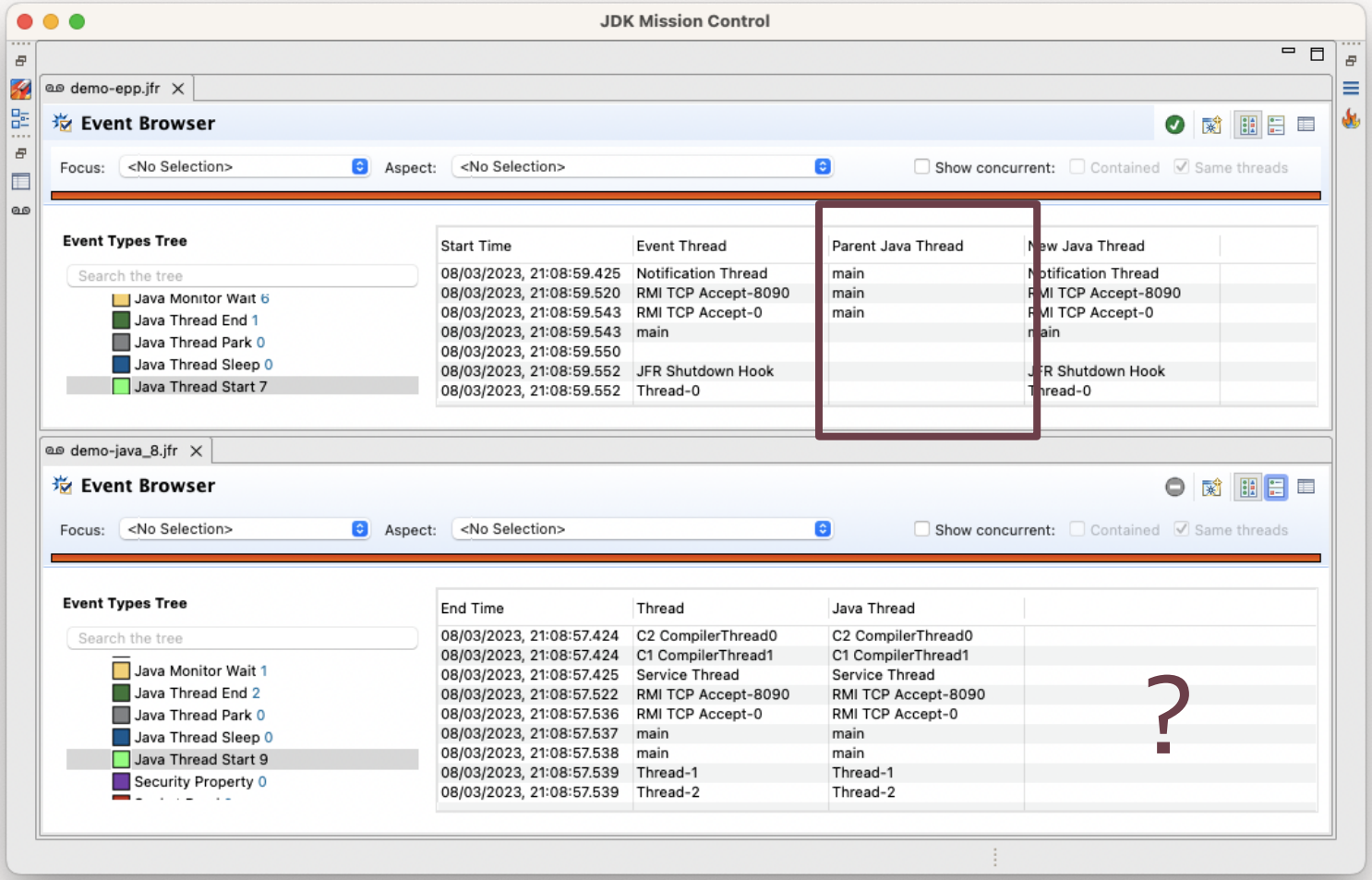
Oracle Java	JFR Version
Oracle Java 7	0.9
Oracle Java 8	0.9
Oracle Enterprise Performance Pack	2.1
Oracle Java 9	1.0
Oracle Java 11	2.0
Oracle Java 14 (and newer)	2.1

Based on Oracle Java 17

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JDK Flight Recorder (JFR)

JDK Mission Control

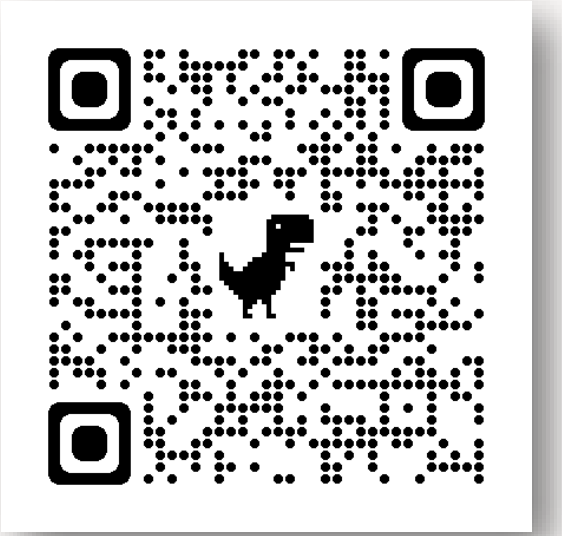


EPP, like Java 17, may have **different** event details, compared to Java 8

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Oracle Java 8

- Please note that JFR needs to be enabled when using Oracle Java 8 (technical notes)



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Oracle Java 8

```
> java \
  -XX:+UnlockCommercialFeatures \
  -XX:+FlightRecorder \
  -jar 'app.jar'
```

Needed for releases prior to JDK8u40

OR

```
> jcmd <PID> \
  VM.unlock_commercial_features
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



jlink

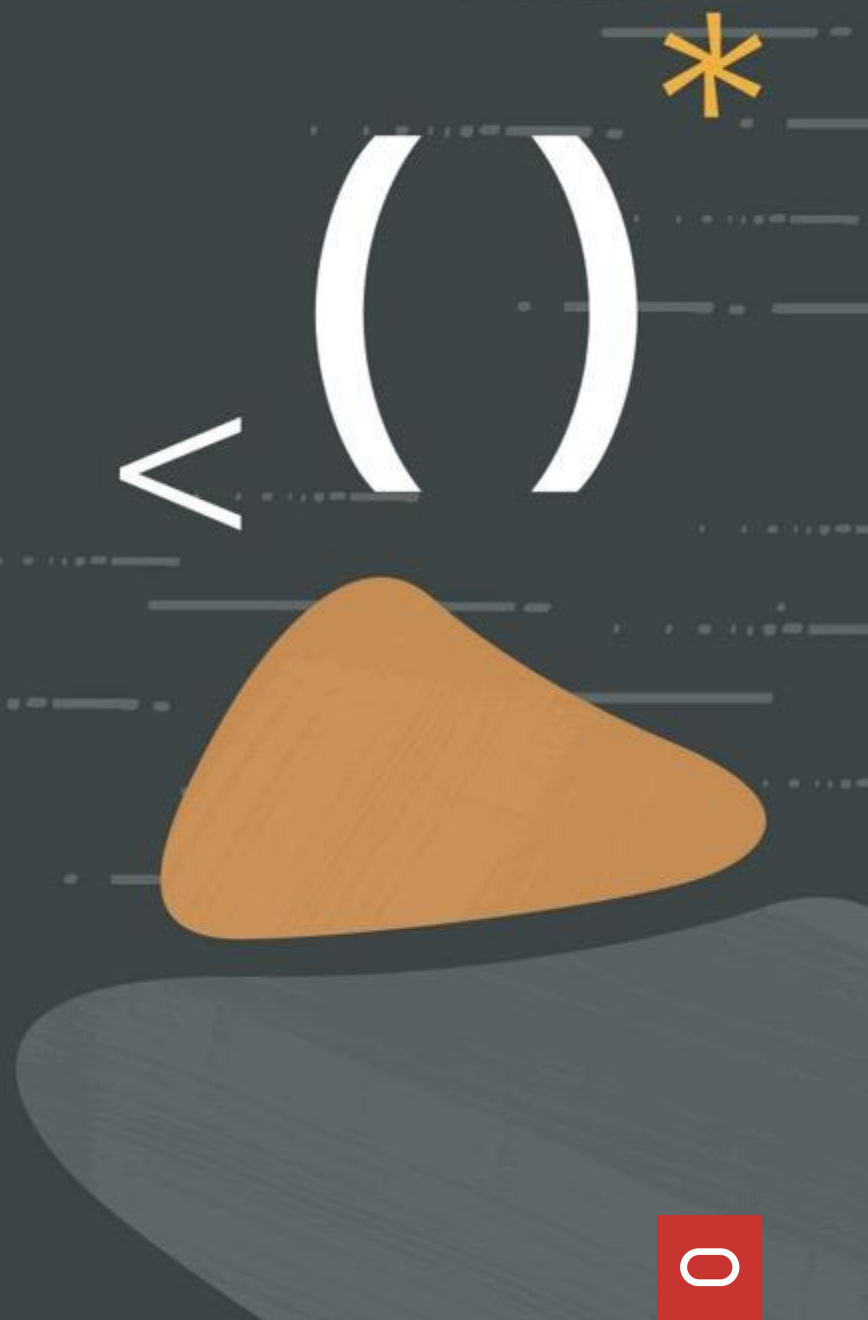
- When using `jlink`, make sure to include the following modules, otherwise JMC will not be able to connect to the running JVM

```
> jlink \
  --add-modules jdk.jfr,jdk.management.jfr,java.management,
↪jdk.management,jdk.management.agent,java.rmi \
  ...
```

This is one single string

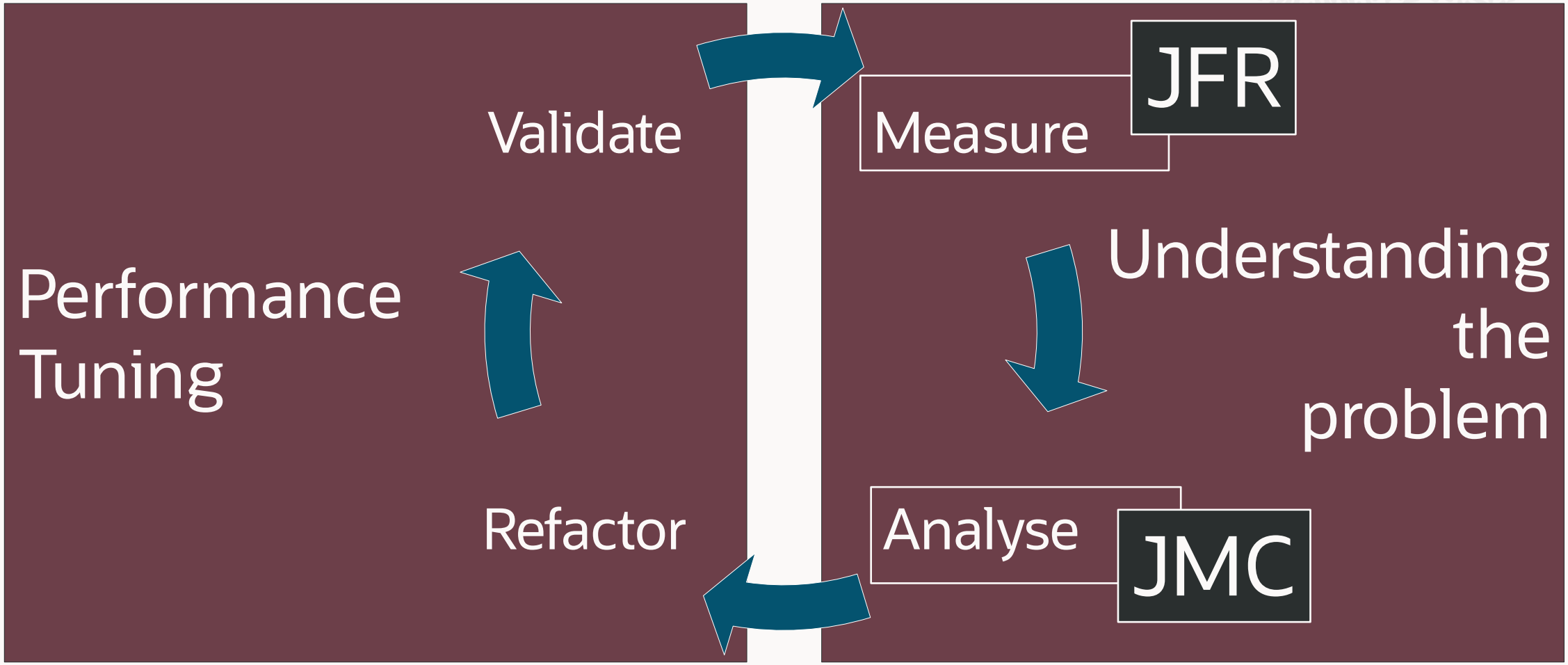
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Where do JFR and JMC fit in the application performance tuning cycle?



Application Performance Tuning

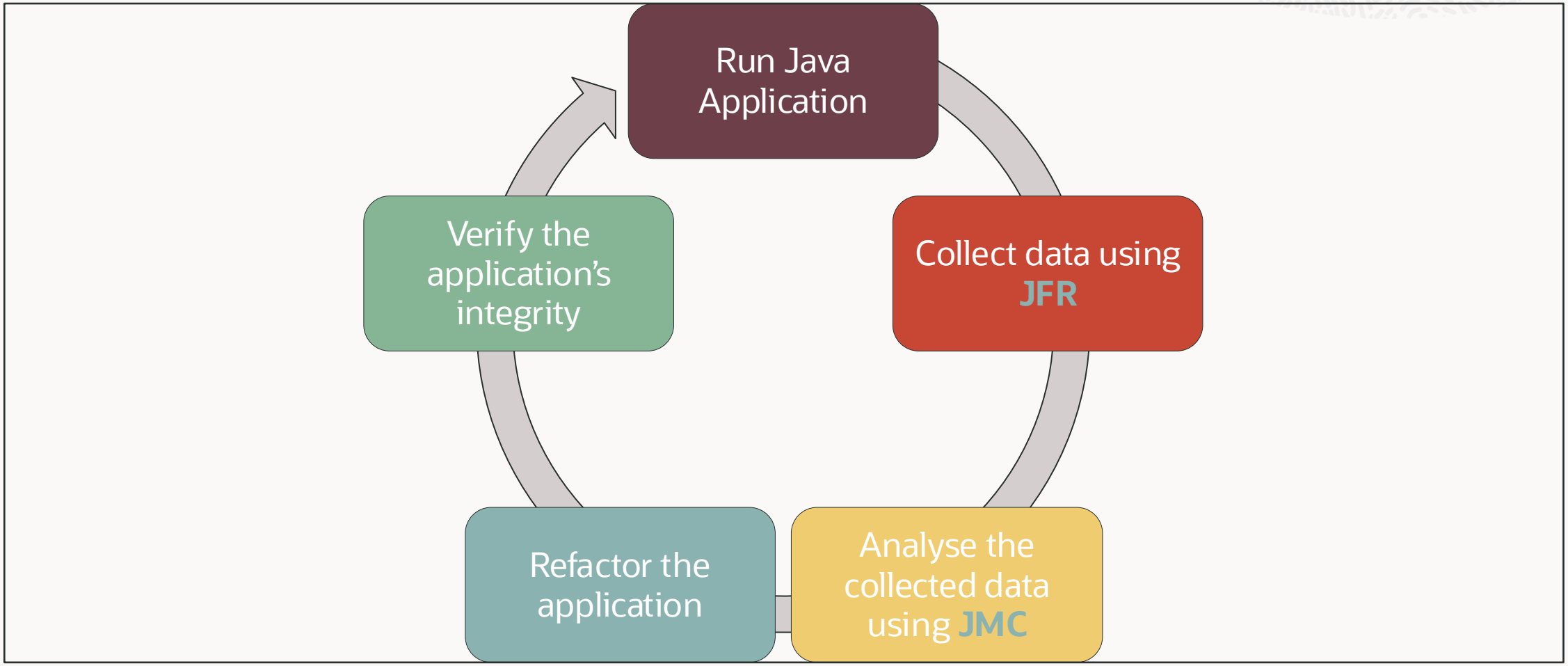
JDK Flight Recorder and JDK Mission Control



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Application Performance Tuning

JDK Flight Recorder and JDK Mission Control



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Examples

- Start JFR
- Continuous Rec.
- JFR Configuration
- Custom Events
- Crypto Events
- Socket I/O
- Hot Methods
- Latency
- GC
- Memory Leaks
- Exceptions

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Start JFR



Simple Application

```
package demo;

import module java.base;

public final class Main {

    static void main() throws InterruptedException {
        System.out.println("The application will keep running ...");
        new CountDownLatch(1).await();
    }
}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



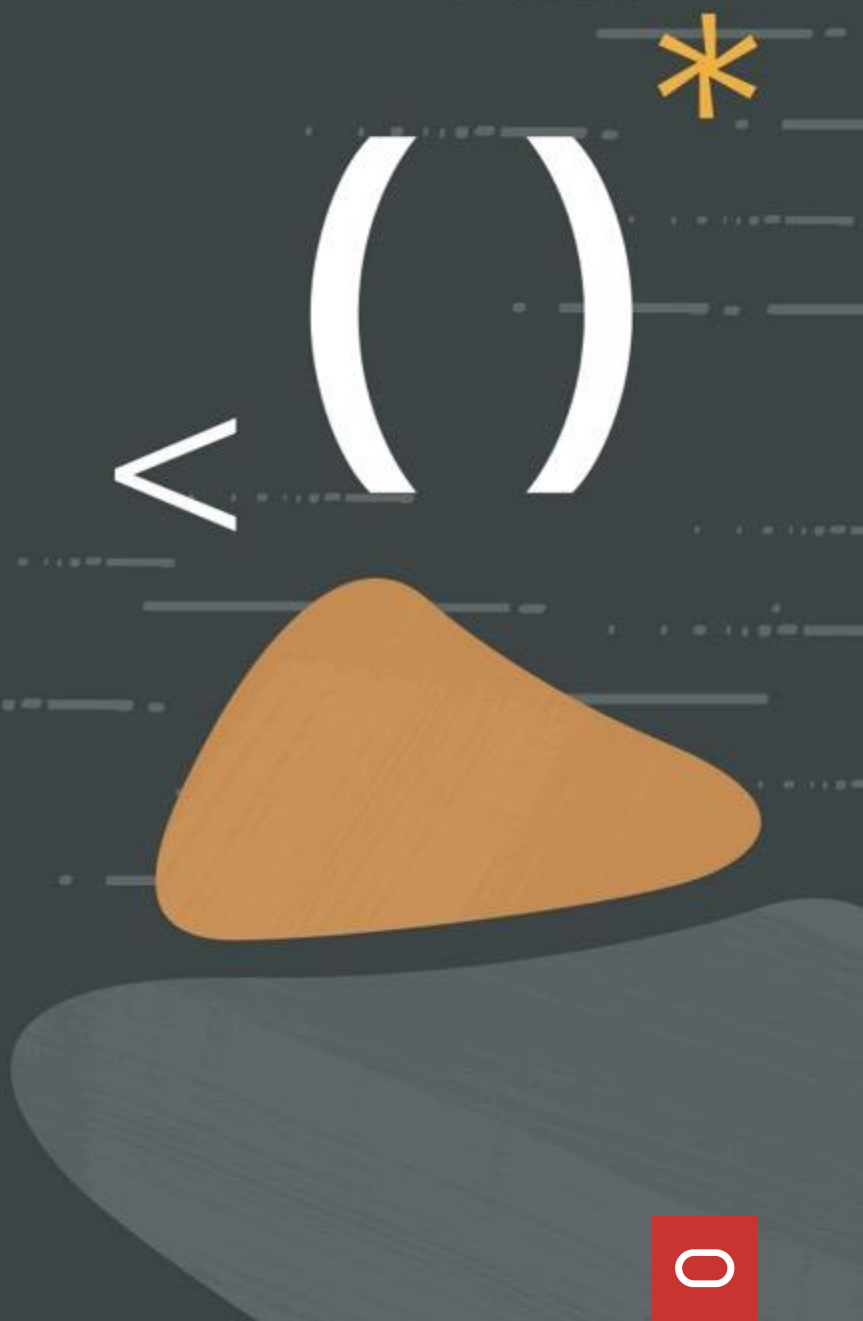
How to start JFR?

- Using JDK Mission Control (JMC)
- Using `jcmd`
- During application start time
- Programmatically
- Using Java Management Service (JMS)

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Using JDK Mission Control (JMC)



Using JDK Mission Control (JMC)

Run the application

- Start the application without providing any flight recording related options

```
> java -jar 'app.jar'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

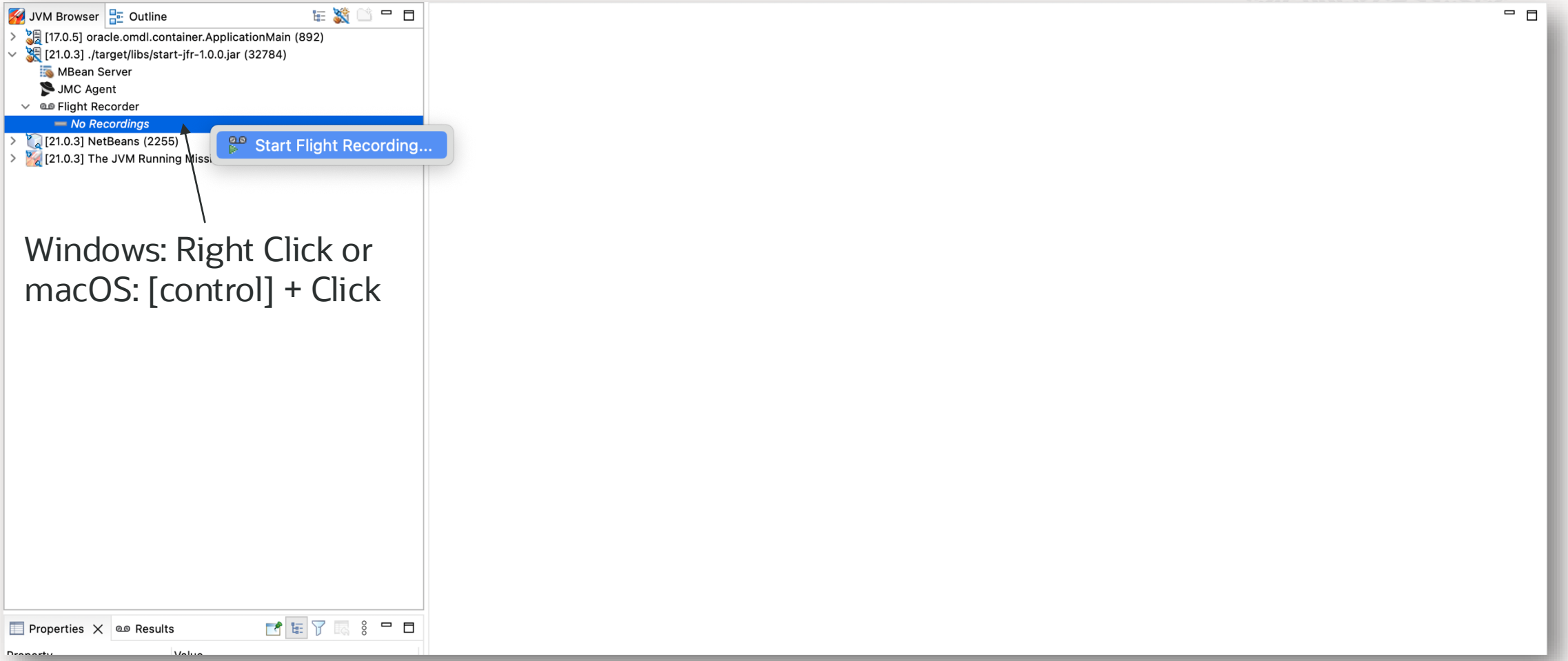
Find the application



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

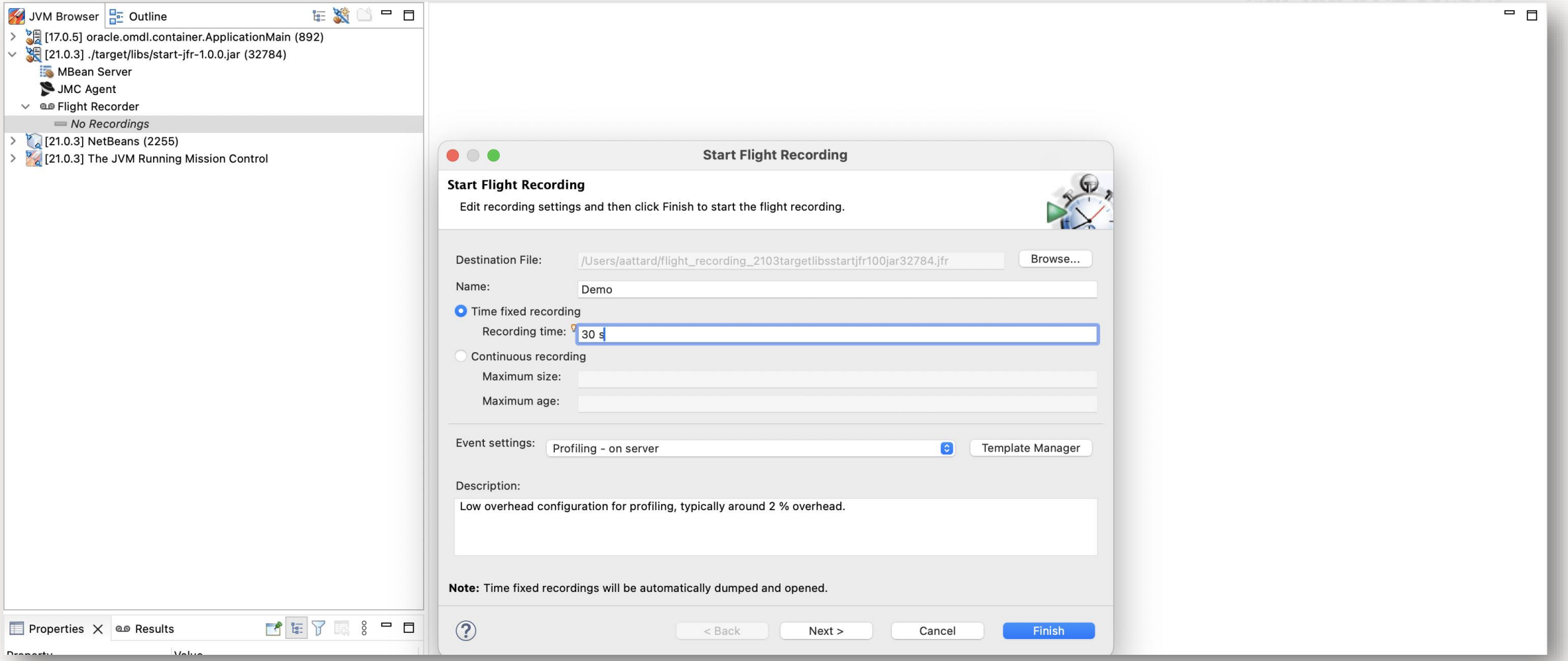
Start the flight recording



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

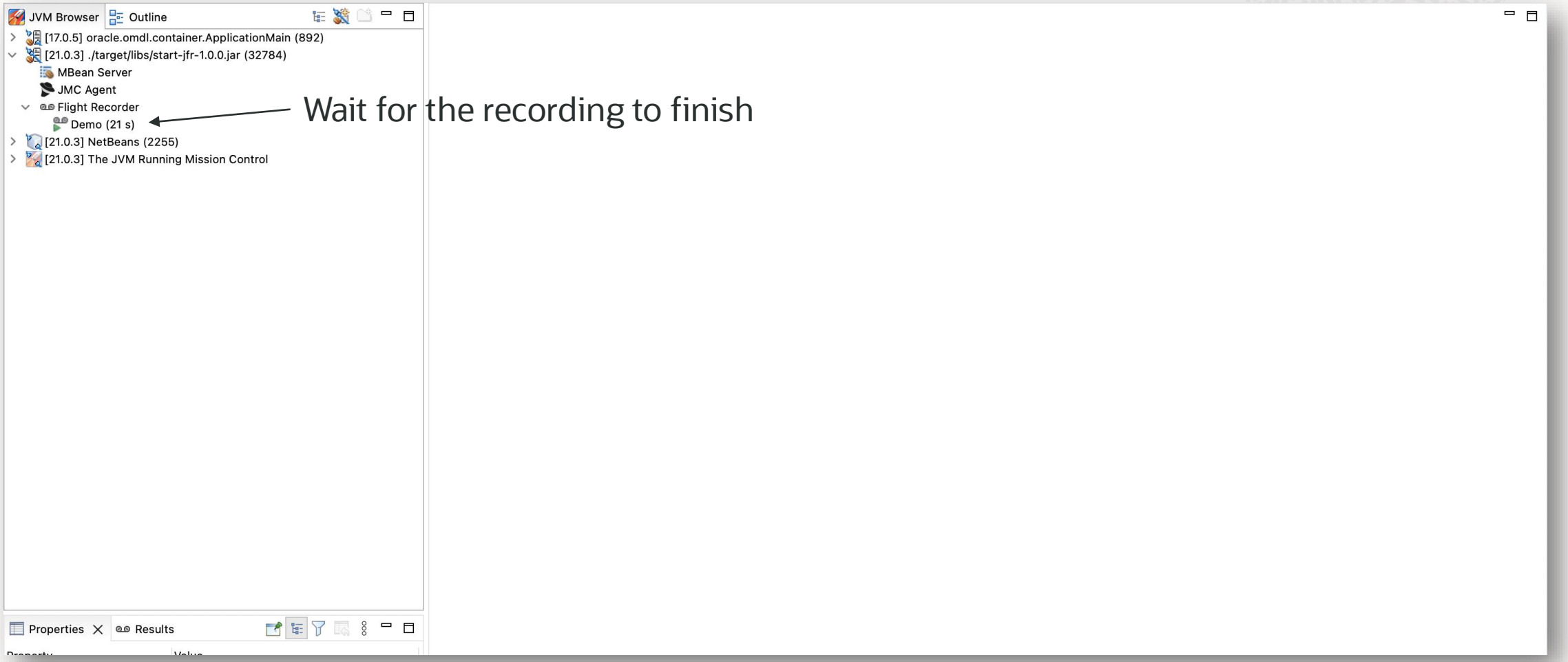
Provide the flight recording options



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

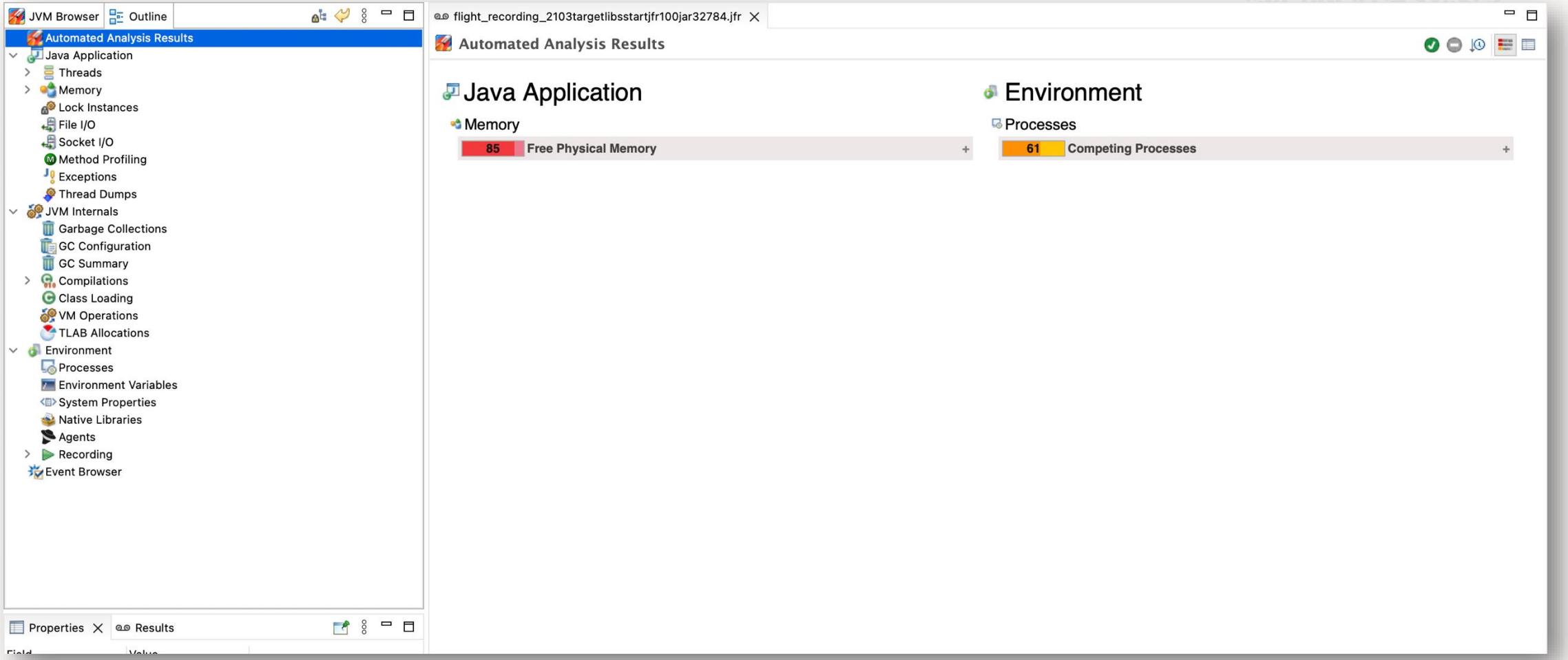
Wait for the recording to complete



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

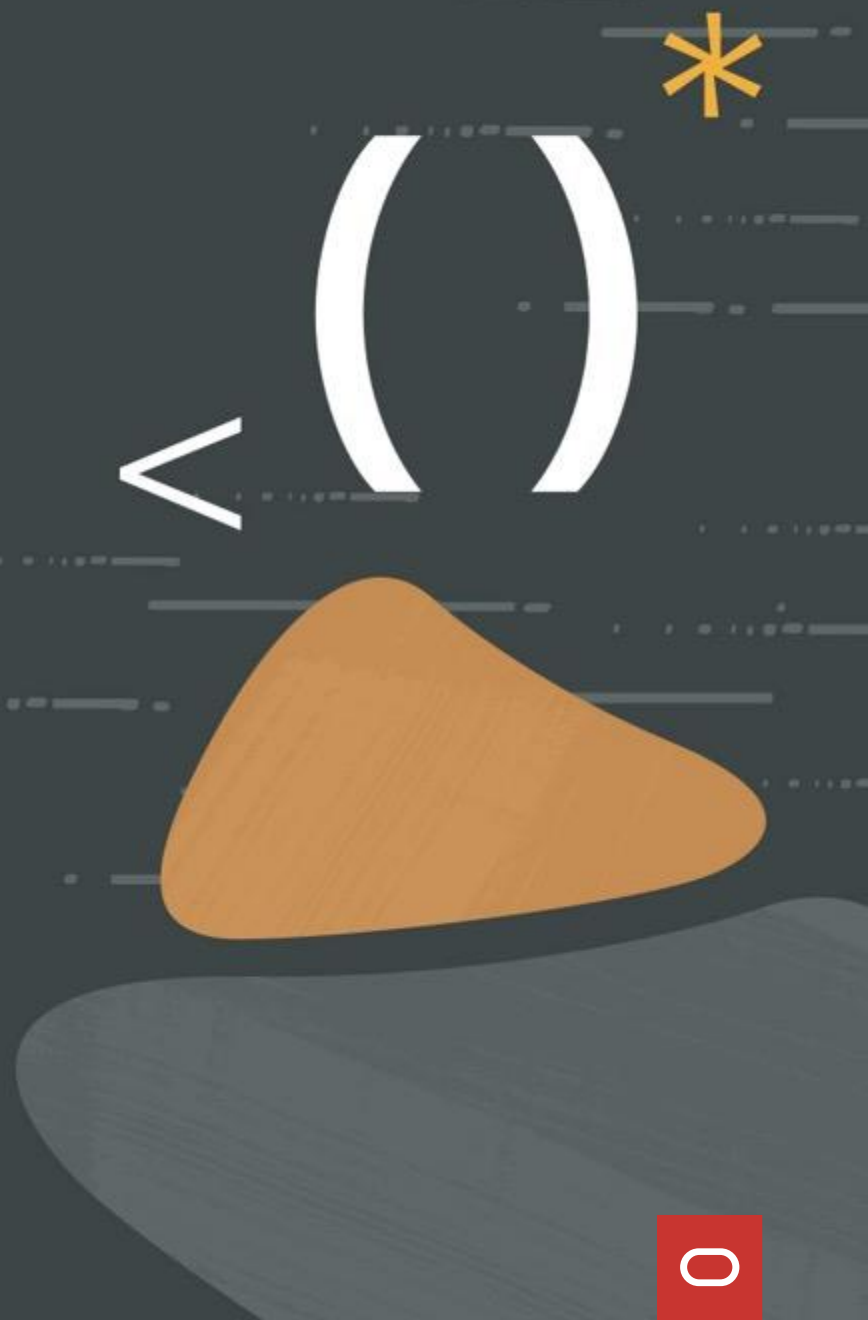
Using JDK Mission Control (JMC)

View the flight recording



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using `jcmd`



Using `jcmd`

Run the application

- Start the application without providing any flight recording related options

```
> java -jar 'app.jar'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Using `jcmd`

Identify the Java application PID

- List all Java applications

```
> jcmd
```

- Get the PID of our application

```
> jcmd \
  | grep 'app.jar' \
  | cut -d' ' -f1
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using `jcmd`

Start flight recording

- Start flight recording

```
> jcmd <PID> \
  JFR.start \
  name='Start-JFR' \
  duration=30s \
  filename='./jfr/demo.jfr'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using `jcmd`

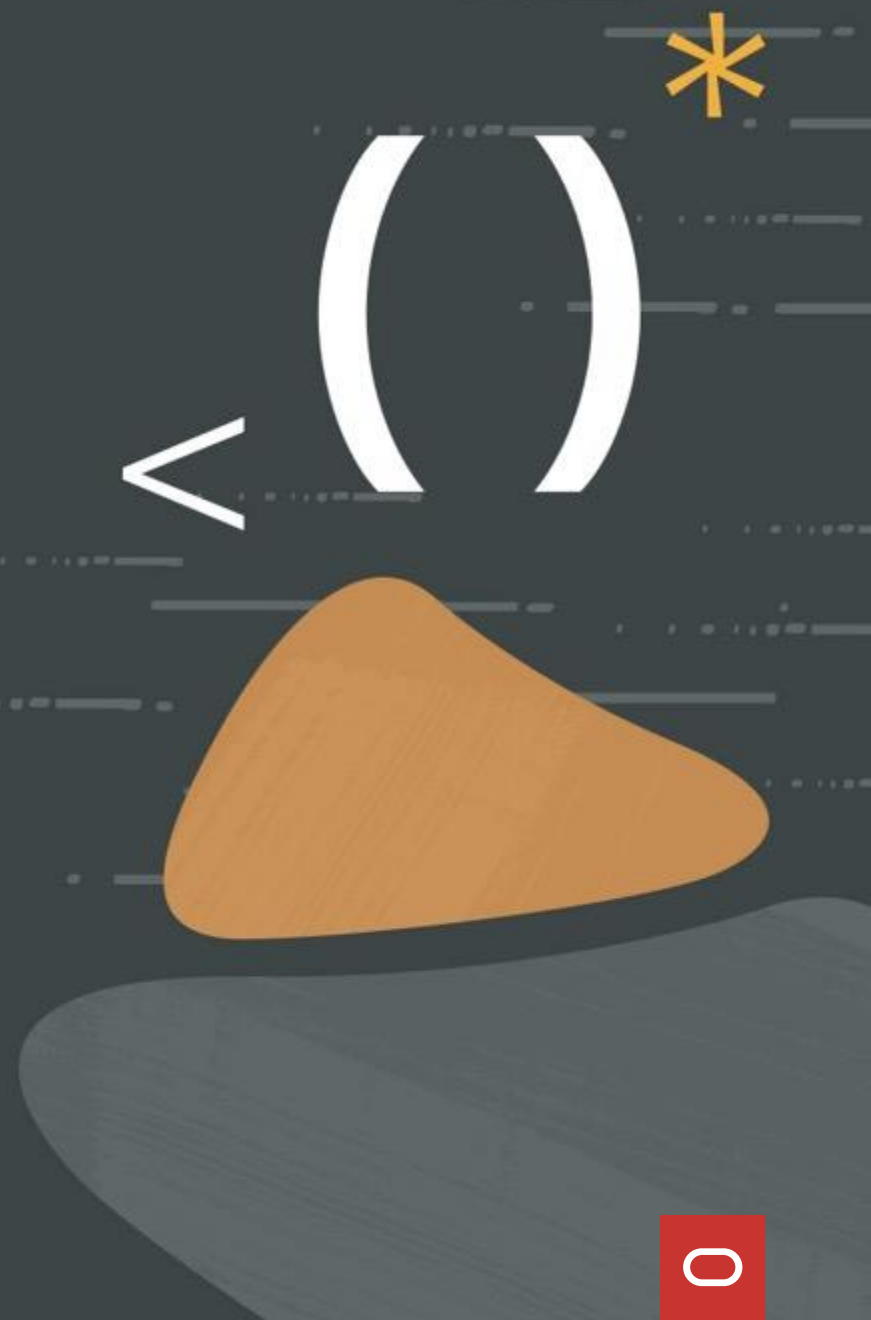
Oracle Java 8

```
> jcmd <PID> \
  VM.unlock_commercial_features

> jcmd <PID> \
  JFR.start \
  name='Start-JFR' \
  duration=30s \
  filename='./jfr/demo.jfr'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

During application start time



During application start time

Run the application and start flight recording

- Start the application by providing the flight recording options

```
> java \
  -XX:StartFlightRecording=... \
  -jar 'app.jar'
```

Complete example on the next slide

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

During application start time

Run the application and start flight recording

```
> java \
  -XX:StartFlightRecording=
↪duration=30s,
↪delay=5s,
↪name='Start-JFR',
↪filename='./jfr/demo.jfr' \
  -jar 'app.jar'
```

This is one single string

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



During application start time
Oracle Java 8

- The commercial features need to be unlocked and JFR enabled

```
> java \
  -XX:+UnlockCommercialFeatures \
  -XX:+FlightRecorder
  -XX:StartFlightRecording=... \
  -jar 'app.jar'
```

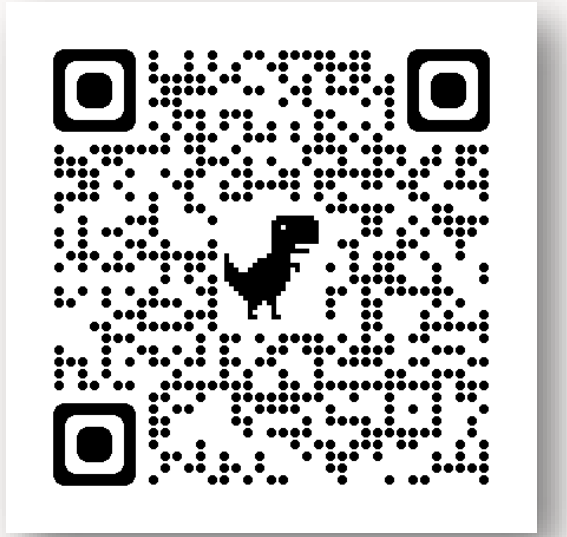
Needed for releases prior to JDK8u40

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

During application start time

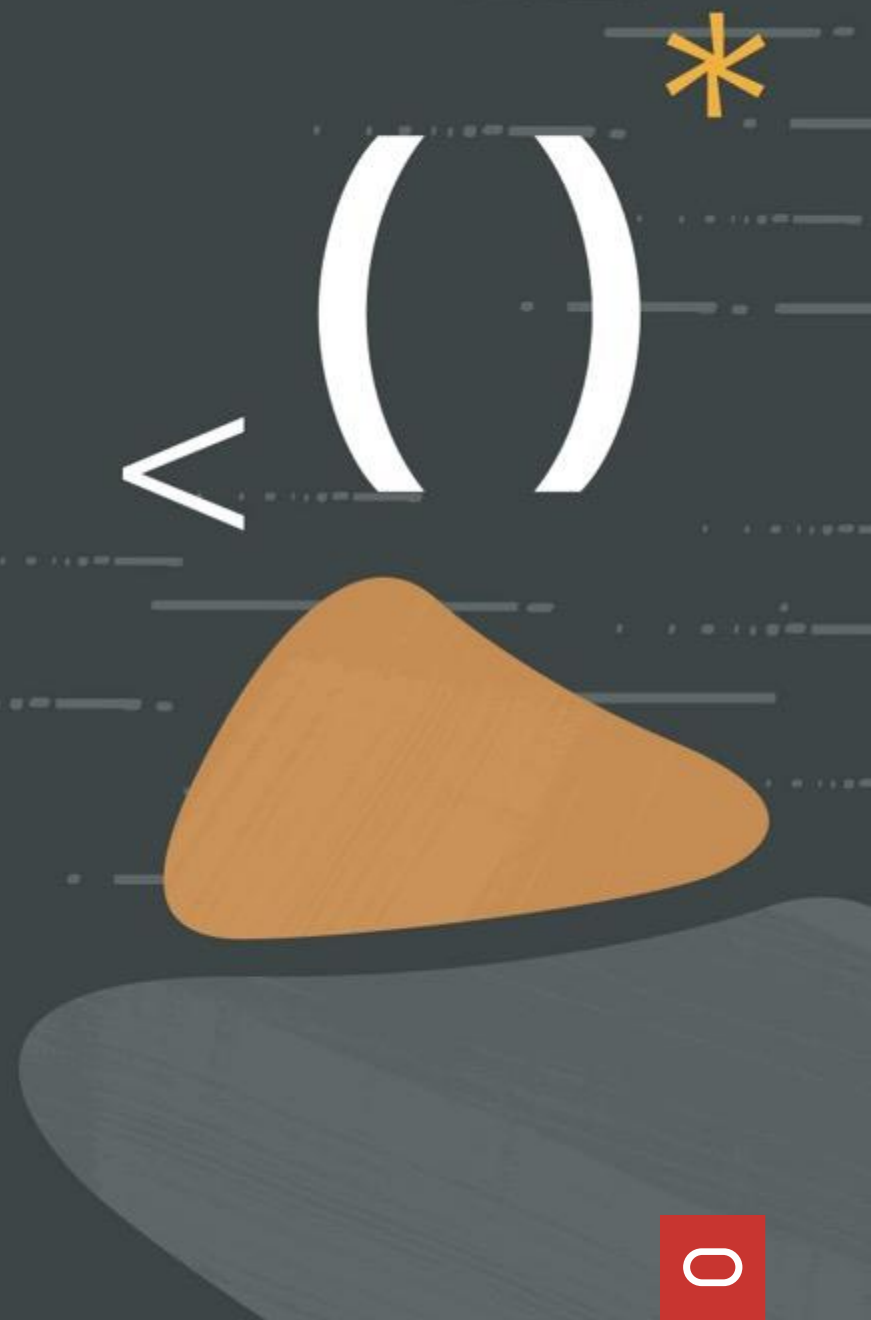
`-XX:StartFlightRecording`

- For more information about the `-XX:StartFlightRecording` command line option, please visit the `java` command documentation



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Programmatically



Programmatically

Modified application

```
try (Recording recording =
      new Recording(Configuration.getConfiguration("default"))) {
    recording.setDestination(Path.of("./jfr/demo-programmatically.jfr"));
    recording.setName("Start-JFR");
    recording.setDuration(Duration.ofSeconds(30));
    recording.start();

    /* Wait while capturing the events and saved the to the recording. */
    System.out.println("The application will terminate automatically …");
    TimeUnit.SECONDS.sleep(40);
}
/* The recording is completed and written to file. */
/* No more events are captured by this recording. */
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Programmatically
Run the application

- Start the application without providing any flight recording related options

```
> java -jar 'app.jar'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

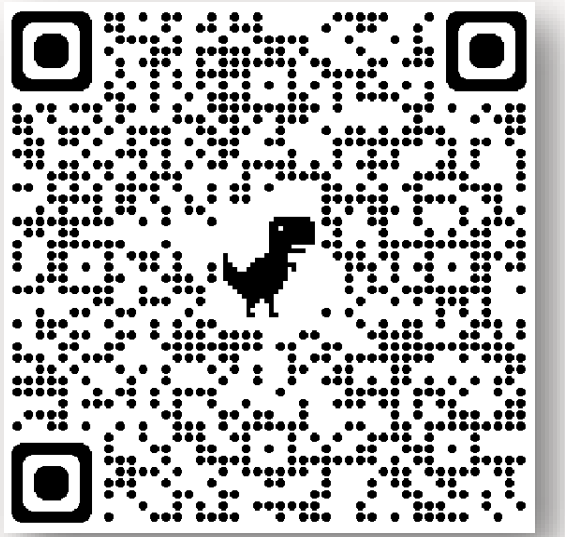


Using Java Management Service (JMS)



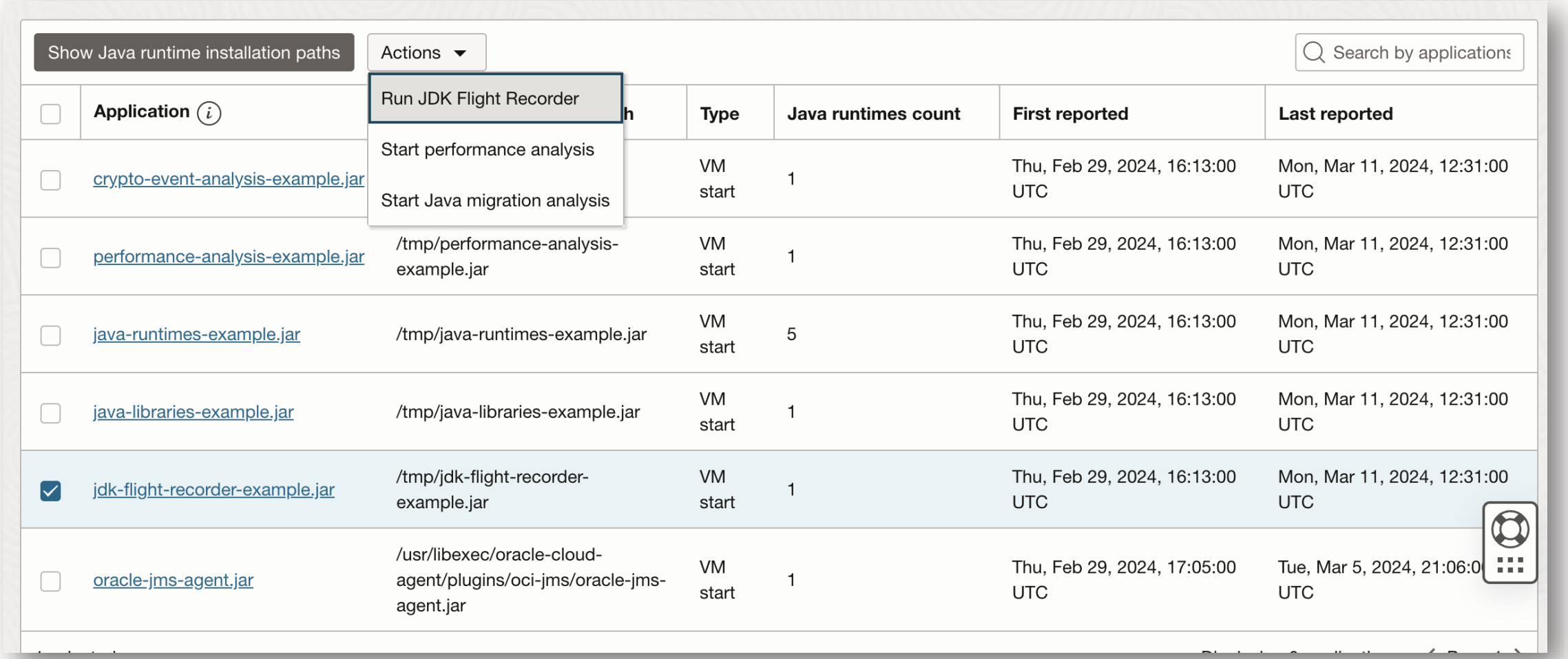
Using Java Management Service (JMS)

- JFR can be started for any Java application that is captured by the JMS.
- For more information about this option, please visit this article.



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

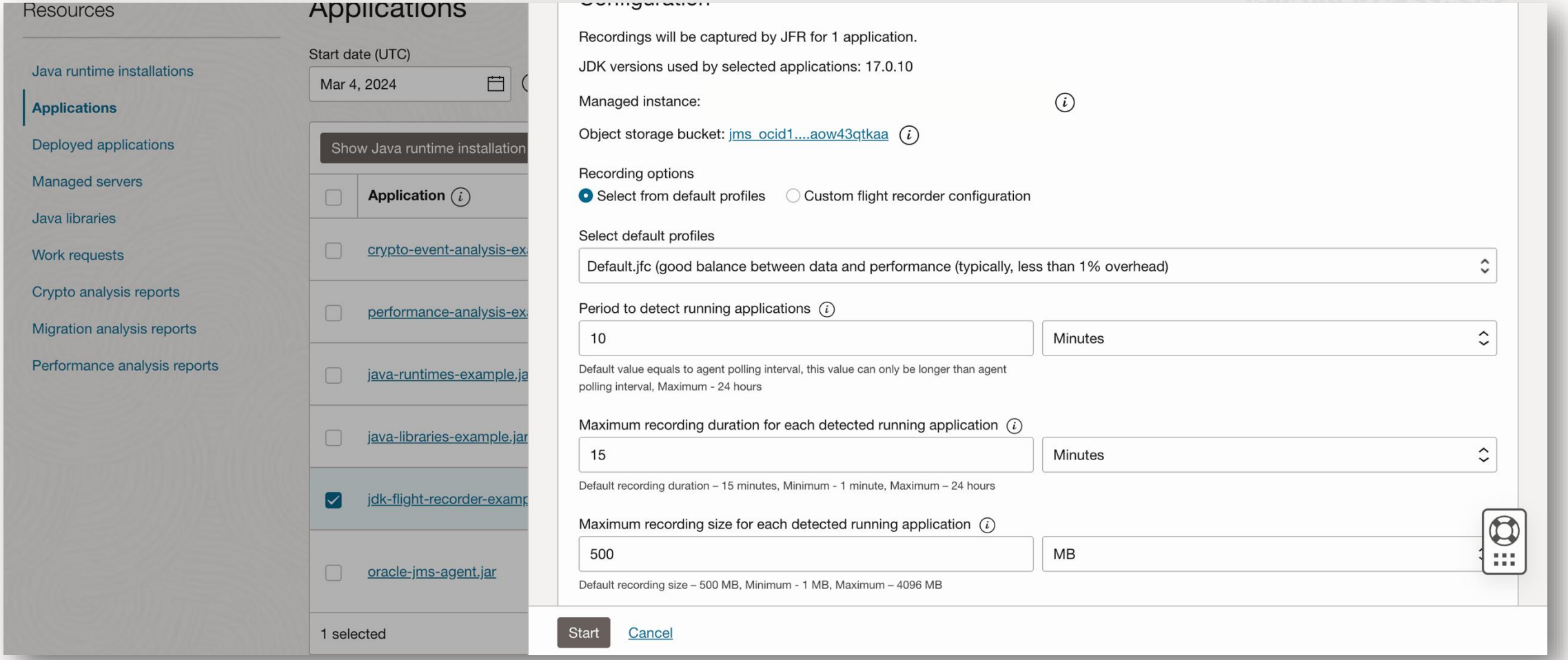
Using Java Management Service (JMS)



Application		Type	Java runtimes count	First reported	Last reported
crypto-event-analysis-example.jar		VM start	1	Thu, Feb 29, 2024, 16:13:00 UTC	Mon, Mar 11, 2024, 12:31:00 UTC
performance-analysis-example.jar	/tmp/performance-analysis-example.jar	VM start	1	Thu, Feb 29, 2024, 16:13:00 UTC	Mon, Mar 11, 2024, 12:31:00 UTC
java-runtimes-example.jar	/tmp/java-runtimes-example.jar	VM start	5	Thu, Feb 29, 2024, 16:13:00 UTC	Mon, Mar 11, 2024, 12:31:00 UTC
java-libraries-example.jar	/tmp/java-libraries-example.jar	VM start	1	Thu, Feb 29, 2024, 16:13:00 UTC	Mon, Mar 11, 2024, 12:31:00 UTC
jdk-flight-recorder-example.jar	/tmp/jdk-flight-recorder-example.jar	VM start	1	Thu, Feb 29, 2024, 16:13:00 UTC	Mon, Mar 11, 2024, 12:31:00 UTC
oracle-jms-agent.jar	/usr/libexec/oracle-cloud-agent/plugins/oci-jms/oracle-jms-agent.jar	VM start	1	Thu, Feb 29, 2024, 17:05:00 UTC	Tue, Mar 5, 2024, 21:06:0 UTC

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using Java Management Service (JMS)



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

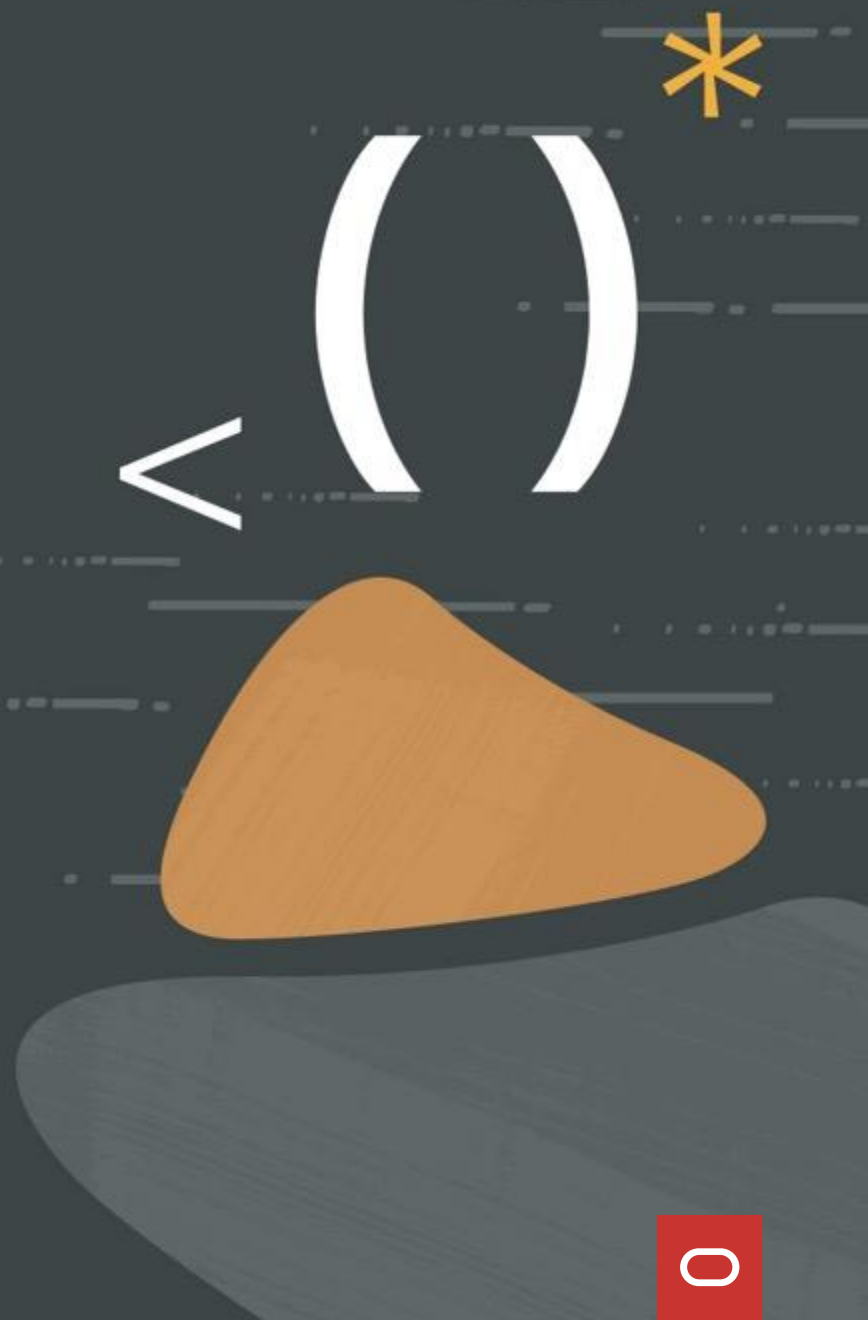
Demo

```
start-jfr
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Continuous Recording



Continuous Recording

Ring Buffer



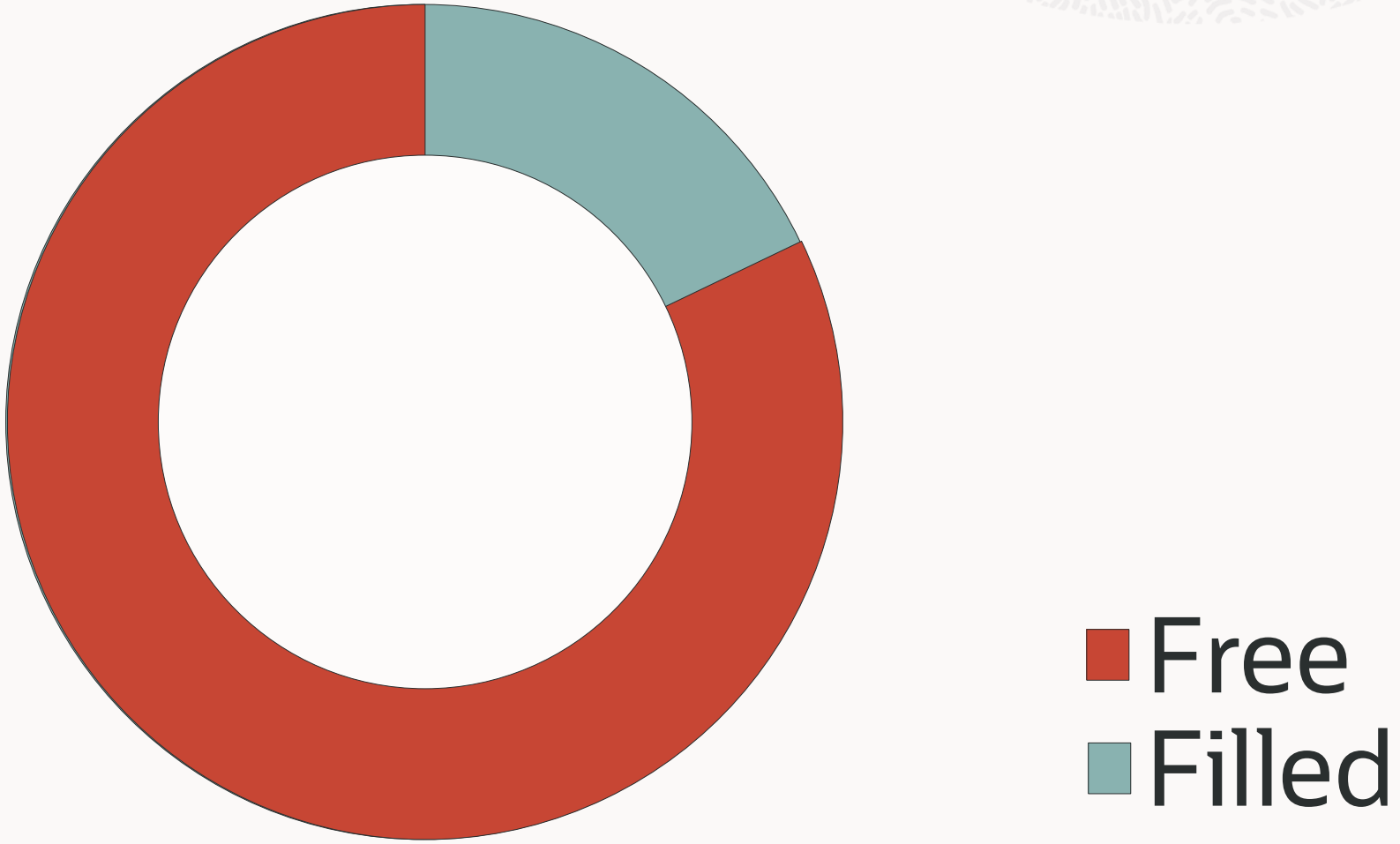
Free

Filled

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Continuous Recording

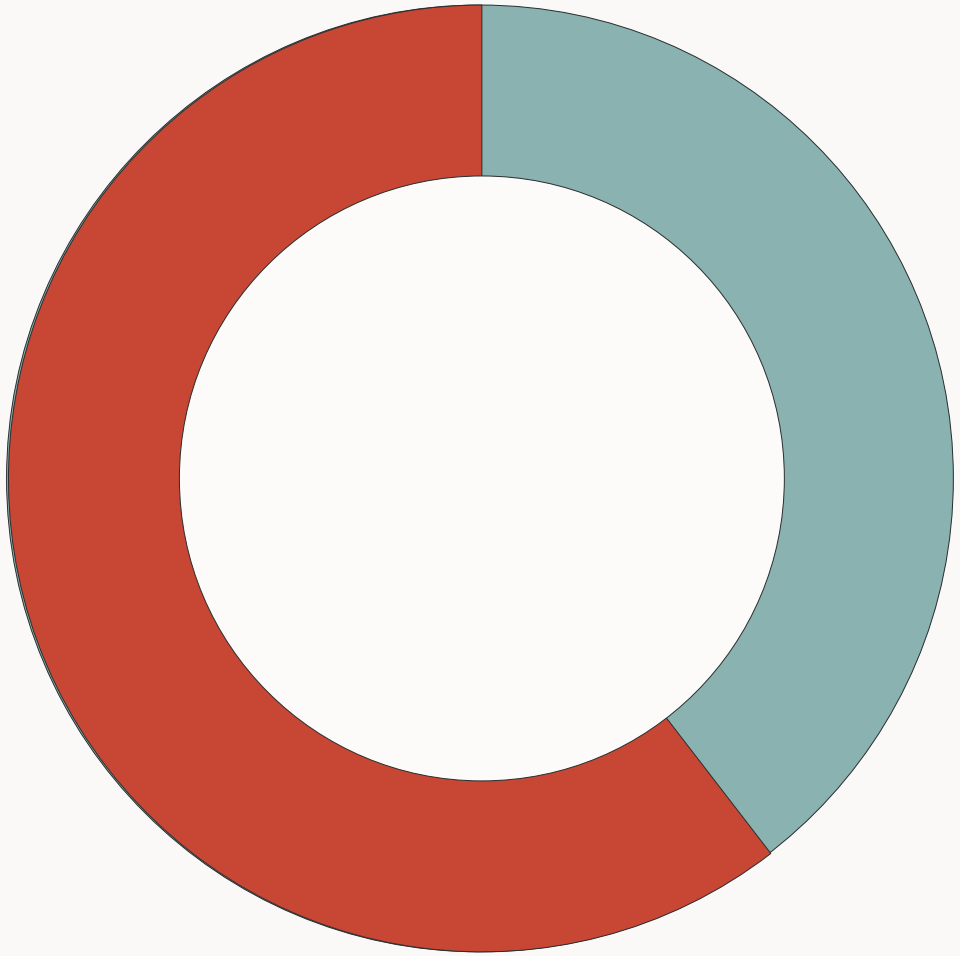
Ring Buffer



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Continuous Recording

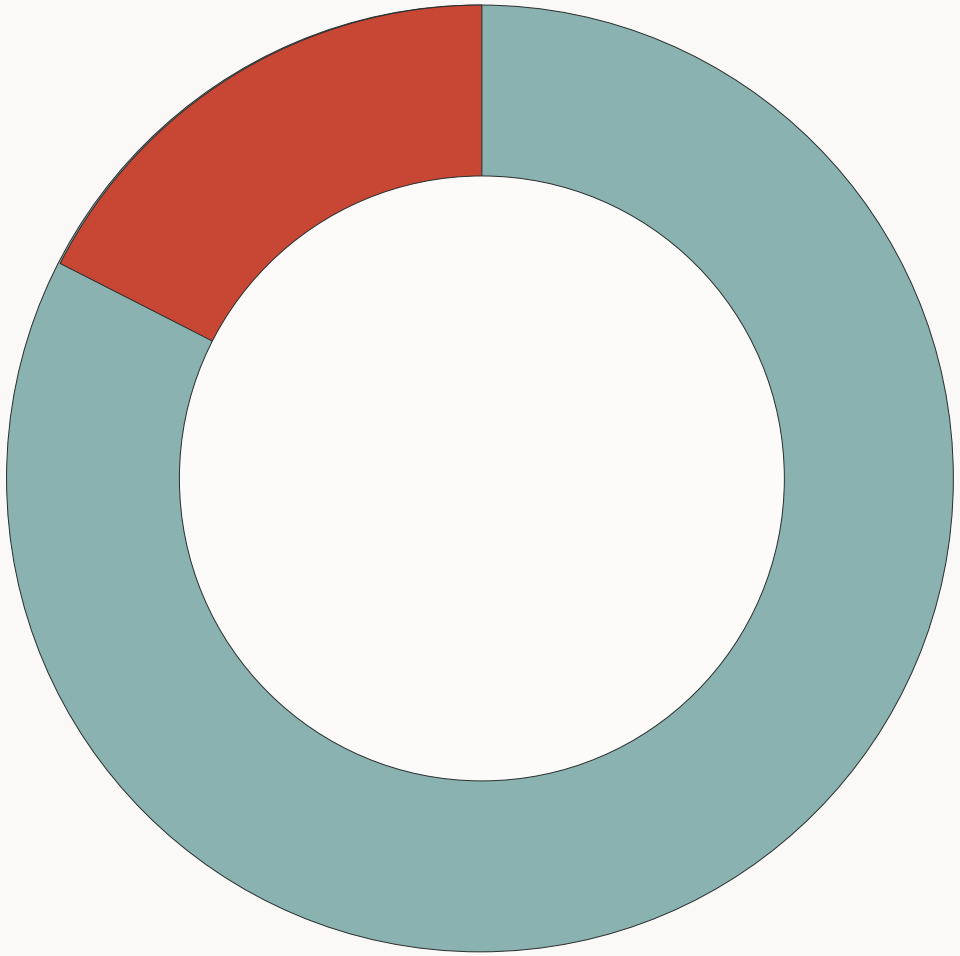
Ring Buffer



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Continuous Recording

Ring Buffer



Free

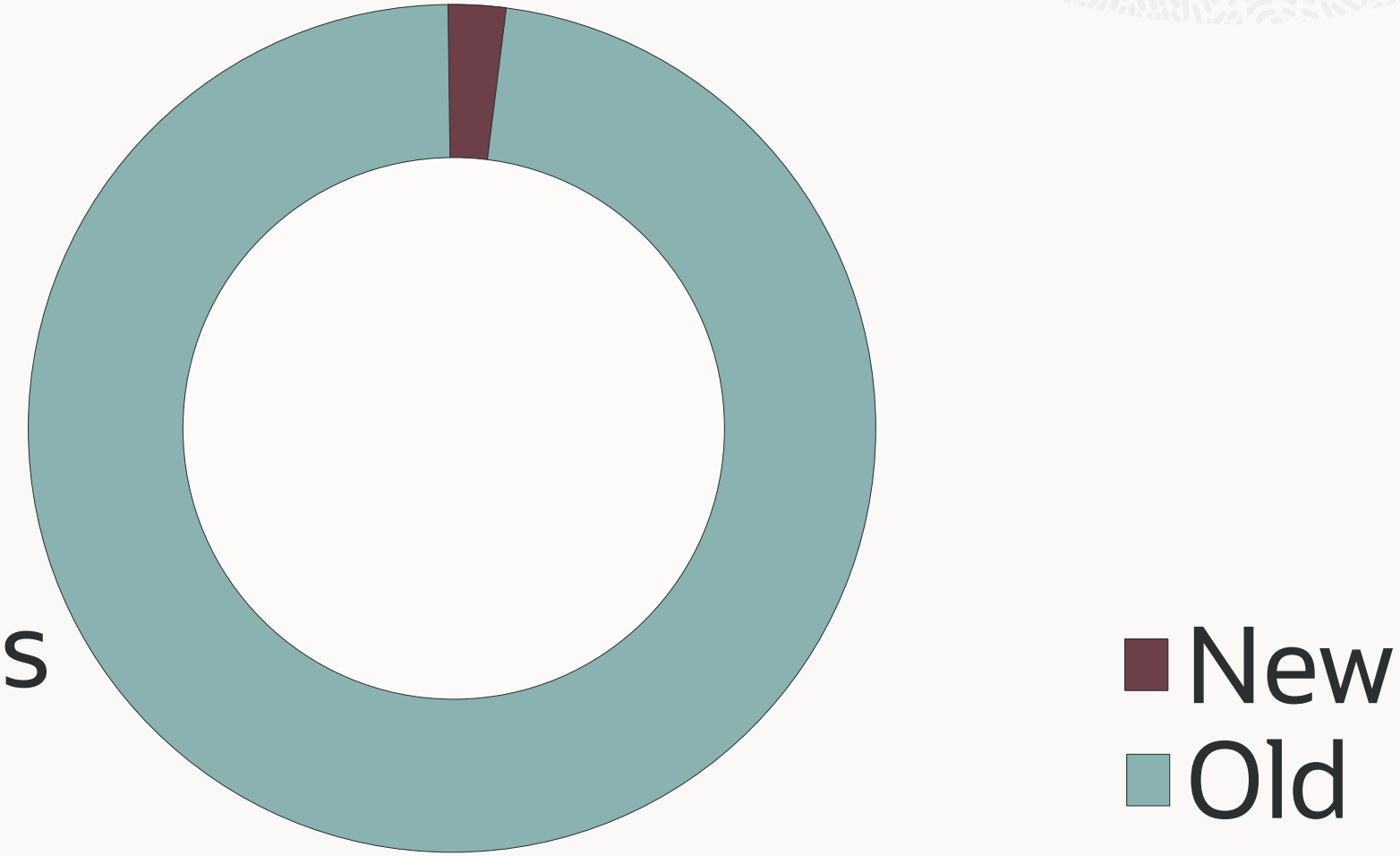
Filled

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Continuous Recording

Ring Buffer

Once full, it starts overwriting the previous data



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Simple Application

```
package demo;

import module java.base;

public final class Main {

    static void main() throws InterruptedException {
        System.out.println("The application will keep running ...");
        new CountDownLatch(1).await();
    }
}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

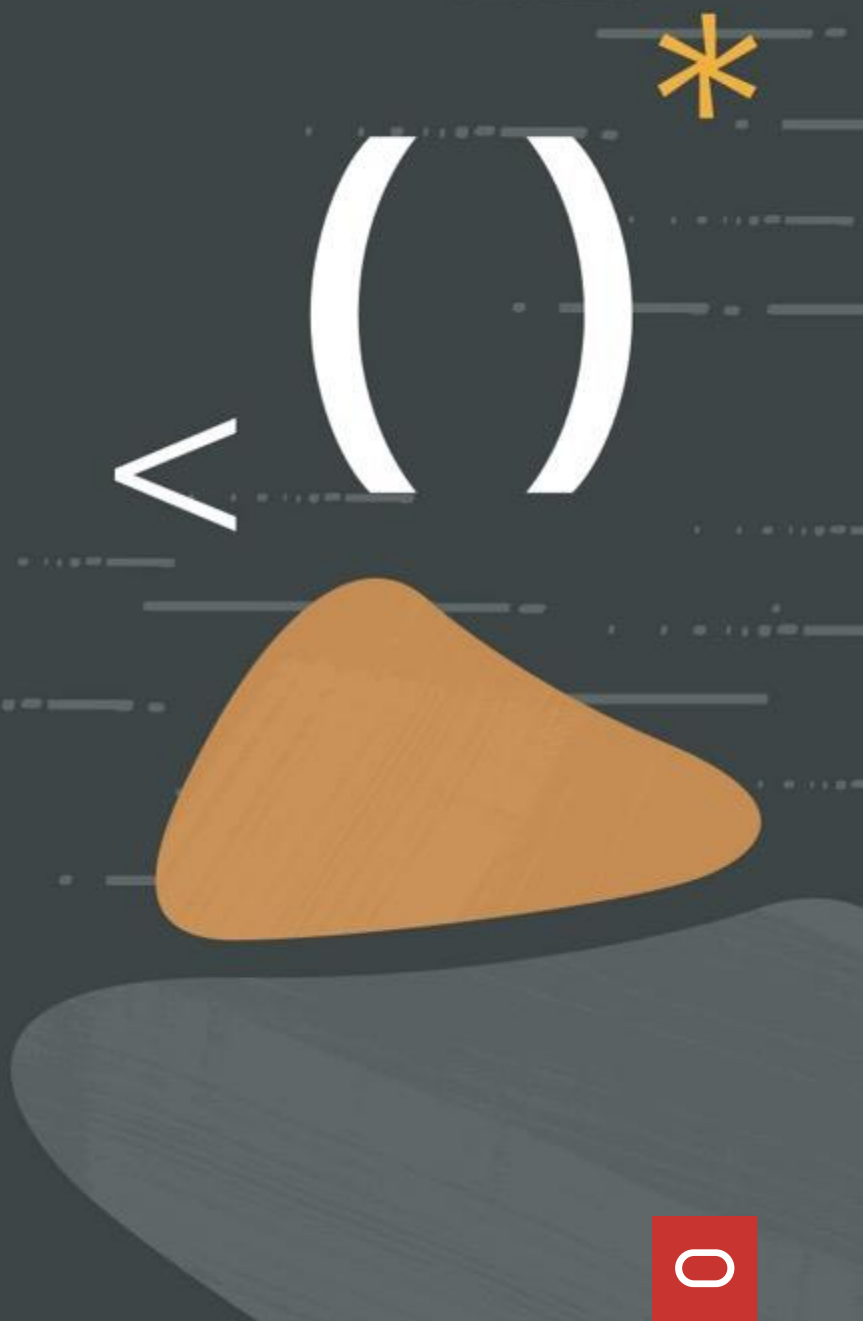


How to stream JFR?

- Using JDK Mission Control (JMC)
- Using `jcmd`
- During application start time
- Programmatically
- Cannot start continuous recording using Java Management Service (JMS)

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)



Using JDK Mission Control (JMC)

Run the application

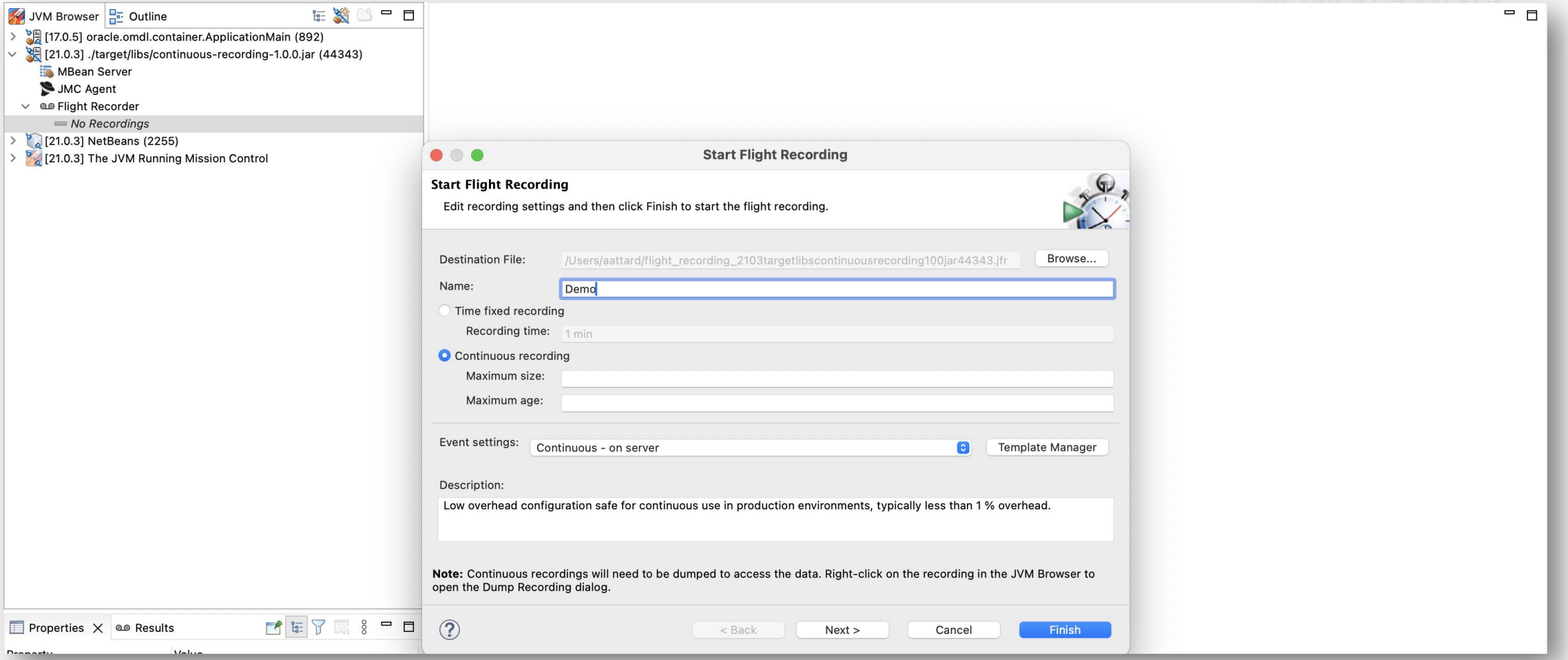
- Start the application without providing any flight recording related options

```
> java -jar 'app.jar'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

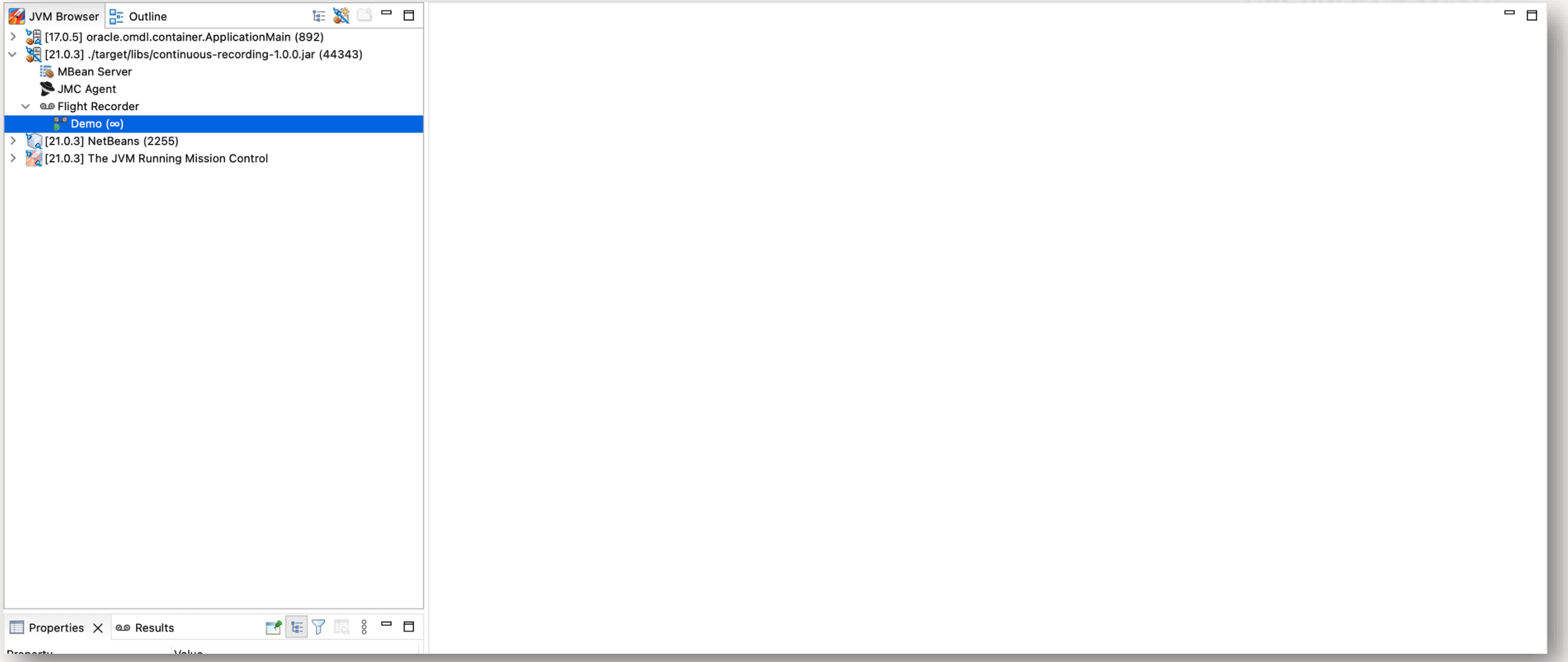
Start flight recording



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

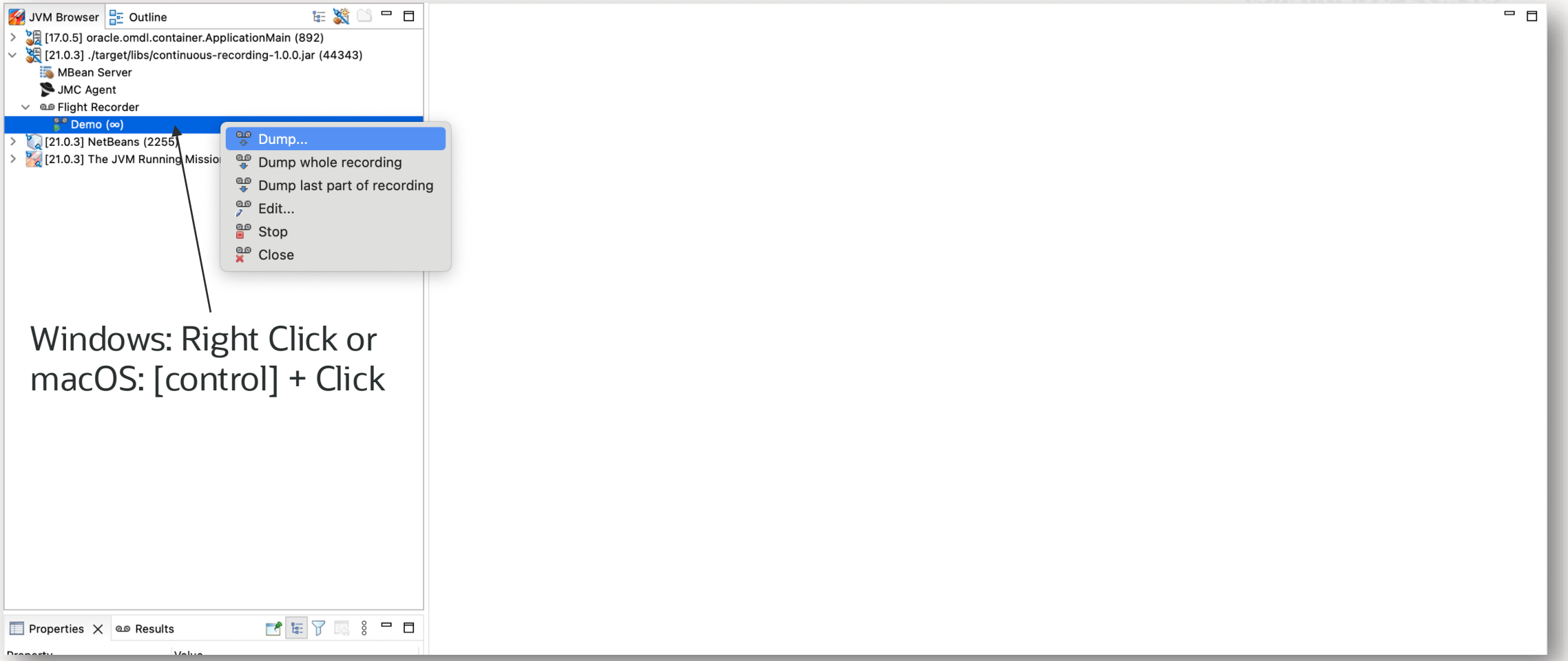
Allow recording data to be collected



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

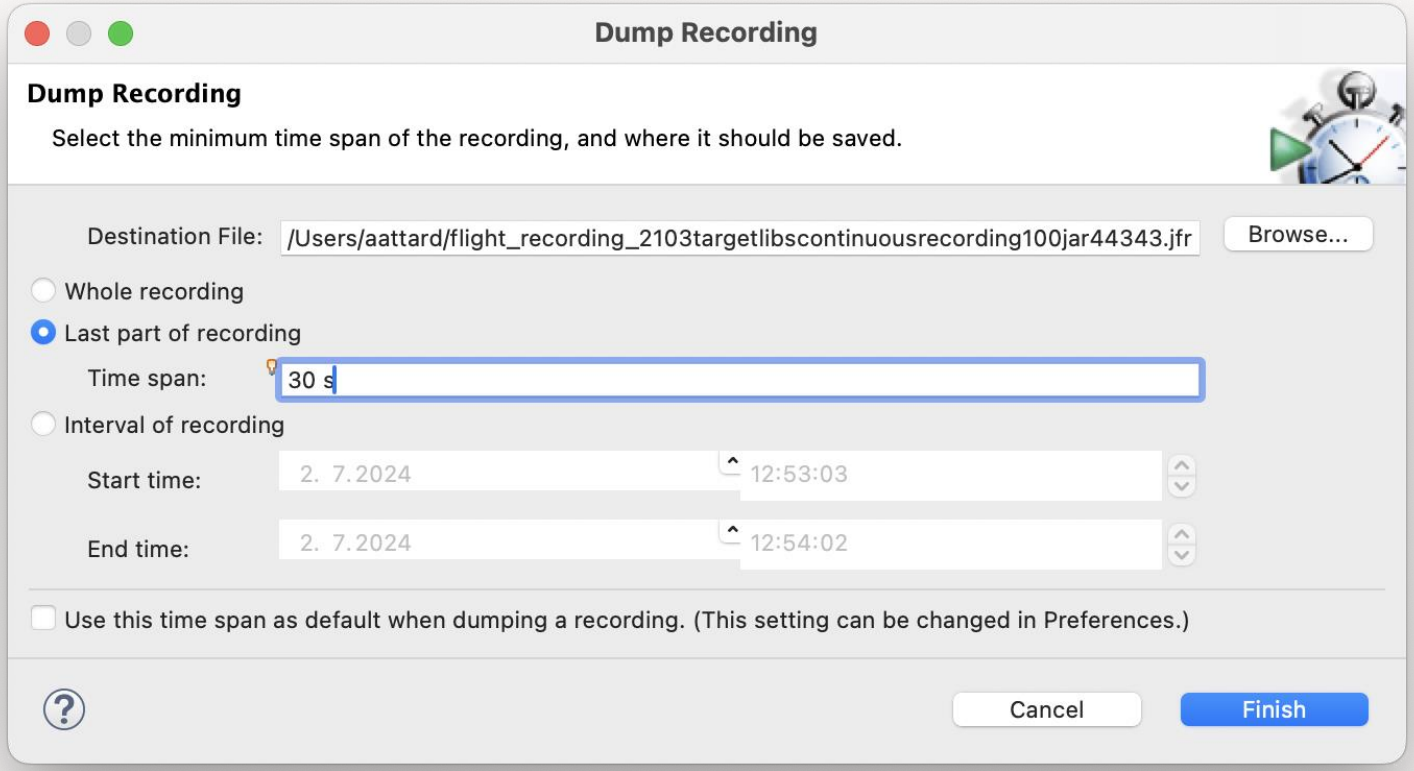
Dump JFR recording



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using JDK Mission Control (JMC)

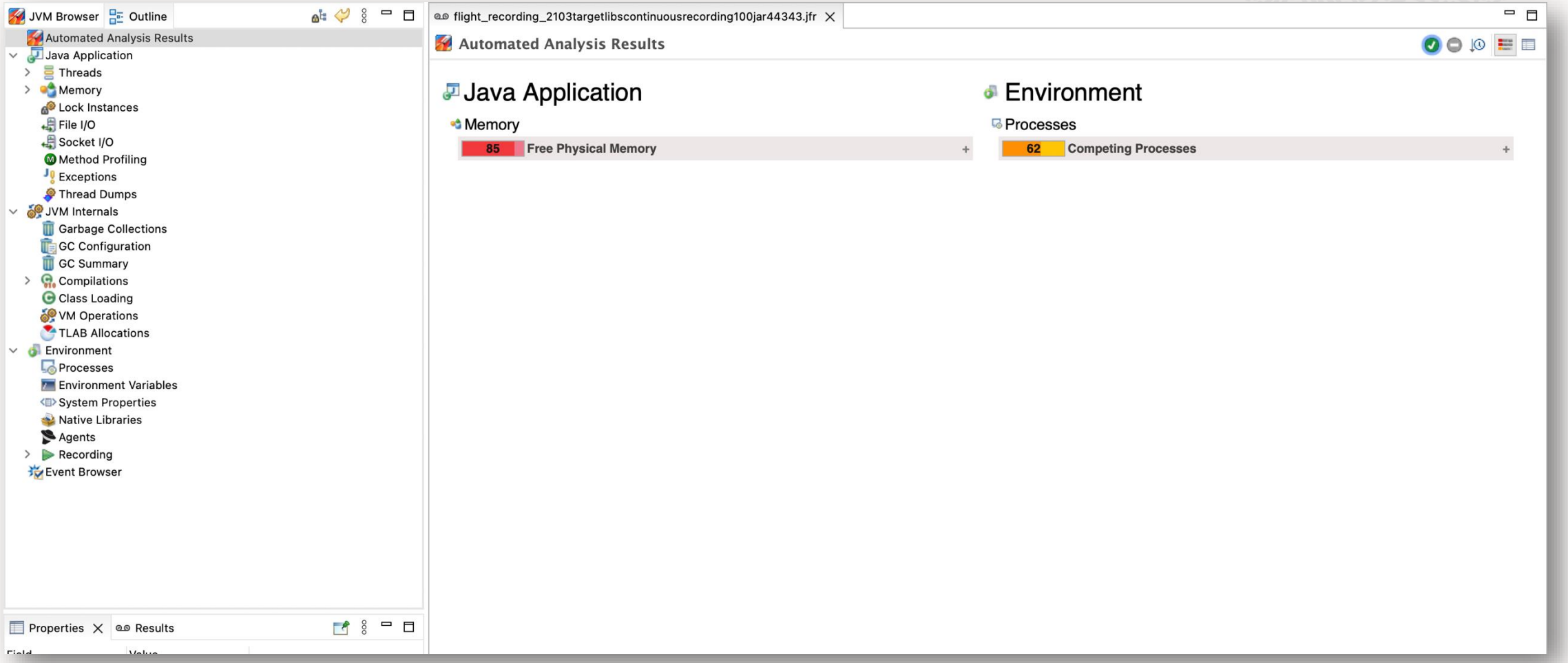
Provide the time span



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

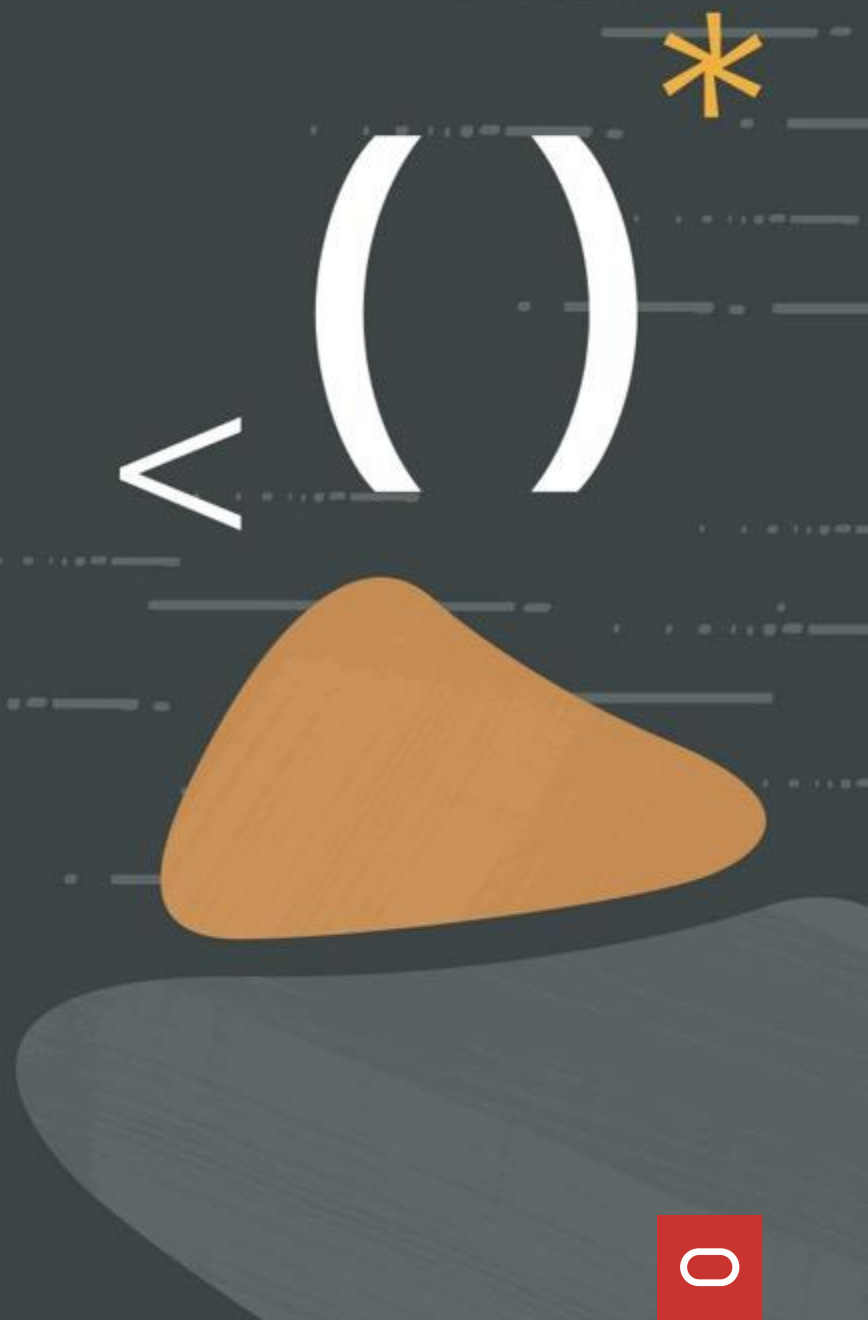
Using JDK Mission Control (JMC)

View the recording



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Using `jcmd`



Using `jcmd`

Run the application

- Start the application without providing any flight recording related options

```
> java -jar 'app.jar'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Using `jcmd`

Start flight recording

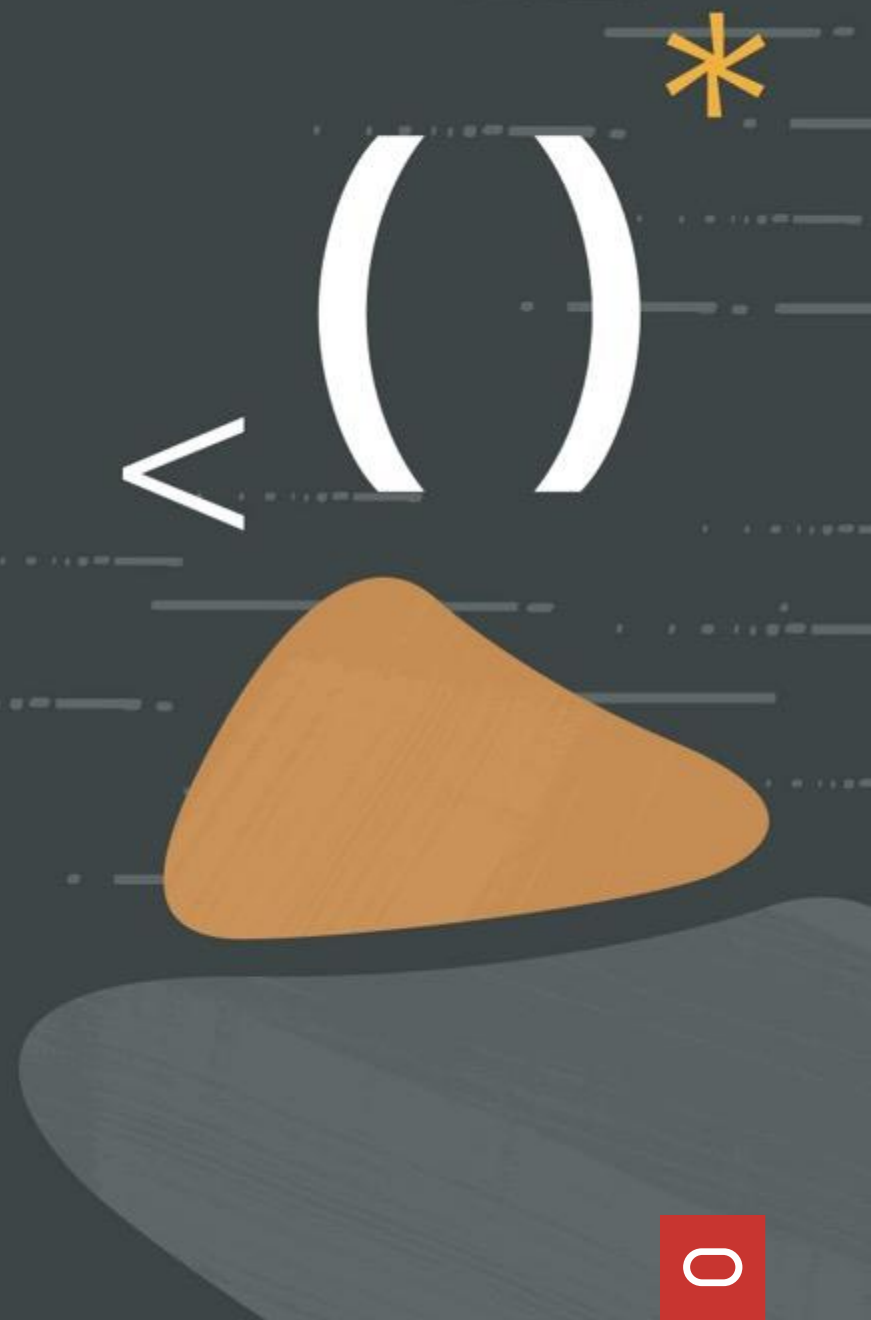
- Start flight recording, without providing the `duration` parameter

```
> jcmd <PID> \
  JFR.start \
  name='Continuous-Recording' \
  filename='./jfr/demo.jfr'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



During application start time



During application start time

Run the application with flight recording enabled

- Start the application by providing the flight recording options but without the `duration` parameter

```
> java \
  -XX:StartFlightRecording=... \
  -jar 'app.jar'
```

Complete example on the next slide

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

During application start time

Run the application and start flight recording

```
> java \
   -XX:StartFlightRecording=
↪delay=5s,
↪name='Continuous-Recording',
↪filename='./jfr/demo.jfr' \
   -jar 'app.jar'
```

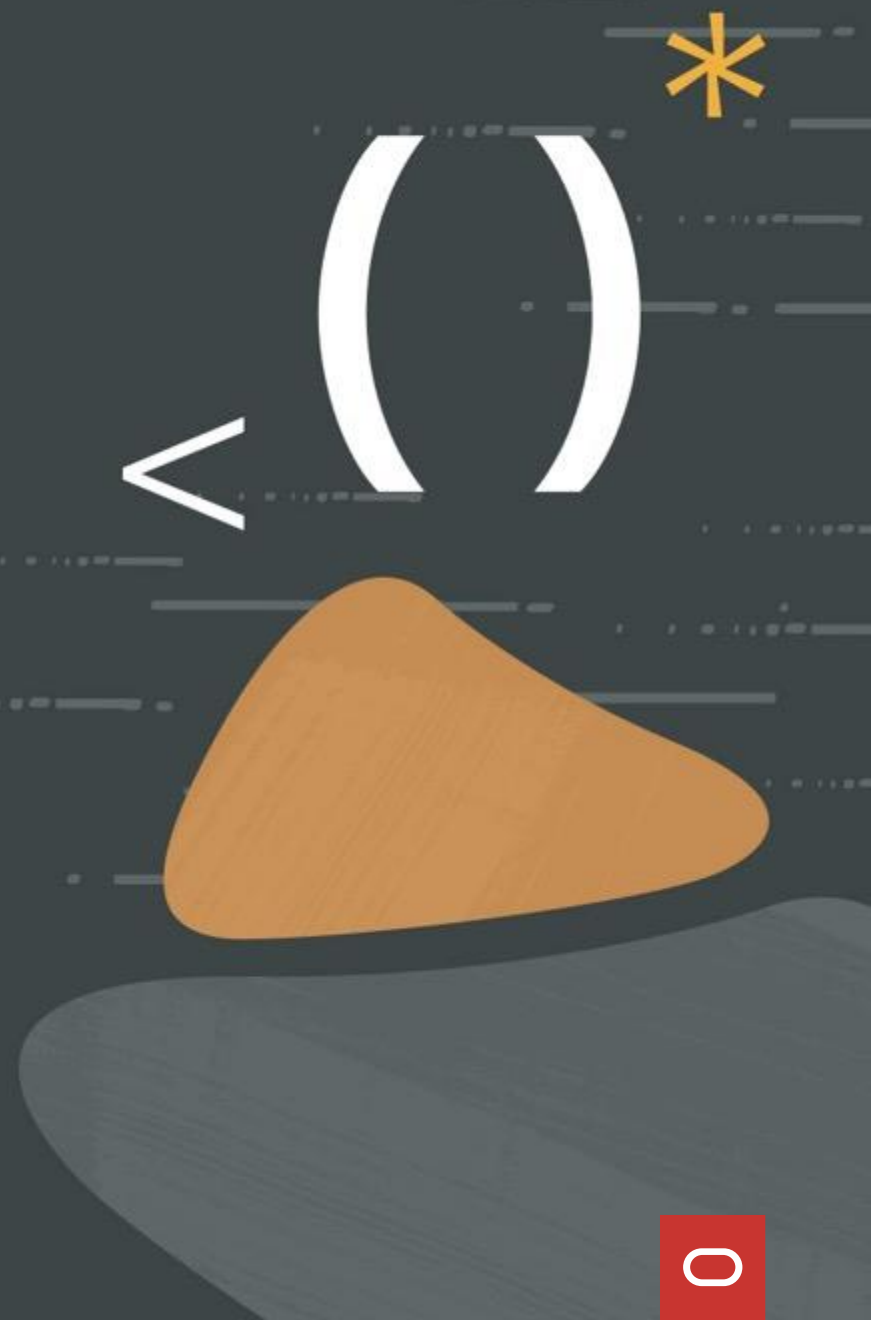
This is one single string

Without the `duration` parameter!

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Programmatically



Programmatically

Modified application

```
try (Recording recording =
      new Recording(Configuration.getConfiguration("default"))) {
    recording.setDestination(Path.of("./jfr/demo-programmatically.jfr"));
    recording.setName("Start-JFR");
    recording.start();

    /* Wait while capturing the events and saved the to the recording. */
    System.out.println("The application will terminate automatically …");
    TimeUnit.SECONDS.sleep(40);
}
/* The recording is completed and written to file. */
/* No more events are captured by this recording. */
```

Without the `duration` parameter!

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Programmatically
Run the application

- Start the application without providing any flight recording related options

```
> java -jar 'app.jar'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Demo

```
continuous-recording
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JFR Print



JFR Print

- Prints the contents of a flight recording file to standard out

```
> jfr print [--xml|--json] \
  [--categories <filters>] \
  [--events <filters>] \
  [--stack-depth <depth>] \
  <file>
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



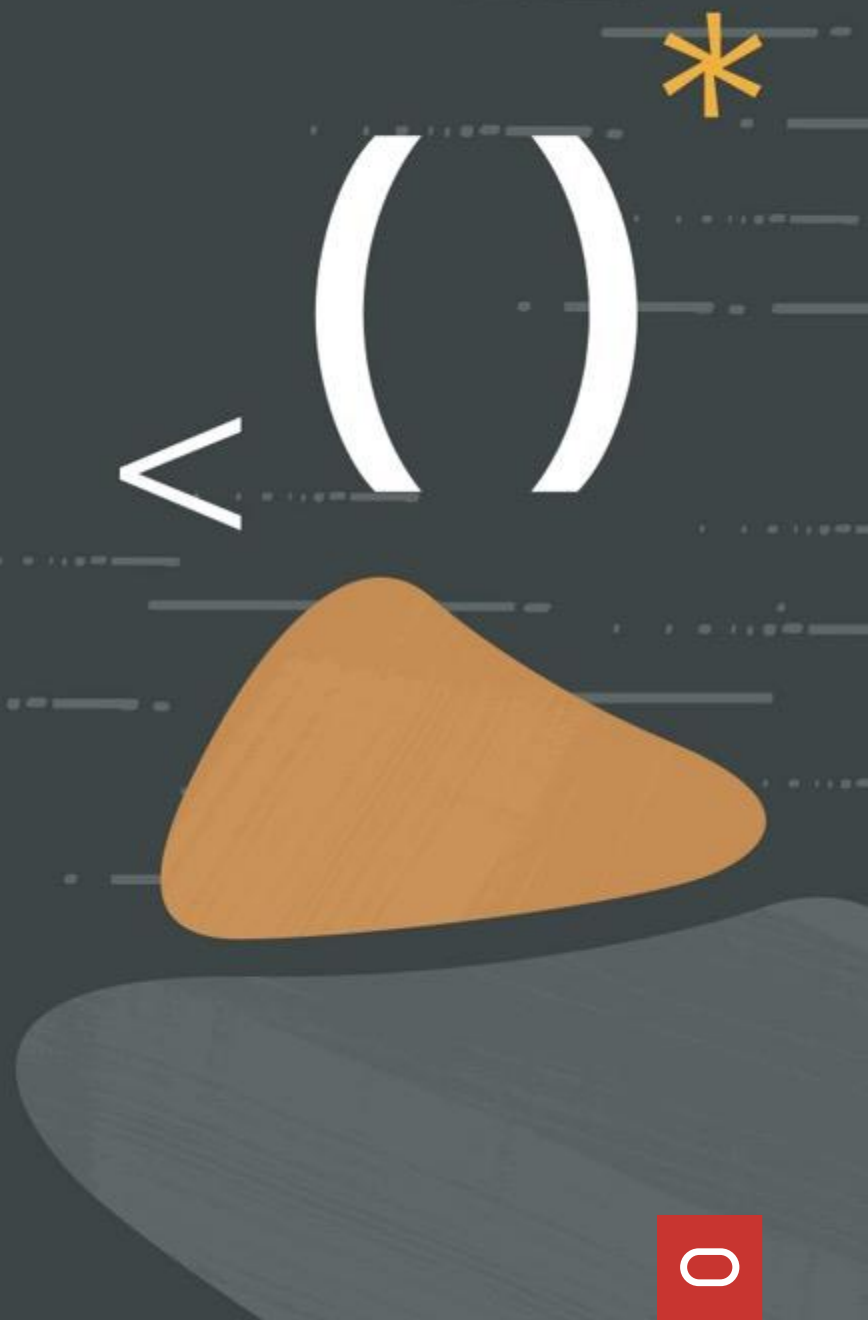
Demo

`jfr-print`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



JFR Metadata



JFR Metadata

- Displays information about events, such as event names, categories and field layout within a flight recording file

```
> jfr metadata \
  [--categories <filters>] \
  [--events <filters>] \
  <file>
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Demo

`jfr-metadata`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



JFR Scrub



JFR Scrub

- Removes sensitive contents from a file or reduces its size

```
> jfr scrub \
  [--exclude-events <filter>] \
  [--exclude-categories <filter>] \
  ... \
  <input-file> [<output-file>]
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



JFR Scrub

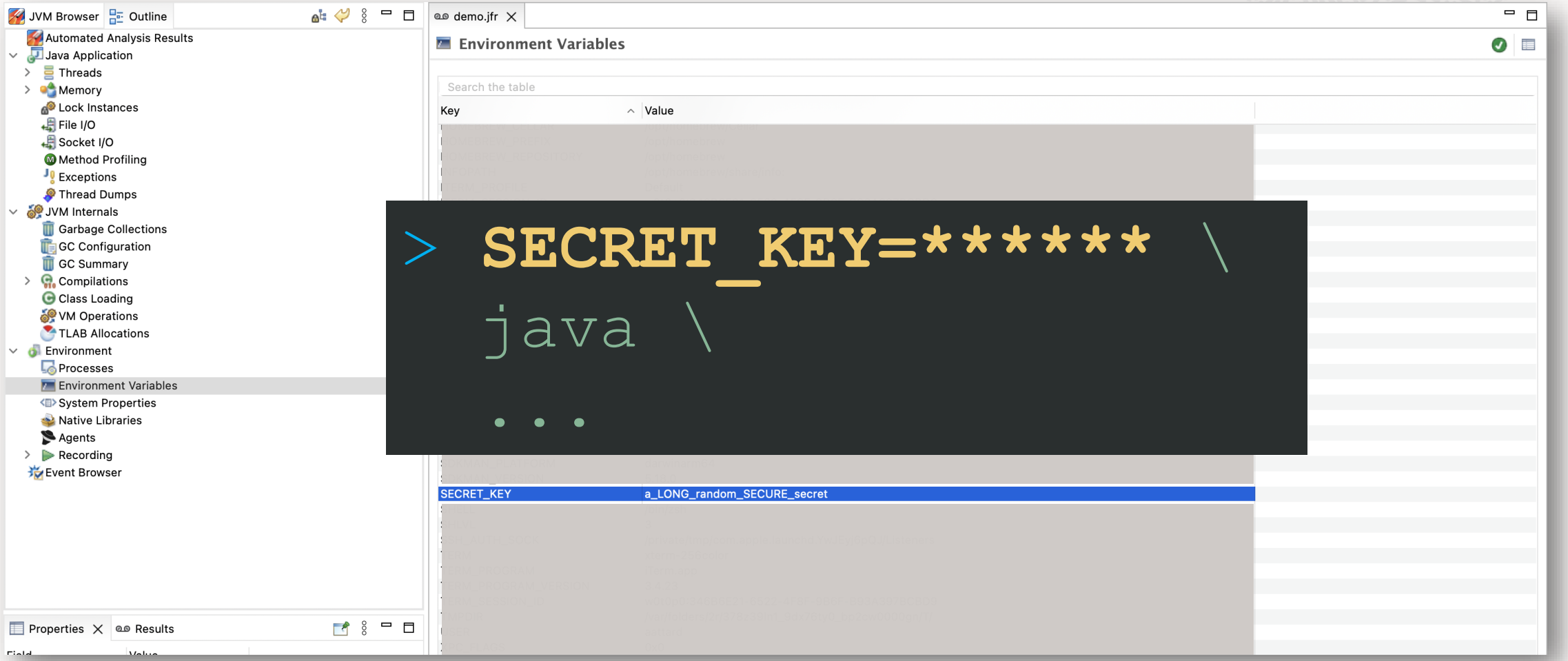
- Consider the following example, where a secret key is passed as an environment variable (`SECRET KEY`)

```
> SECRET_KEY=****** \
  java \
  -XX:StartFlightRecording=... \
  -jar 'app.jar'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JFR Scrub

Environment Variables



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JFR Scrub

Remove environment variables

```
> jfr scrub \
  --exclude-events \
    'jdk.InitialEnvironmentVariable' \
  './jfr/demo.jfr' \
  './jfr/demo-scrubbed-secrets.jfr'
```

Exclude both environment variables and system properties

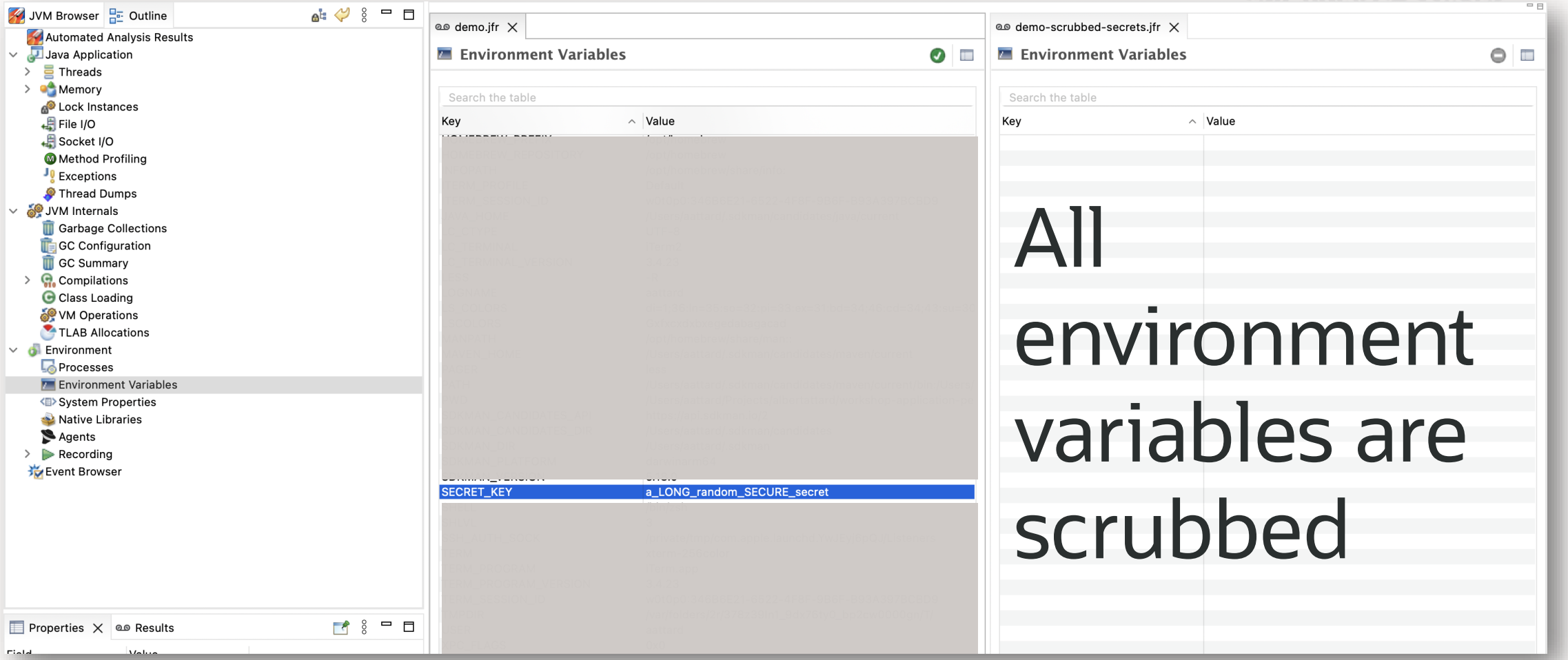
```
--exclude-events 'jdk.InitialEnvironmentVariable,jdk.InitialSystemProperty'
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



JFR Scrub

Environment Variables



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JFR Scrub

Custom scrubber

```
final Predicate<String> filter = Pattern.compile(args[0]).asPredicate();
final Path input = Path.of(args[1]);
final Path output = Path.of(args[2]);
final Set<String> events = Set.of("jdk.InitialSystemProperty",
                                  "jdk.InitialEnvironmentVariable");

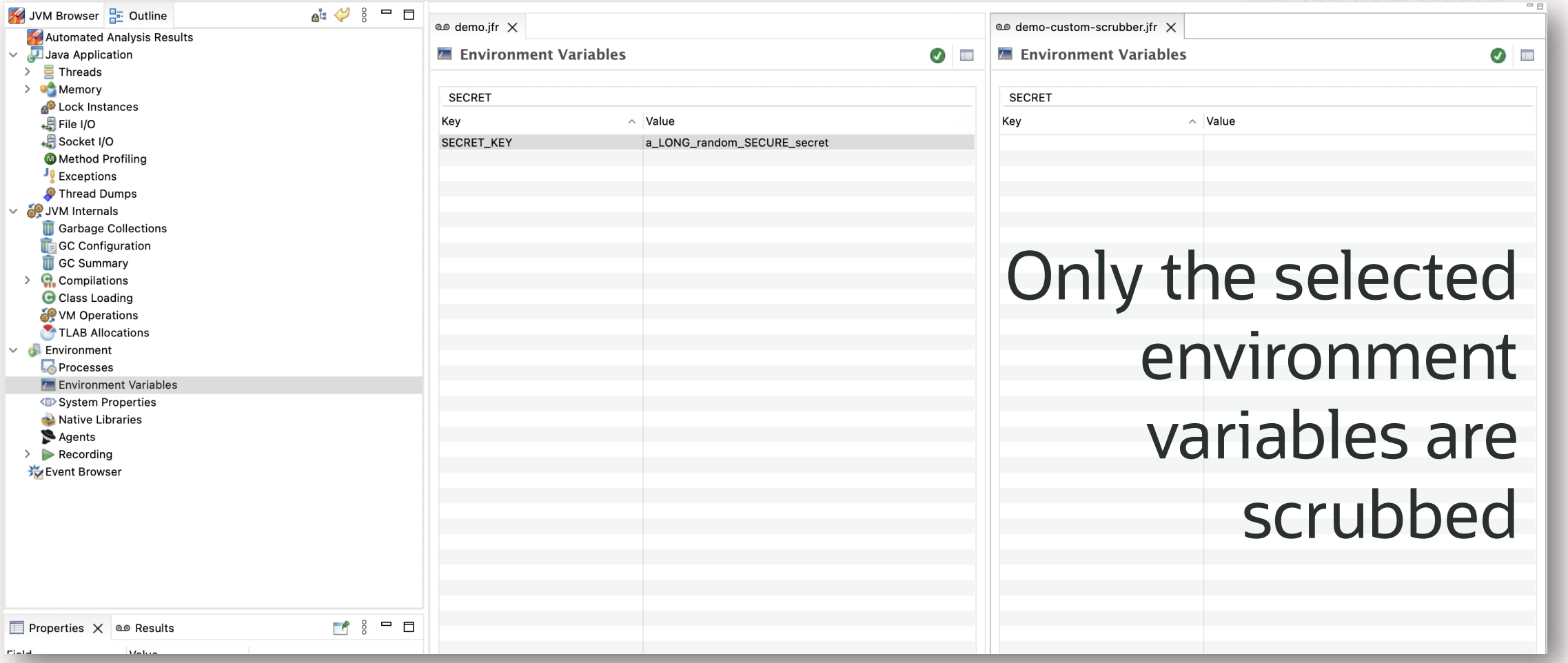
try (RecordingFile file = new RecordingFile(input)) {
    file.write(output, event -> {
        final String name = event.getEventType().getName();
        if (events.contains(name)) {
            final String value = event.getString("key");
            return !filter.test(value);
        }
        return true;
    });
} catch (final IOException e) {…}
```

```
.*SECRET.*
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JFR Scrub

Custom scrubber



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JEP 536: JFR In-Process Data Redaction

- Java 27 introduced a new feature where sensitive information is redacted at source
- Events that contain key-value pairs such as: *api*key*, *password*, *client*secret*, *credential*, and others are redacted automatically

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

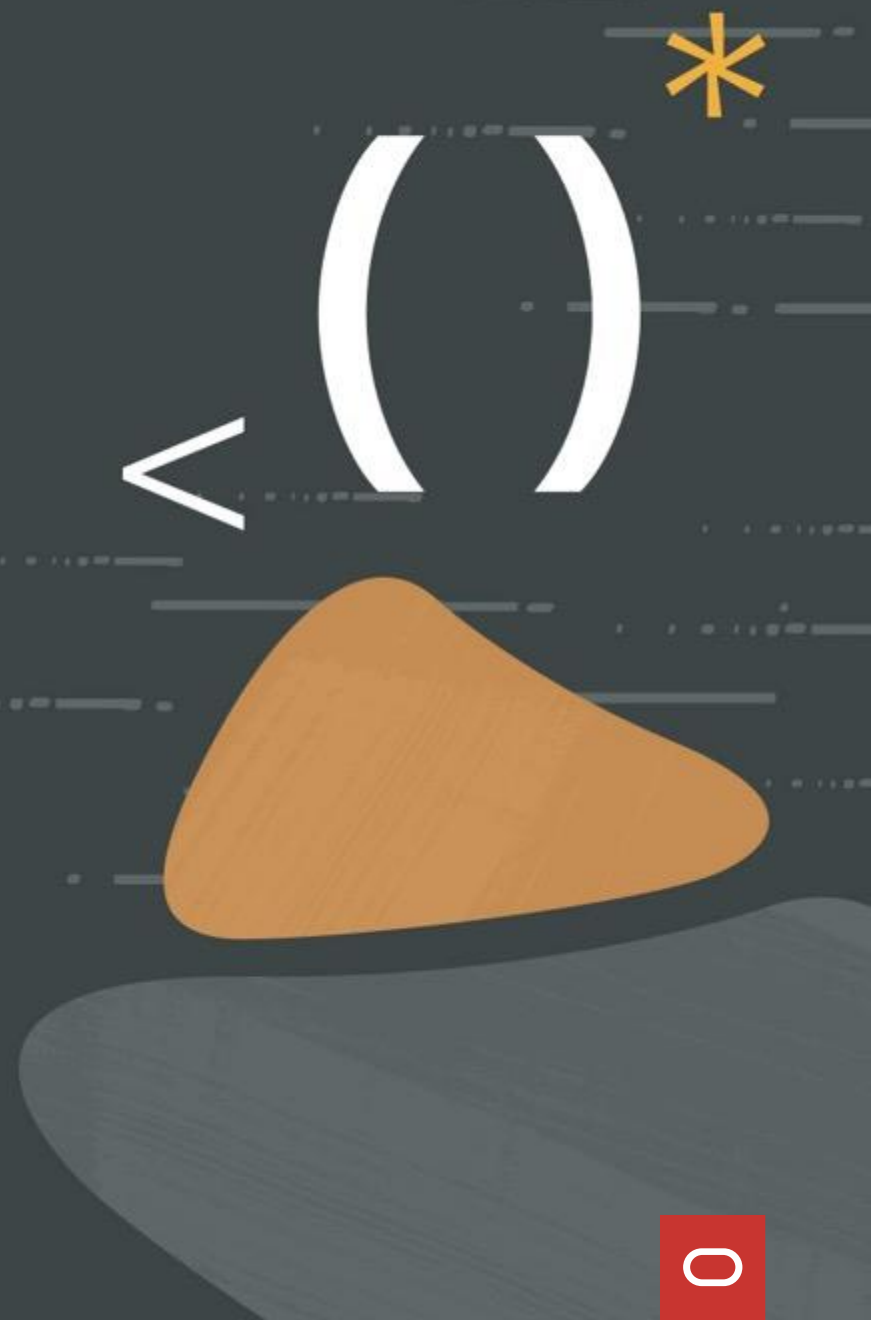
Demo

`jfr-scrub`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



JFR Configuration



JFR Configuration

- JFR comes with two configurations, *default* and *profile*
- Custom configurations can be created

```
> jfr configure --interactive
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



JFR Configuration
Default configuration

- Provides a good balance between amount of information collected and low overhead (under 1%)
- Configuration file can be found at: `${JAVA_HOME}/lib/jfr/default.jfc`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



JFR Configuration
Profile configuration

- Provides more information but at a slightly higher overhead cost
- Do not hesitate to switch to this configuration when more information is required
- Found at

```
${JAVA_HOME}/lib/jfr/profile.jfc
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JFR Configuration

Command line options

Type	Options
Default	`-XX:StartFlightRecording=...` `-XX:StartFlightRecording=setting=default,...`
Profile	`-XX:StartFlightRecording=setting=profile,...`
Custom	`-XX:StartFlightRecording=setting=<PATH>,...`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JFR Configuration

Run the application

```
> java \
  -XX:StartFlightRecording=
↪duration=30s,
↪delay=5s,
↪name='Configuration',
↪filename='jfr/demo-custom.jfr',
↪settings='jfr/custom.jfc' \
  -jar 'app.jar'
```

This is one single string

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Demo

`jfr-configuration`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Custom JFR Events



Custom JFR Events

Oracle Java support

- Oracle Java 8 does not support custom JFR Events
- Enterprise Performance Pack (EPP) supports custom JFR Event
- Oracle Java 11 and newer versions are required

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Custom JFR Events

Custom event

```
import module jdk.jfr;

@Name("demo.customevents.WorkEvent")
@Label("Work Event")
@Description("Data from one loop run in the worker thread")
@Category({"Demo", "Custom Events"})
public final class WorkEvent extends Event {

    @Label("Transaction Id")
    @Description("The unique id that identifies the transaction")
    private long transactionId;

    public void transactionId(final long transactionId) {
        this.transactionId = transactionId;
    }
}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Custom JFR Events

Create custom event

```
final WorkEvent event = new WorkEvent();
try {
    event.begin();

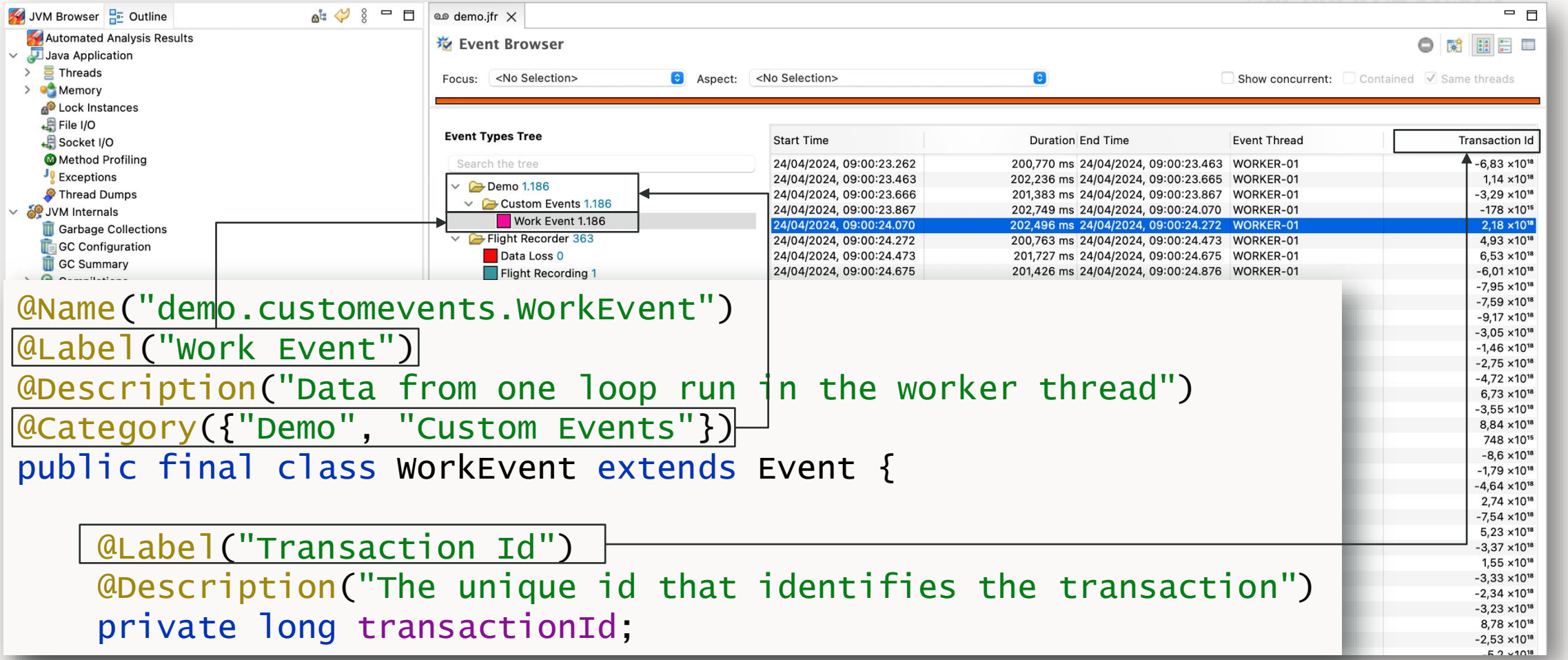
    /* Do Important things */
    final long transactionId = generator.nextLong();
    sleepForTwoHundredMilliseconds();

    /* Set custom values */
    event.transactionId(transactionId);
} finally {
    event.end();
    event.commit();
}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Custom JFR Events

View custom event



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Custom JFR Events

A functional approach

```
public static void accept(final Consumer<WorkEvent> consumer) {
    requireNonNull(consumer);

    final WorkEvent event = new WorkEvent();
    event.begin();
    try {
        consumer.accept(event);
    } finally {
        event.commit();
    }
}
```

```
WorkEvent.accept(event -> {
    /* Do Important things */
    final long transactionId = generator.nextLong();
    sleepForTwoHundredMilliseconds();

    /* Set custom values */
    event.transactionId(transactionId);
});
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Custom JFR Events

Additional annotations

Annotation	Description
`@Enabled(false)`	Specifies whether the event is recorded
`@Threshold("2 s")`	Specifies the duration below which an event is not recorded
`@Period("30 s")`	Specifies the interval at which the event is emitted, if it is periodic
`@StackTrace(false)`	Specifies whether the stack trace from the `Event.commit()` method is recorded

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Demo

`custom-events`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Crypto Events





Monitoring Java Application Security with JDK tools and JFR Events

dev.java

https://dev.java/learn/security/monitor/

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Crypto Events

- JFR can reveal cryptographic information, such as the protocols used, and help you detect the use of insecure algorithms
- While newer versions of Java disable insecure protocols by default, older versions of Java still support them

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

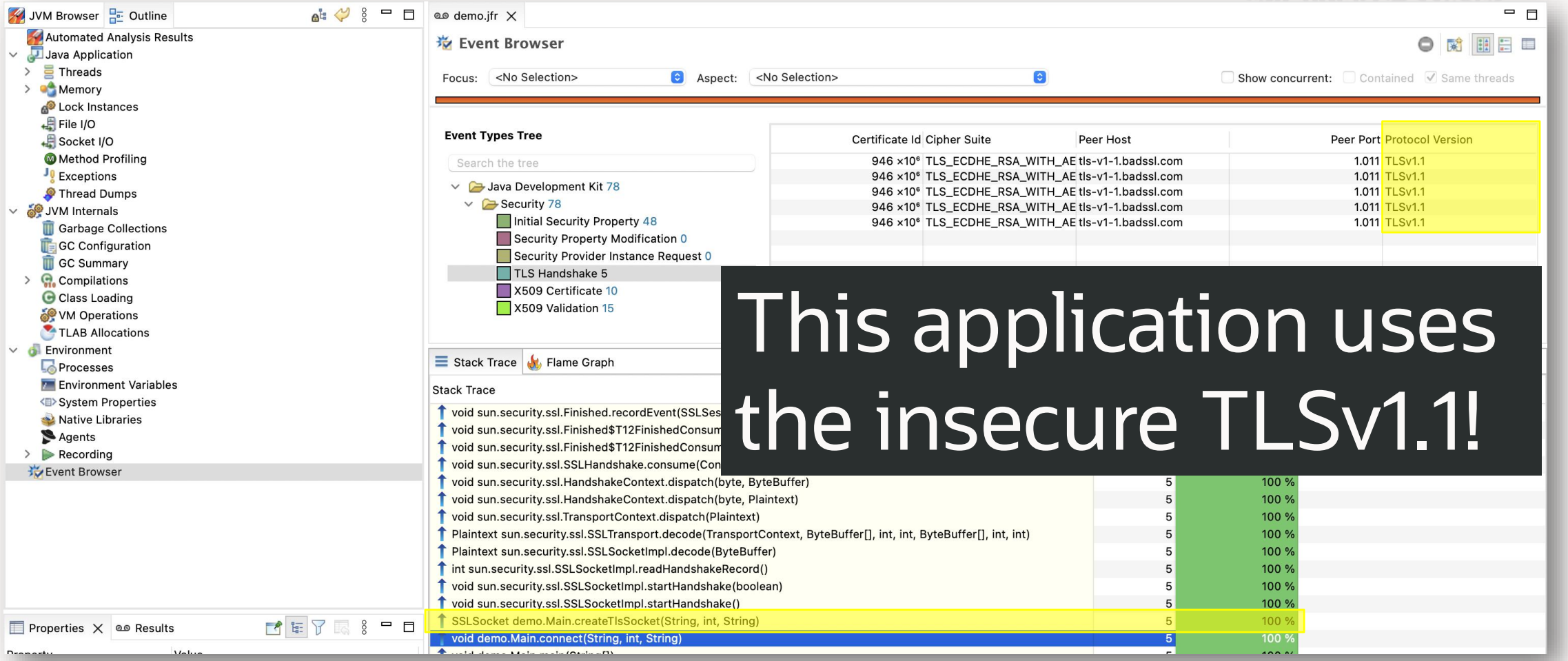
Crypto Events
Enable additional events

- `jdk.TLSHandshake` captures core information on every TLS handshake performed by the JDK
- `jdk.X509Certificate` captures information about every X.509 cert generated by the JDK security libraries

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Crypto Events

jdk.TLSHandshake



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Based on these observations, where should we look first?



Crypto Events
jdk.TLSHandshake

```
package demo;

import …;

public final class Main {

    public static void main(final String[] args) {…}

    private static void connect(final String host, int

    private static SSLSocket createTlsSocket(final Str
        final SocketFactory factory = SSLSocketFactory
        final SSLSocket socket = (SSLSocket) factory.
        /* The insecure TLS 1.1 is used for demonstra
           do not use this in production as this prot
        if ("TLSv1.1".equals(protocol)) {
            System.out.println("Making use of insecure cryptographic protocol " + protocol + ". Do not use in production!!");
        }
        socket.setEnabledProtocols(new String[]{protocol});
        socket.setSoTimeout((int) TimeUnit.SECONDS.toMillis(5));
        socket.startHandshake();
        return socket;
    }

    /* Other methods removed for brevity */
}
```

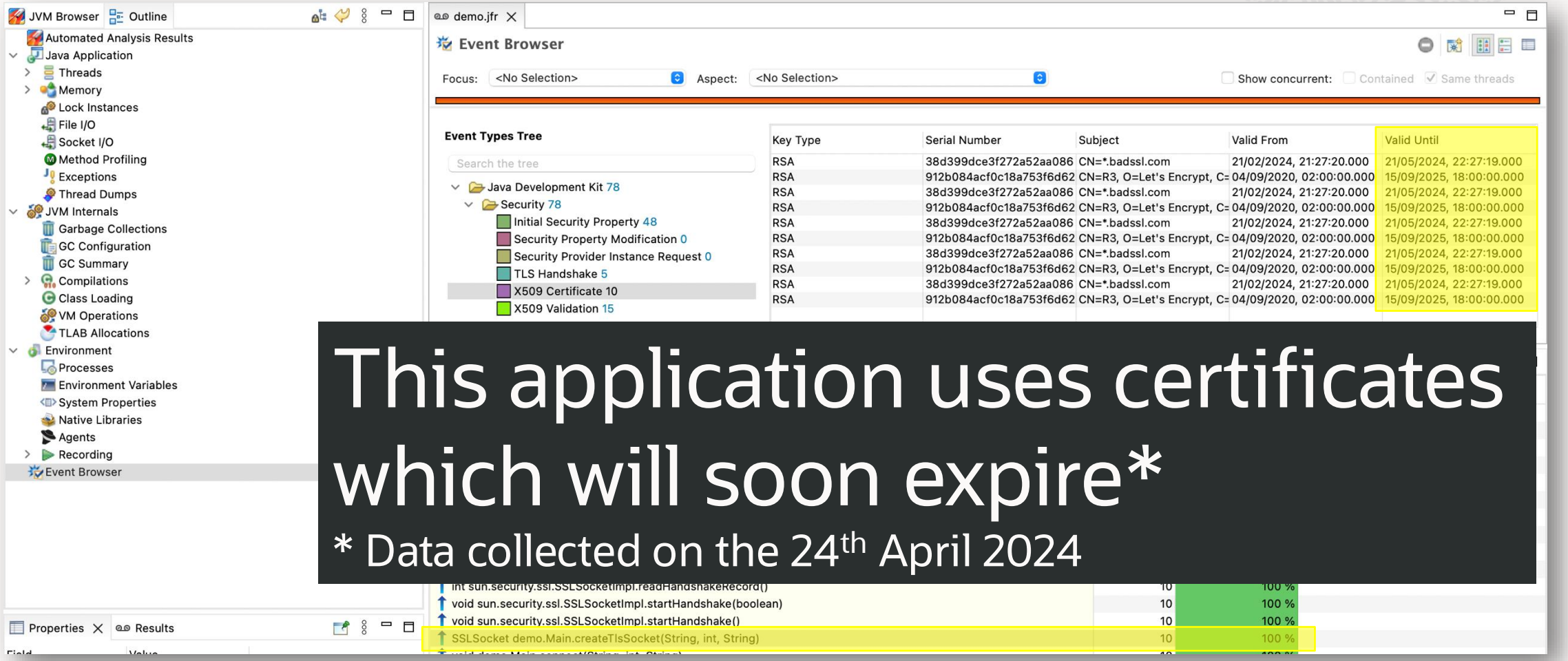
Stack Trace | Flame Graph

Stack Trace	Samples	Percentage
void sun.security.ssl.Finished.recordEvent(SSLSessionImpl)	5	100 %
void sun.security.ssl.Finished$T12FinishedConsumer.onConsumeFinished(ClientHandshakeContext, ByteBuffer)	5	100 %
void sun.security.ssl.Finished$T12FinishedConsumer.consume(ConnectionContext, ByteBuffer)	5	100 %
void sun.security.ssl.SSLHandshake.consume(ConnectionContext, ByteBuffer)	5	100 %
void sun.security.ssl.HandshakeContext.dispatch(byte, ByteBuffer)	5	100 %
void sun.security.ssl.HandshakeContext.dispatch(byte, Plaintext)	5	100 %
void sun.security.ssl.TransportContext.dispatch(Plaintext)	5	100 %
Plaintext sun.security.ssl.SSLTransport.decode(TransportContext, ByteBuffer[], int, int, ByteBuffer[], int, int)	5	100 %
Plaintext sun.security.ssl.SSLSocketImpl.decode(ByteBuffer)	5	100 %
int sun.security.ssl.SSLSocketImpl.readHandshakeRecord()	5	100 %
void sun.security.ssl.SSLSocketImpl.startHandshake(boolean)	5	100 %
void sun.security.ssl.SSLSocketImpl.startHandshake()	5	100 %
SSLSocket demo.Main.createTlsSocket(String, int, String)	5	100 %
void demo.Main.connect(String, int, String)	5	100 %
void demo.Main.main(String[])	5	100 %

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Crypto Events

jdk.X509Certificate



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

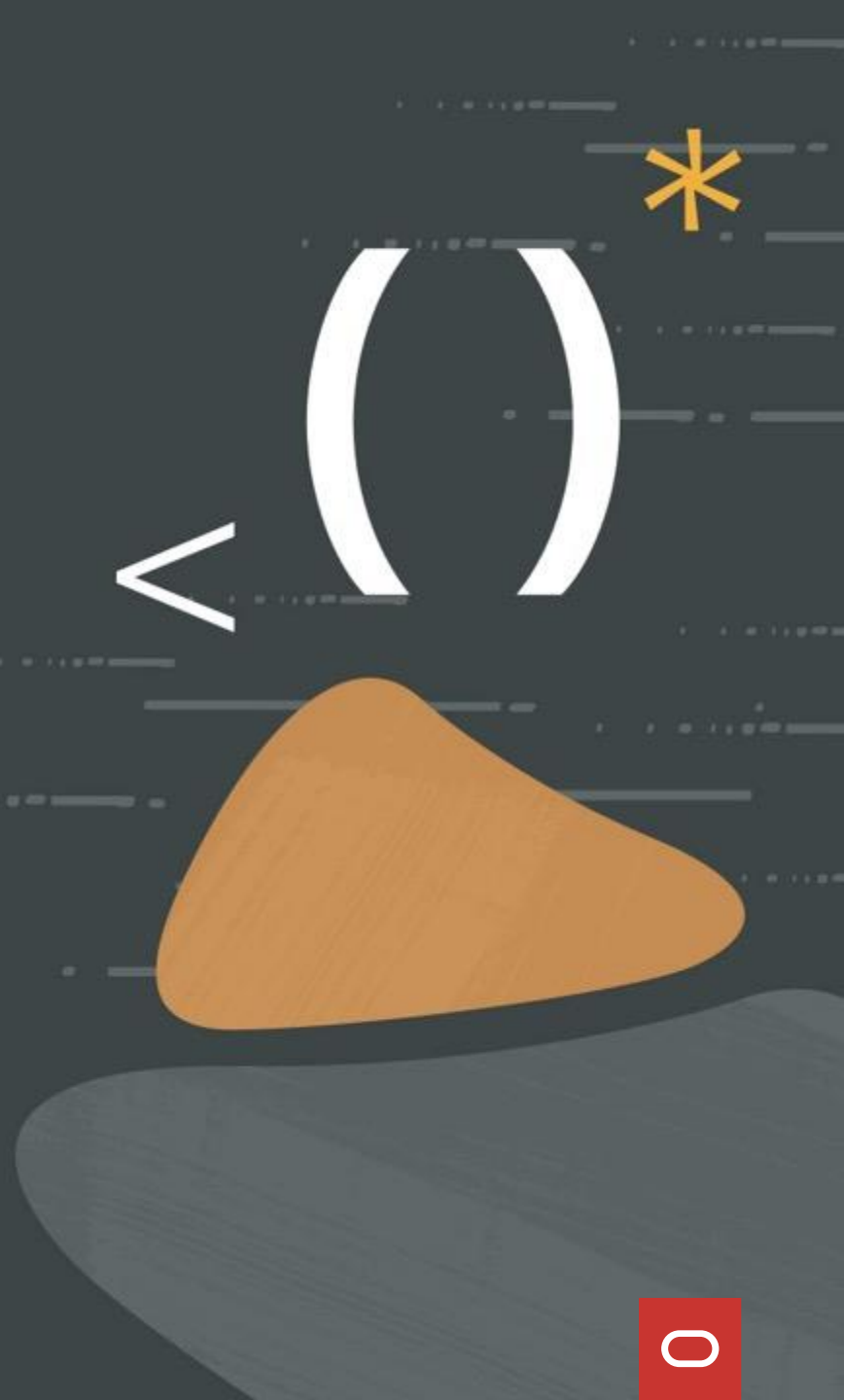
Demo

`crypto-events`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Hot Methods



Hot Methods

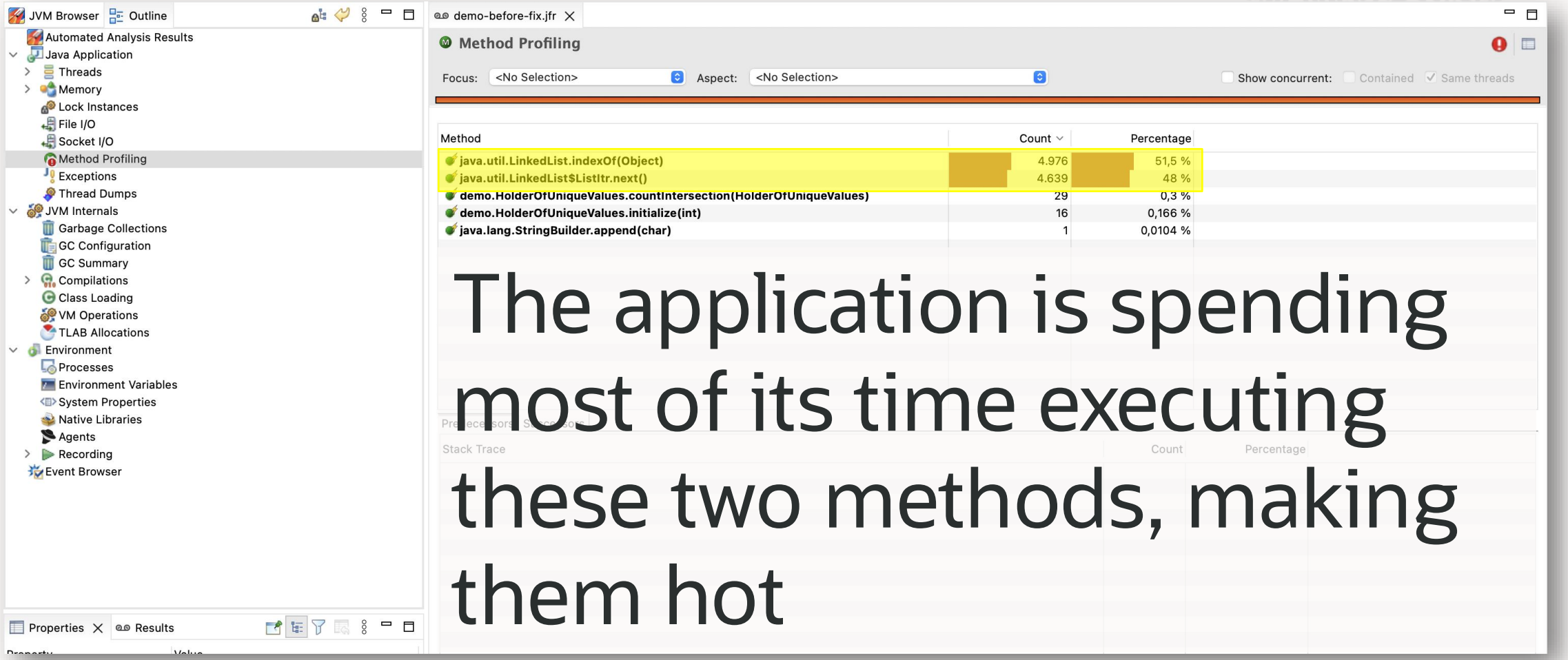
- Hot methods are defined as the methods where the JVM spends most of its time during application execution
- Being "*hot*" might indicate that a specific method may be causing system problems

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Hot Methods

Example



The application is spending most of its time executing these two methods, making them hot

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Based on these observations, where should we look first?



Hot Methods

Example

```
public final class HolderOfUniqueValues {

    private final Collection<Integer> collection;

    public HolderOfUniqueValues() { collection = new LinkedList<>(); }

    public int countIntersection(final HolderOfUniqueValues other) {
        int count = 0;
        for (final Integer i : collection) {
            if (other.collection.contains(i)) {
                count++;
            }
        }
        return count;
    }
}
```

This is where the application is spending most of its time.

Given that we will only contain unique values, is `LinkedList` the right collection to be used?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Hot Methods
Example

- Hot methods point to places where may benefit from performance optimisation
- In this case, by swapping the data structure from `LinkedList` to a `HashSet`, the method execution time was reduced by 99.9%

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

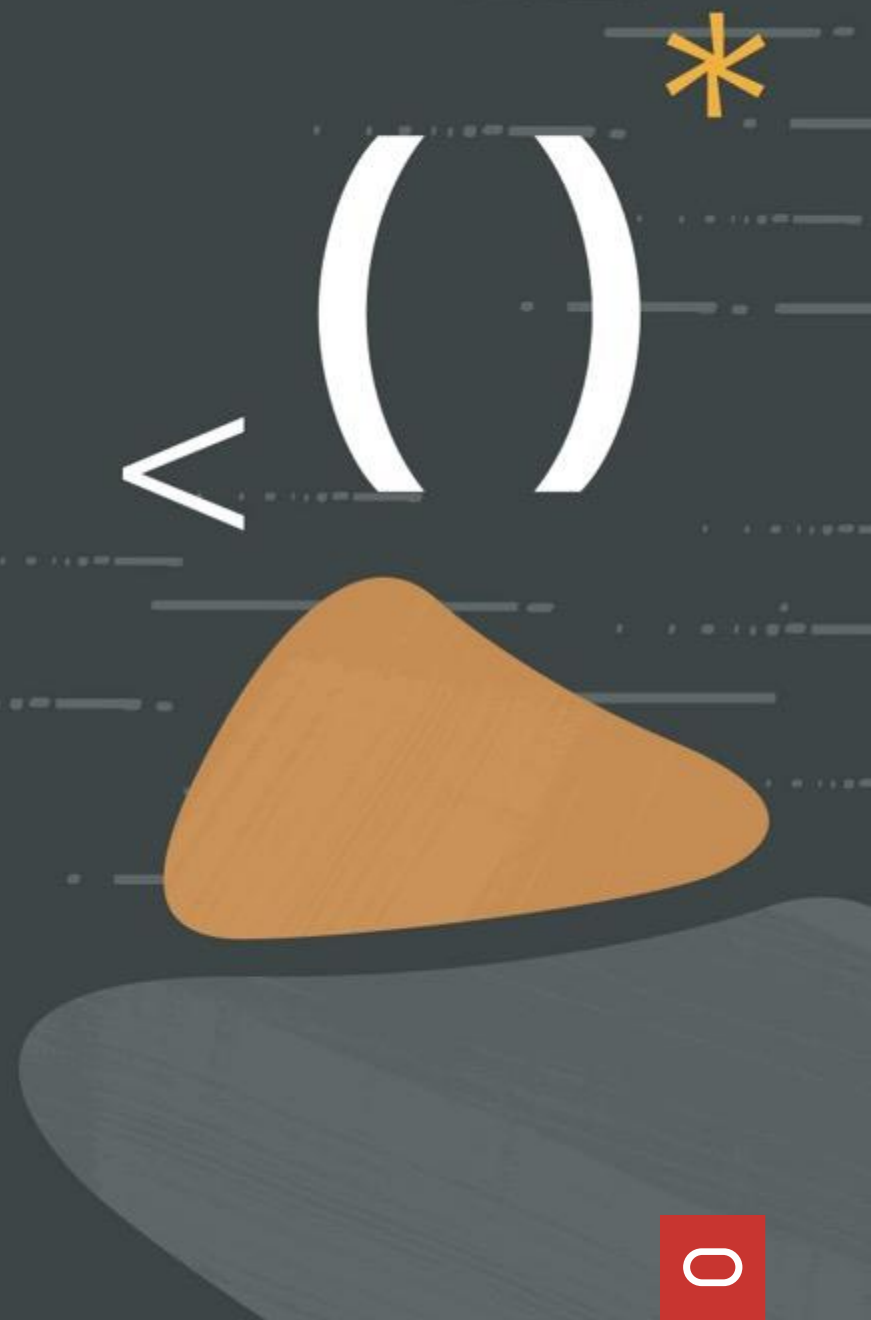
Demo

`hot-methods`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Latency

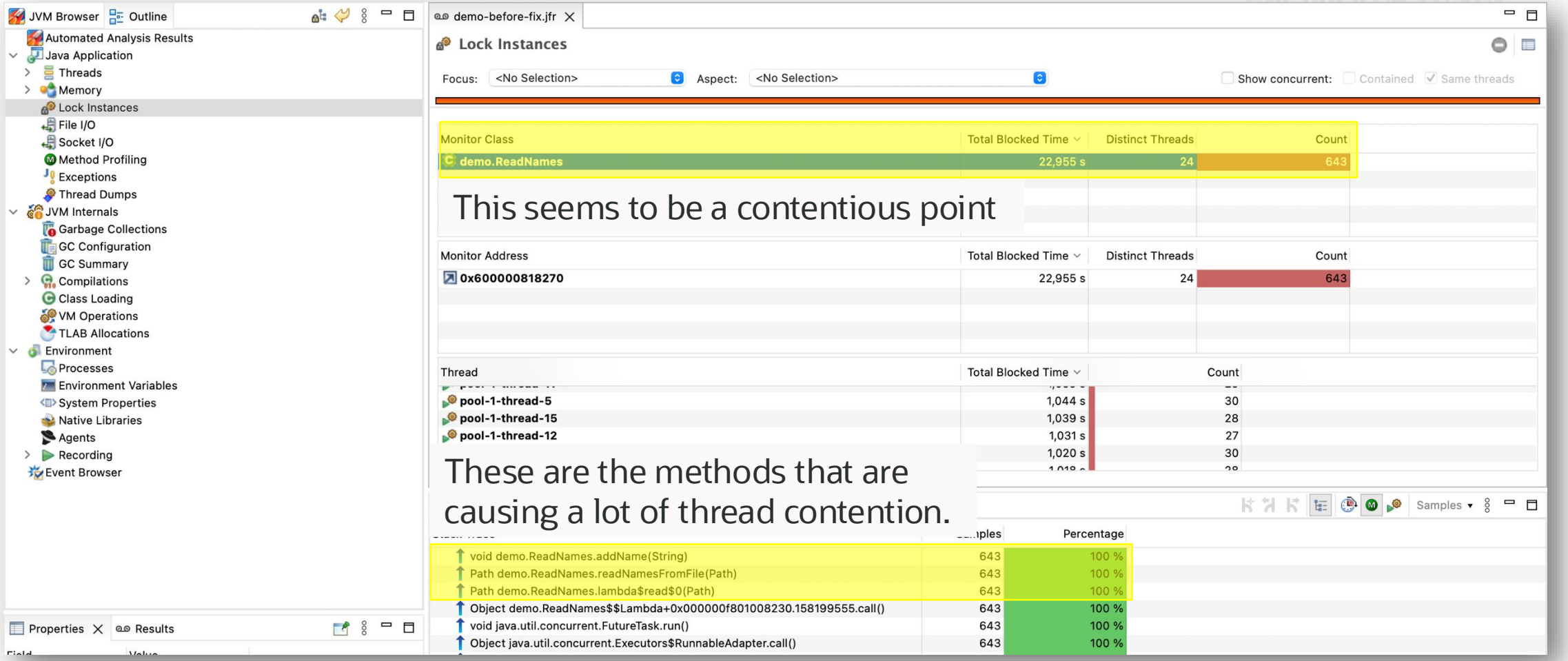


Latency

- Methods may be blocked waiting for something to finish before these can continue
- High lock contention indicates that a specific method is good candidate for investigation

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Latency
Lock Instances



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Based on these observations, where should we look first?



Latency
Lock Instances

```
public ReadNames read(final Path directory) {…}

private Path readNamesFromFile(final Path file) {
    try (BufferedReader reader = Files.newBufferedReader(file)) {
        for (String line; (line = reader.readLine()) != null; ) {
            addName(line);
        }
    } catch (final IOException e) {…}

    return file;
}
```

1 thread per file

This is causing a lot of lock contention

Stack Trace | Flame Graph

Stack Trace	Samples	Percentage
void demo.ReadNames.addName(String)	643	100 %
Path demo.ReadNames.readNamesFromFile(Path)	643	100 %
Path demo.ReadNames.lambda$read$0(Path)	643	100 %
Object demo.ReadNames$$Lambda+0x000000f801008230.158199555.call()	643	100 %
void java.util.concurrent.FutureTask.run()	643	100 %
Object java.util.concurrent.Executors$RunnableAdapter.call()	643	100 %

```
private synchronized void addName(final String name) {
    this.names.add(name);
}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Latency
Lock Instances

- This example reads several files and adds each line into a syncrhonised list
- An alternative approach is to real all files into a separate list and then add the whole list into the syncrhonised list reducing the lock contention

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Latency
Improved version

```
private Path readNamesFromFile(final Path file) {
    final List<String> names = new ArrayList<>();
    try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
        for (String line; (line = reader.readLine()) != null; ) {
            names.add(line);
        }
    } catch (final IOException e) {
        throw new UncheckedIOException("Failed to read names from file: %s".formatted(file), e);
    }

    addNames(names);

    return file;
}

private static List<Path> li

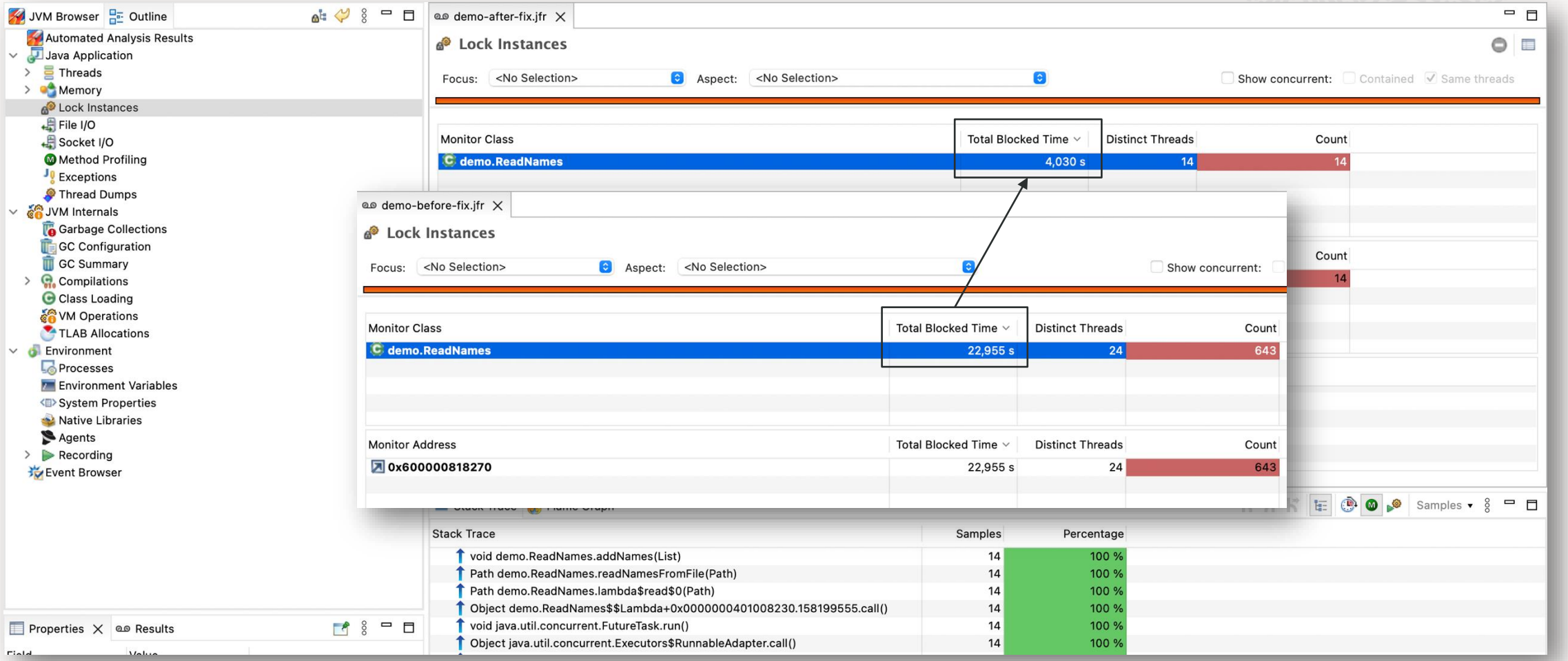
private synchronized void addNames(final List<String> names) {
    this.names.addAll(names);
}
```

Read all file first, and then call the synchronized method once with all the data.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Latency

Lock Instances (after fix)



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Is it necessary for the threads to write to the shared collection?



Latency
Avoid locking

```
public ReadNames read(final Path directory) {
    final List<Path> files = listFiles(directory);

    try (ExecutorService executor = Executors.newFixedThreadPool(files.size())) {
        final CompletionService<List<String>> completionService = new ExecutorCompletionService<>(executor);
        files.forEach(file -> completionService.submit(() -> readNamesFromFile(file)));

        for (int i = 0; i < files.size(); i++) {
            try {
                final List<String> names = completionService.take().get();
                this.names.addAll(names);
            } catch (final InterruptedException e) {
                Thread.currentThread().interrupt();
```

The “reader” threads can read from each file independently and then simply hand back the result to the current thread.

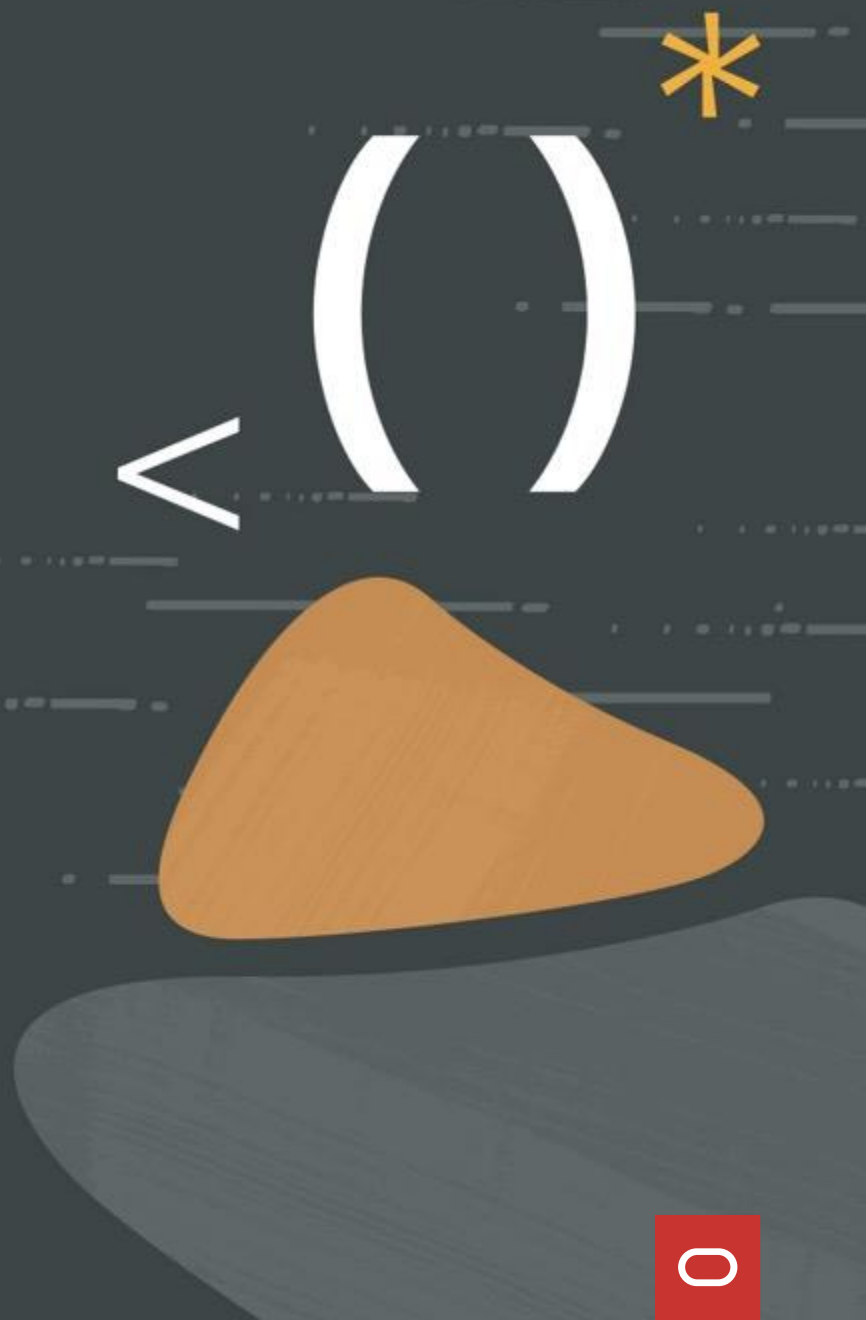
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Demo

```
latency
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Garbage Collection



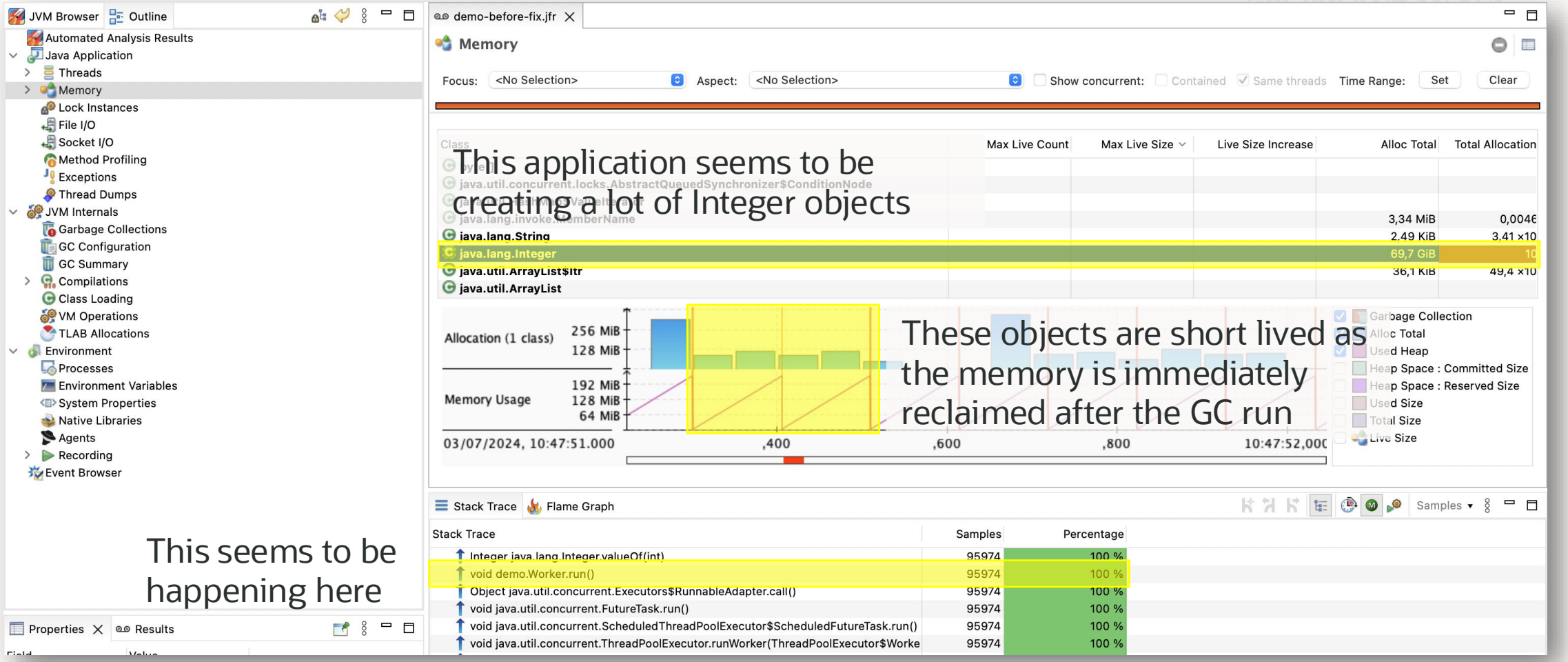
Garbage Collection

- The garbage collector (GC) cleans up unused memory, but this is not free
- The GC needs CPU time, which could be used by the application instead

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

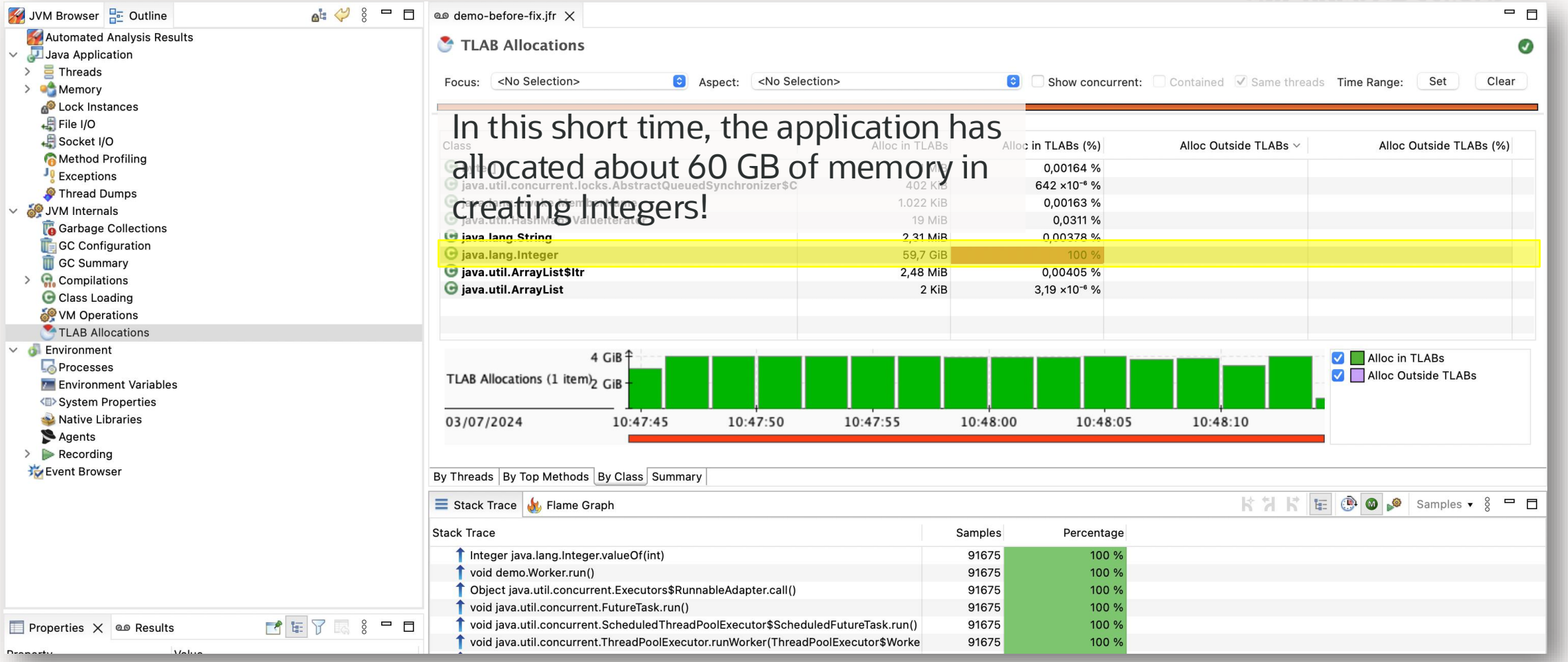
Garbage Collection

Memory



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Garbage Collection
TLAB Allocation



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Based on these observations, where should we look first?



Garbage Collection

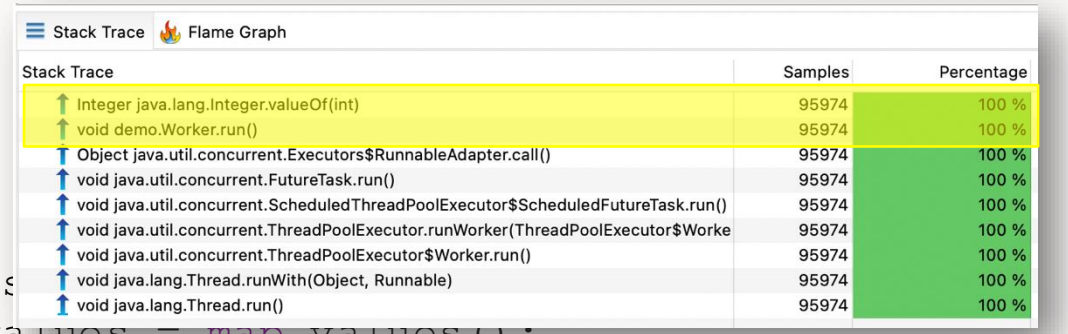
The culprit

```
public final class Worker implements Runnable {

    private final Map<Integer, MapValue>

    @Override
    public void run() {
        while (!Thread.currentThread().is
            final Collection<MapValue> values = map.values();
            for (final MapValue value : values) {
                if (!map.containsKey(value.id())) {
                    throw new IllegalStateE
                }
            }
            Thread.yield();
        }
    }

    private static Map<Integer, Ma
```




```
package demo;

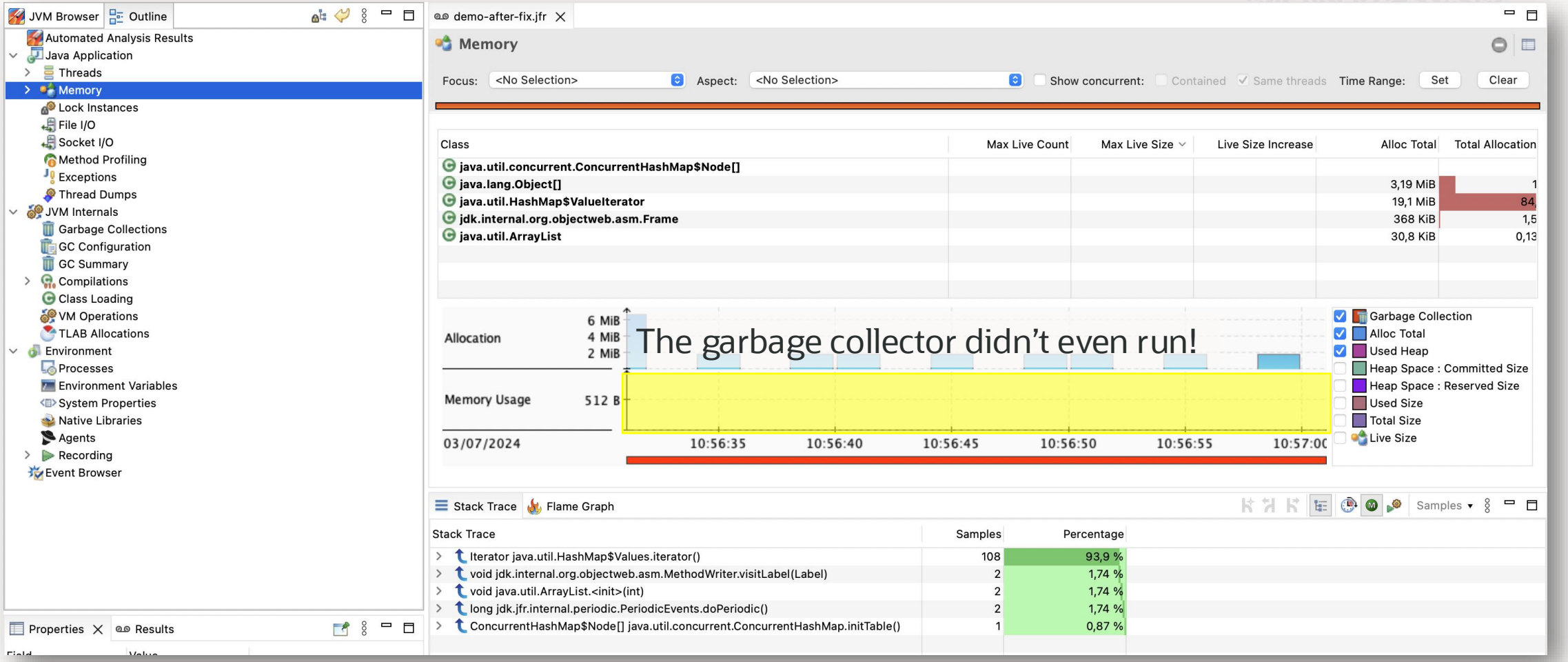
public record MapValue(int id) {}
```

The `id()` returns an `int` which is autoboxed to `Integer` every time

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Garbage Collection

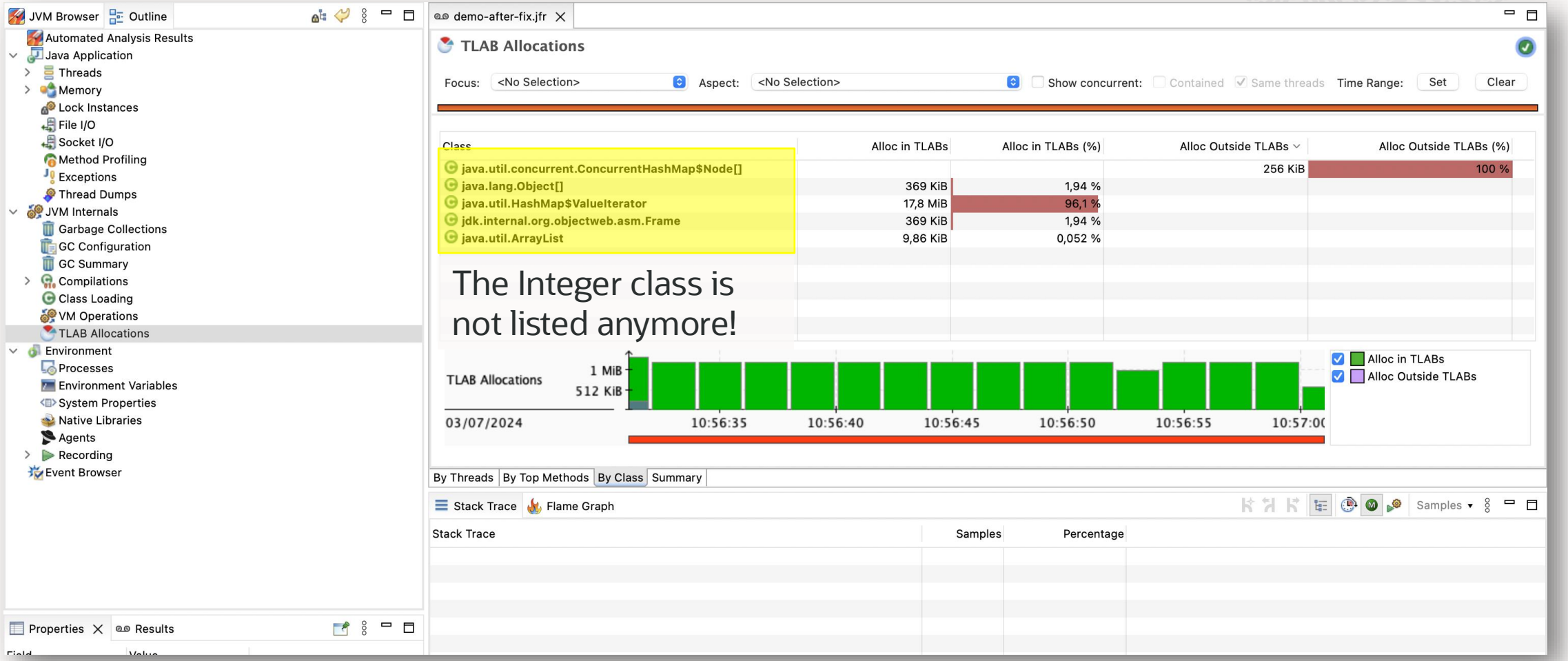
Memory (after fix)



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Garbage Collection

TLAB Allocations (after fix)



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

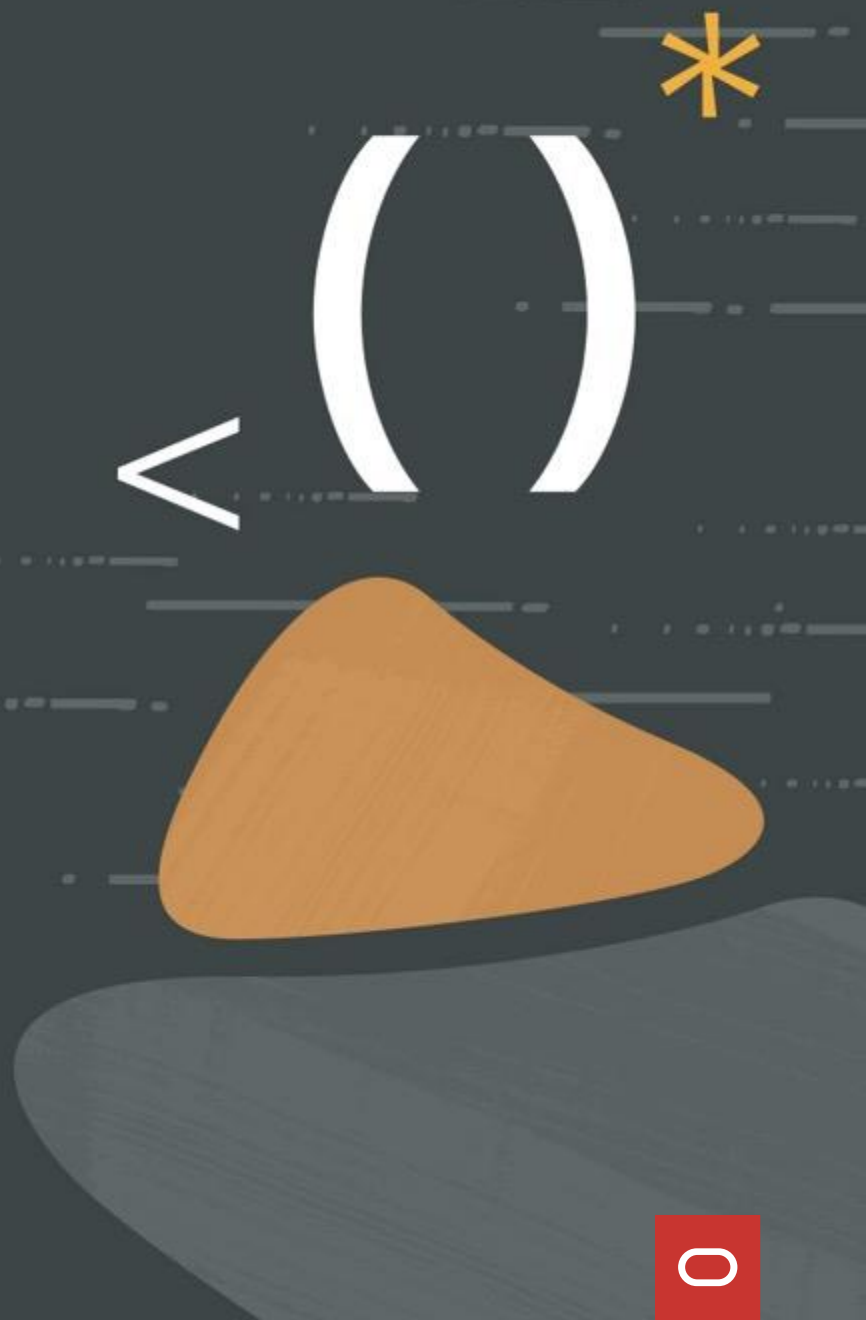
Demo

`garbage-collection`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Memory Leak



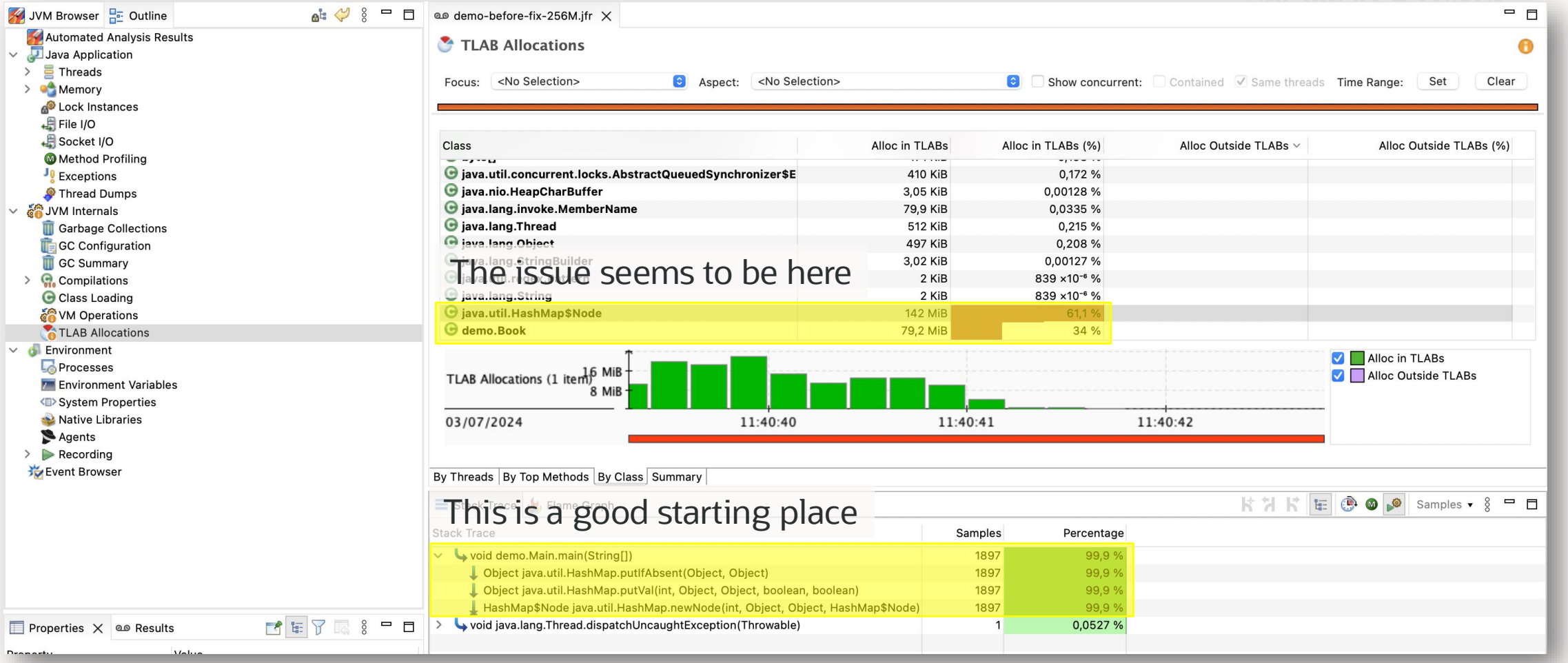
Memory Leak

- There are many ways a Java application can leak memory
- Objects live longer then needed
- Unnecessary objects are created due to a bug

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Memory Leak
TLAB Allocations



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Based on these observations, where should we look first?



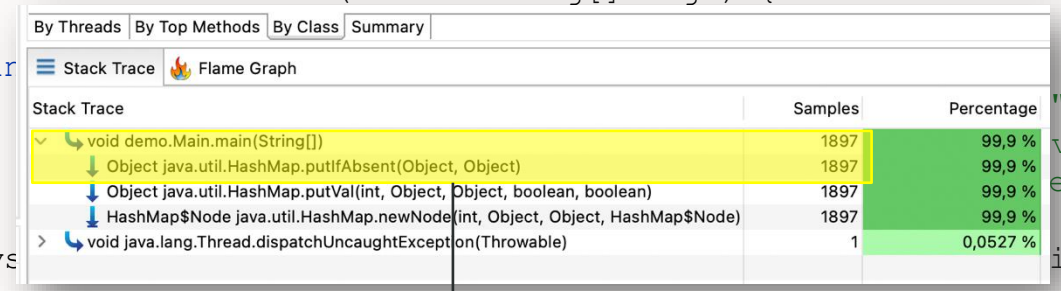
Memory Leak
Simple application

```
public final class Main {

    public static void main(final String[] args) {
        fir                                                    A Beginner's Guide", "Java for Dummies",
                                                              "Spring in Action", "Clean Code",
                                                              velopers", "Test-Driven Java Development",
                                                              entals", "Java - The Complete Reference",
        Sys                                                   ize());

        final Map<Book, Integer> map = new HashMap<>();
        for (int i = 0; i < 10_000_000; i++) {
            for (final String name : set) {
                map.putIfAbsent(new Book(name), 1);
            }
        }

        System.out.println("Number of books in the ma
    }
}
```



The issue seems to be related to this line of code but there is not wrong here that would cause an `OutOfMemoryError`.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Memory Leak

Faulty class

```
public final class Book {

    private final String name;

    public Book(final String name) {
        this.name = name;
    }

    @Override
    public boolean equals(final Object object) {
        return object instanceof final Book other
                && Objects.equals(name, other.name);
    }
}
```

The `hashCode()` method is missing!!

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Memory Leak

Enable compiler warnings (Maven)

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>${maven.compiler.plugin.version}</version>
    <configuration>
        <compilerArgs>
            <arg>-Xlint:all</arg>
            <arg>-Xdiags:verbose</arg>
        </compilerArgs>
    </configuration>
</plugin>
```

```
[INFO] --- compiler:3.11.0:compile (default-compile) @ memory-leak ---
[INFO] Toolchain in maven-compiler-plugin: JDK[/Users/aattard/.sdkman/candidates/java/21.0.2-oracle]
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 2 source files with javac [forked debug release 21] to target/classes
[WARNING] /Users/aattard/Projects/albertattard/workshop-application-performance/examples/memory-leak/src/main/java/demo/Book.java:[5,7] [overrides] Cl
ass Book overrides equals, but neither it nor any superclass overrides hashCode method
[INFO]
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Memory Leak

Enable compiler warnings (Gradle)

```
12  application {
13      mainClass = 'demo.Main'
14  }
15
16  tasks.withType(JavaCompile) {
17      options.compilerArgs += ['-Xlint:all', '-Xdiags:verbose']
18  }
19
```

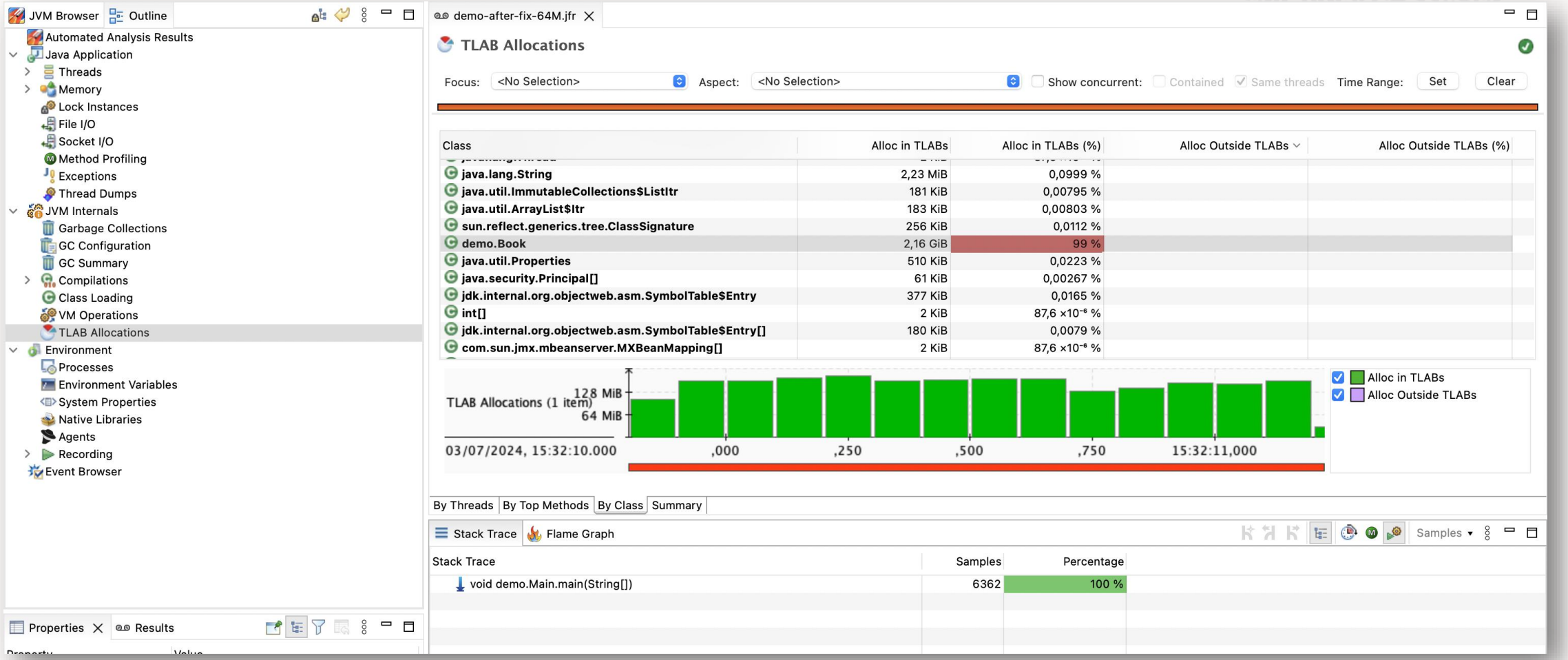
```
→  memory-leak git:(main) ✗ ./gradlew clean assemble

> Task :compileJava
/Users/aattard/Projects/albertattard/workshop-application-performance/examples/memory-leak/src/main/java/d
Book overrides equals, but neither it nor any superclass overrides hashCode method
public class Book {
       ^
1 warning
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Memory Leak

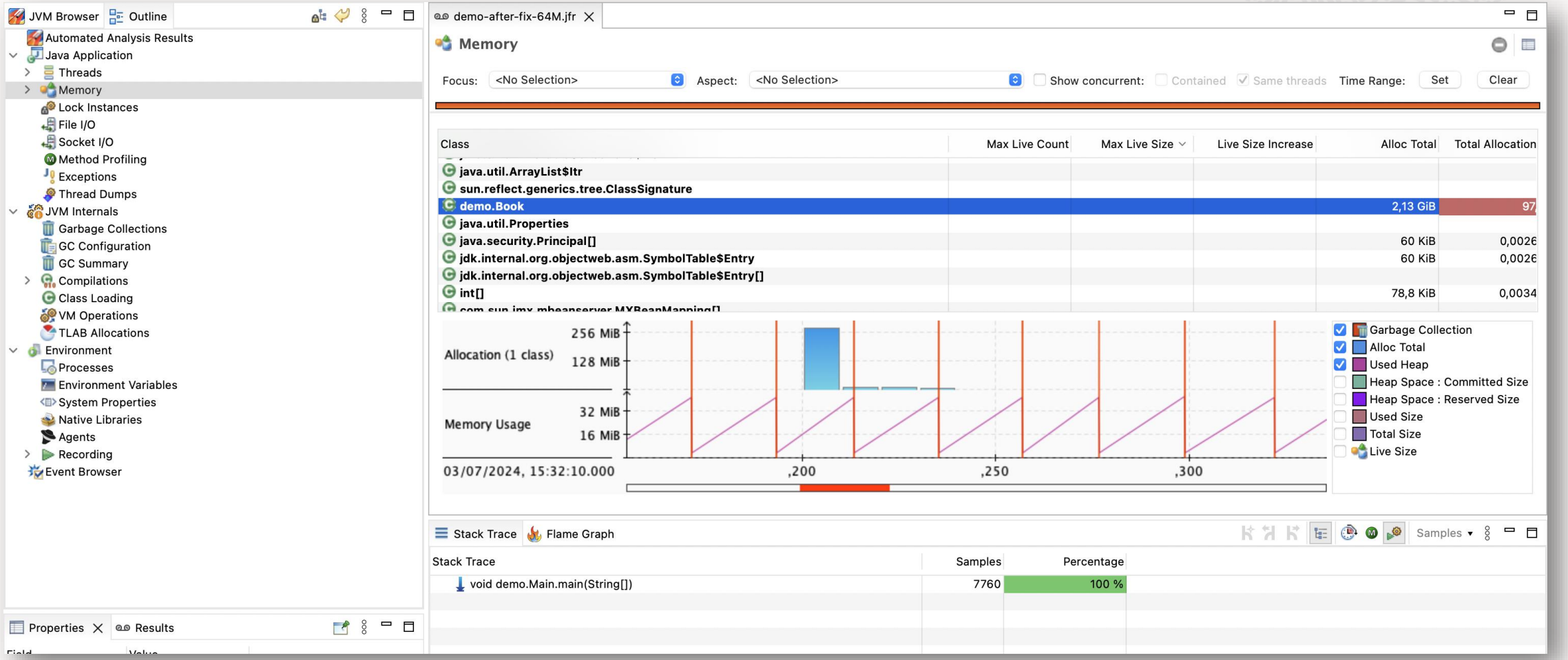
TLAB Allocations (after fix)



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Garbage Collection

Room for further improvement



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Demo

`memory-leak`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Exceptions



Exceptions

How do you track swallowed exceptions?

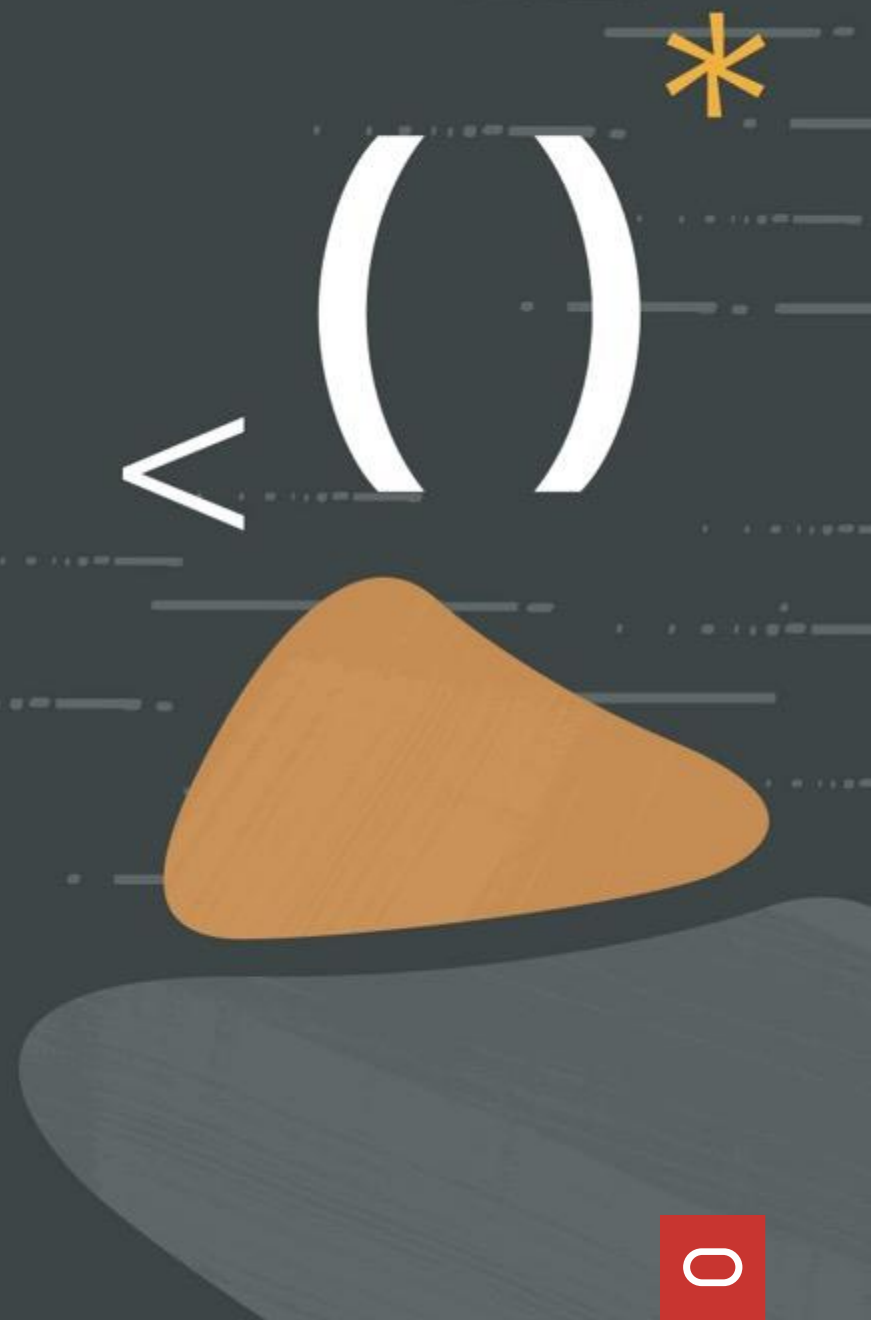
```
@Override
public void run() {
    while (true) {
        try {
            transferMoney();
        } catch (final RuntimeException e) {
            /* Swallow the exception! */
        }

        try {
            TimeUnit.MILLISECONDS.sleep(1);
        } catch (final InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    } }
```

The exception is **not** logged!

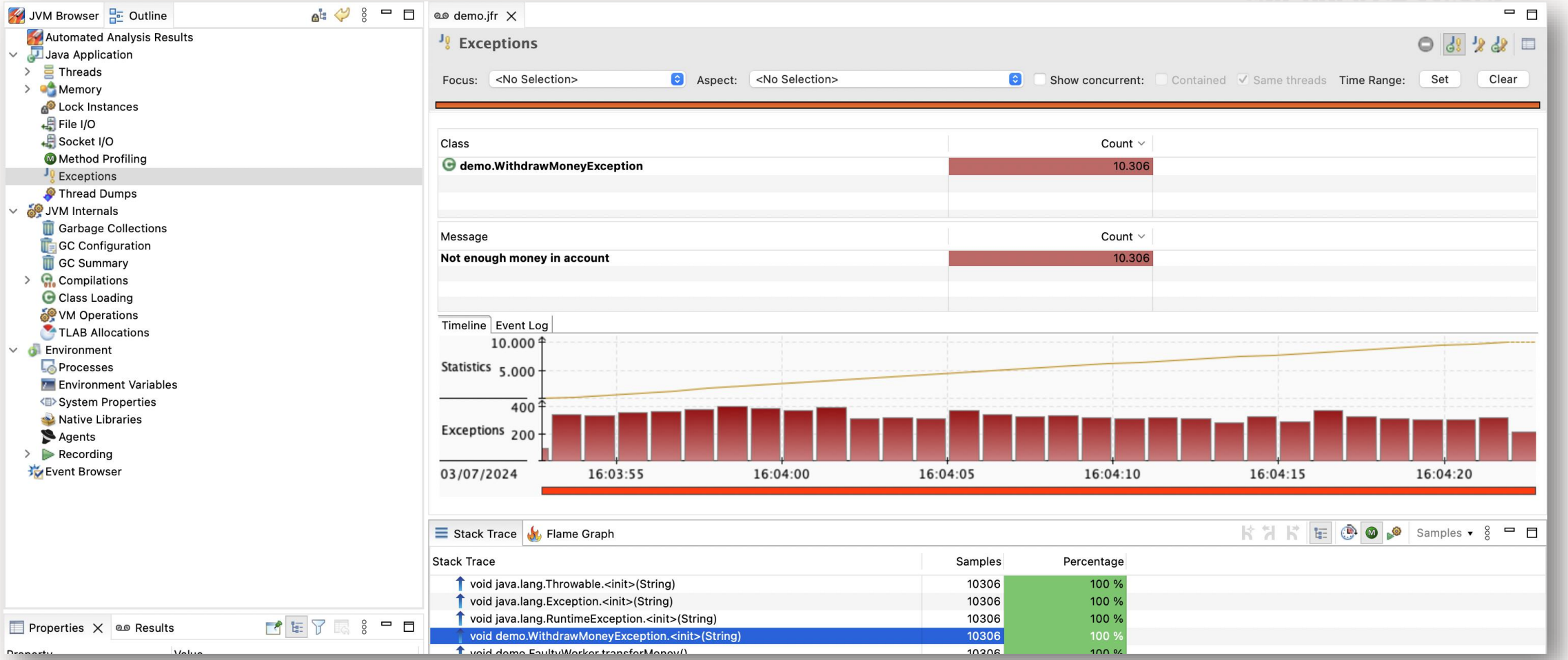
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Can we track such exceptions?



Exceptions

Thrown Exceptions



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

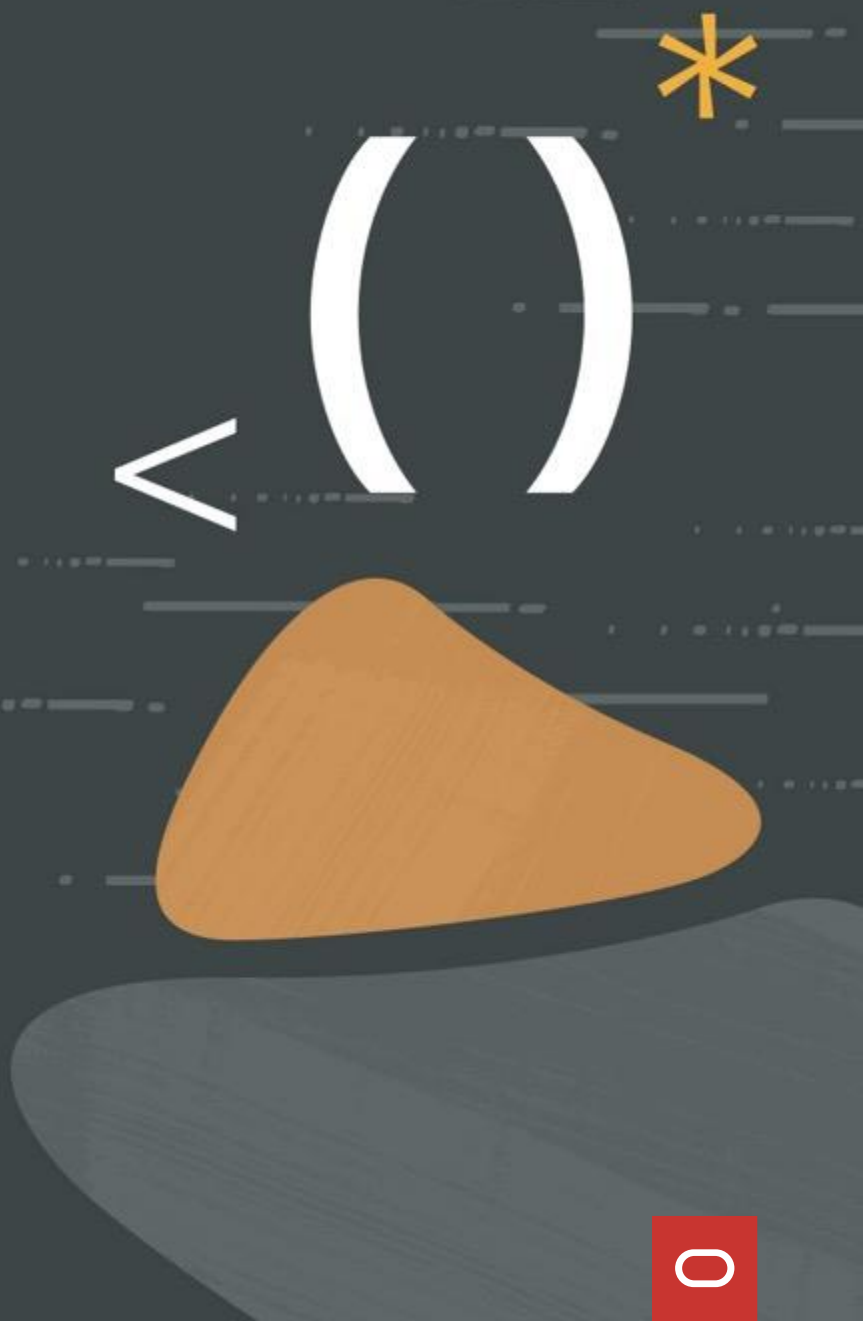
Demo

`exceptions`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

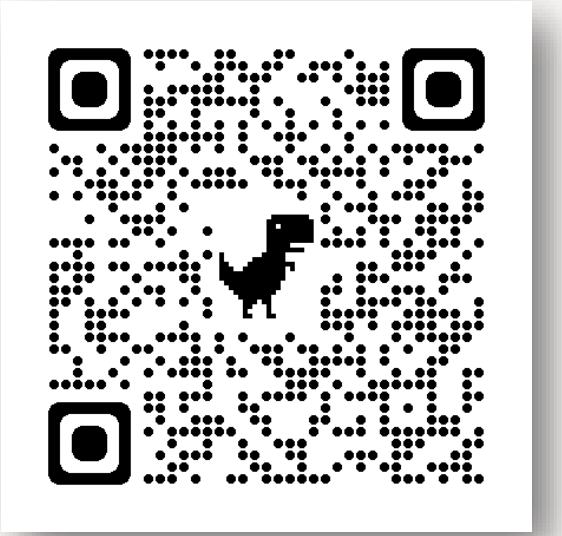


Java Microbenchmark Harness (JMH)



Java Microbenchmark Harness (JMH)

- Java Microbenchmark Harness (JMH) is a framework specifically designed to write, run, and analyse microbenchmarks in Java



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

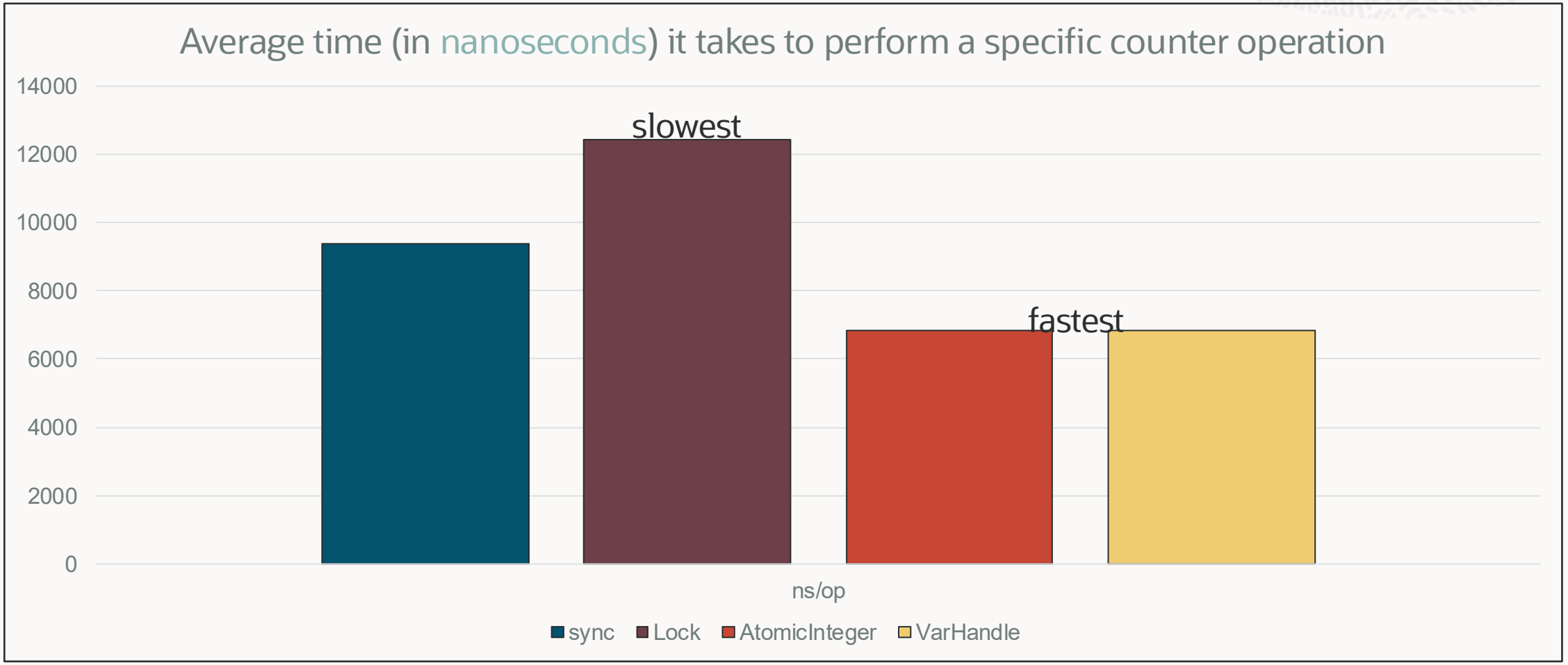
Java Microbenchmark Harness (JMH)

Thread safe, fast, counter

- `synchronised`
- `Lock`
- `AtomicInteger`
- `VarHandle`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java Microbenchmark Harness (JMH)



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

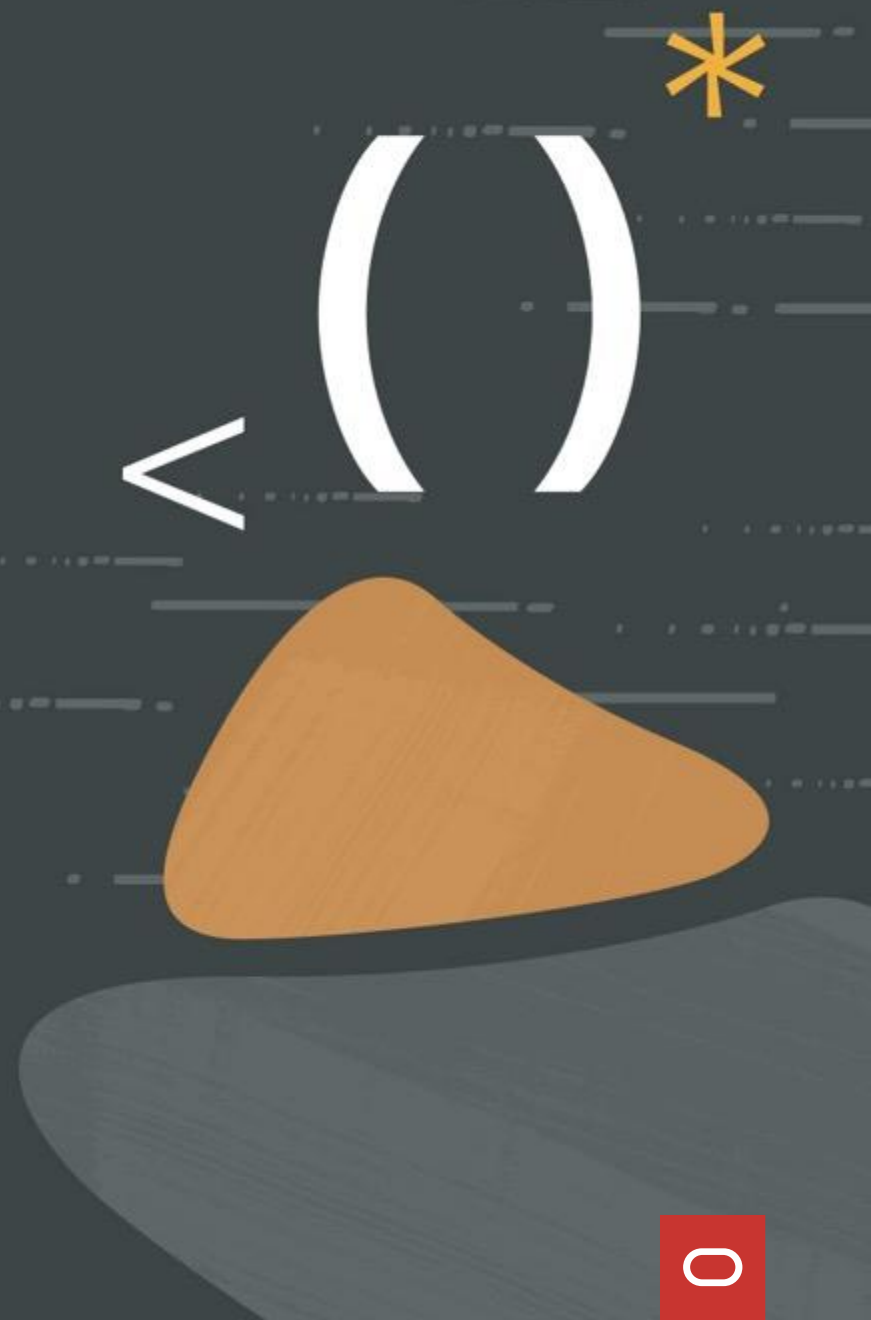
Demo

`jmh-concurrency-benchmark`

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Next



Proof of concept

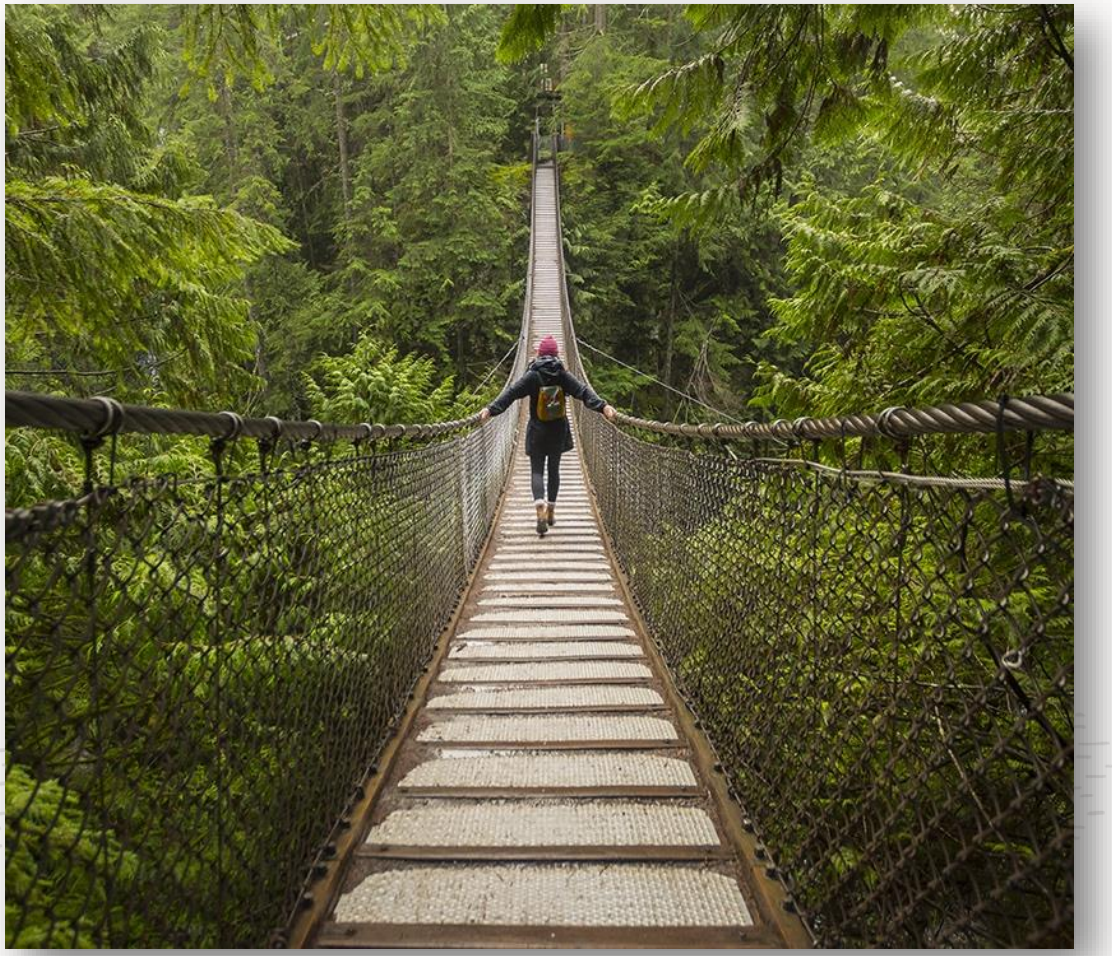
- Identify an application
- Experiment and compare the results



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

How can we help

- Ad hoc conversations
- Workshops
- Knowledge Transfer
- Pairing sessions
- Hands-on



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

We are here to help

You are not alone

Oracle is here to help



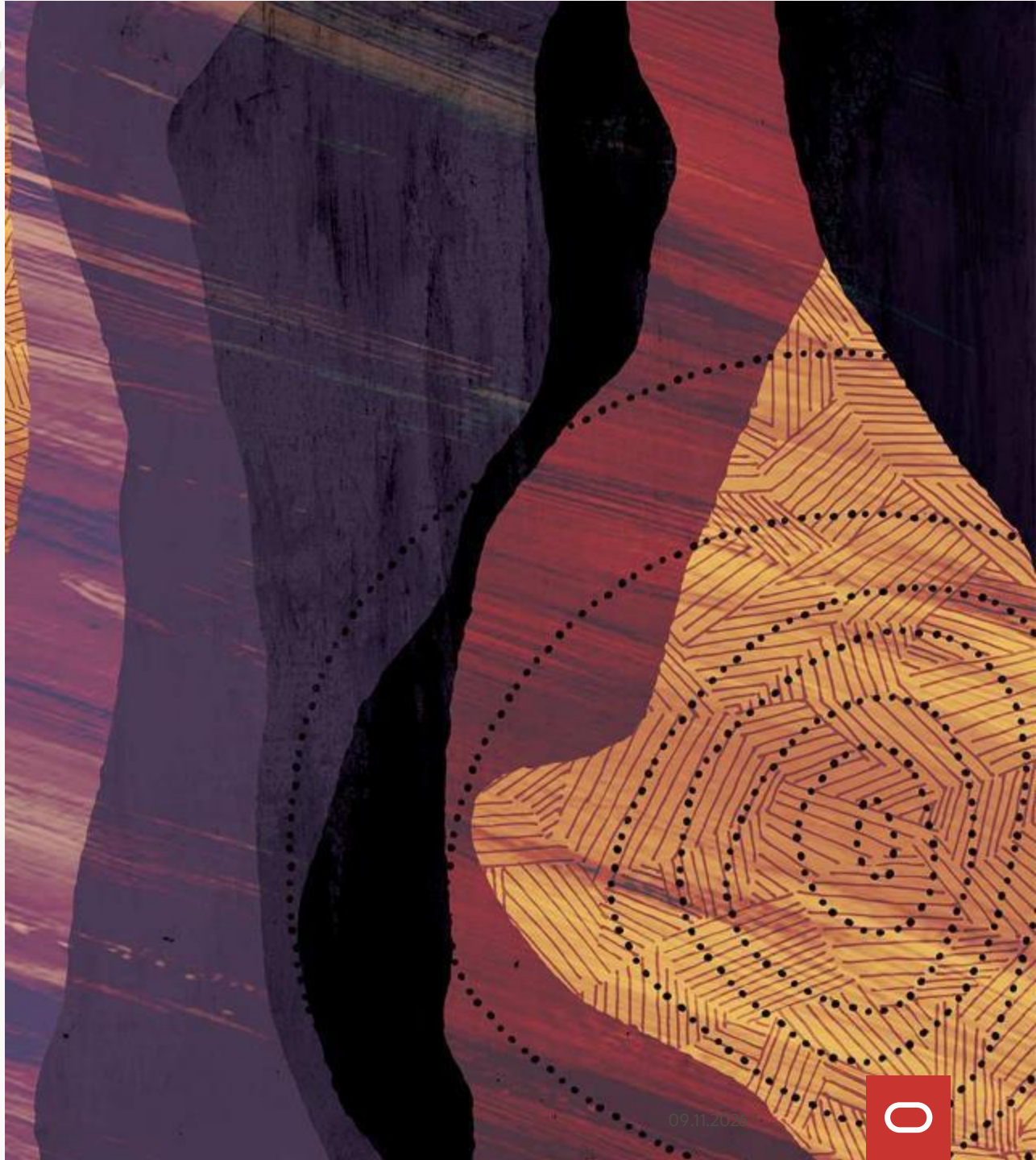
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Thank you

Albert Attard

Principal Java Success Consultant

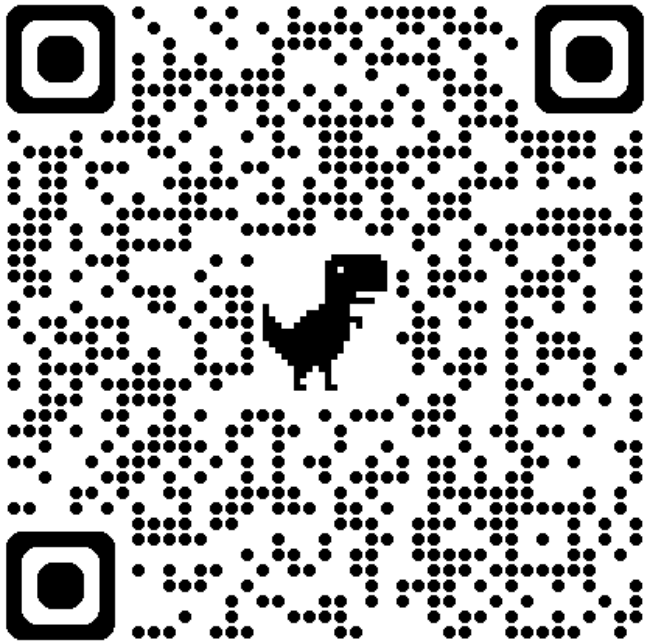
albert.attard@oracle.com



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Your feedback is invaluable to us! Please share your thoughts through our brief survey or by email: albert.attard@oracle.com.

Thank you for helping us improve!



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com


ORACLE



Our mission is to help people see data in new ways, discover insights, unlock endless possibilities.

